

SysML v2, MBSE und ROS2 für Einsteiger

Ein praxisnahes Lehrbuch am Beispiel eines autonomen Indoor-Roboters

Wolfgang Becker

KDP-Softcover-Ausgabe

ISBN: 9798187781393

Imprint: Independently published

© Wolfgang Becker

Alle Rechte vorbehalten.

Diese Ausgabe erscheint im Vertrieb über Amazon KDP (Print on Demand).

Contents

Vorwort

Dieses Lehrbuch richtet sich an Einsteiger ohne Vorkenntnisse in SysML v2, MBSE oder ROS2. Das Ziel ist nicht, alle Spezialfälle der Modellierungstheorie abzudecken. Das Ziel ist, ein funktionierendes Verständnis aufzubauen: Was ist ein Systemmodell? Wie beschreibt man Anforderungen? Wie entsteht daraus eine Architektur? Und wie lässt sich diese Architektur auf ein ROS2-Robotiksystem übertragen?

Als durchgängiges Beispiel dient ein autonomer Indoor-Roboter. Der Roboter soll Hindernisse erkennen, Räume kartieren, Ziele anfahren, Personen erkennen und seinen Akkustand überwachen. Das Beispiel ist klein genug für den Einstieg, aber realistisch genug, um die wichtigsten Ideen von Model-Based Systems Engineering sichtbar zu machen.

Alle SysML-v2-Codebeispiele in diesem Buch wurden mit dem Syside Editor geprüft (Version 0.10.2, geprüft am 07.07.2026). Da sich SysML v2 und die zugehörigen Werkzeuge derzeit noch kontinuierlich weiterentwickeln, kann es bei abweichenden Werkzeugversionen zu kleinen Syntaxunterschieden kommen, etwa bei der Schreibweise von `verify ... by`. Prüfen Sie Codebeispiele im Zweifel gegen Ihre installierte Werkzeugversion.

1 Systems Engineering verstehen

Lernziele

Dieses Kapitel erklärt Ihnen, was ein System ist und warum Systems Engineering für komplexe technische Produkte notwendig ist. Sie verstehen den Unterschied zwischen einer einzelnen Komponente und dem Verhalten eines Systems. Sie lernen, Stakeholder (Personen, Gruppen oder Organisationen mit Interesse am System) zu benennen, eine Systemgrenze zu ziehen und typische Anfangsfehler zu erkennen. Anhand eines autonomen Indoor-Roboters wenden Sie das Denken in Anforderungen, Funktionen, Schnittstellen und Nachweisen praktisch an.

1.1 Einleitung

Viele technische Projekte beginnen mit einer einfachen Idee. Jemand sagt: „Wir bauen einen Roboter, der selbstständig durch ein Gebäude fährt.“ Das klingt zunächst überschaubar. Der Roboter braucht Räder, Motoren, Sensoren, einen Rechner, etwas Software und eine Batterie. Wenn die Bauteile verfügbar sind, müsste das Projekt doch vor allem aus Konstruktion, Programmierung und Tests bestehen.

In der Praxis zeigt sich schnell, dass diese Sicht zu einfach ist. Ein autonomer Indoor-Roboter soll nicht nur fahren. Er soll sicher fahren. Er soll Hindernisse erkennen, Menschen nicht gefährden, Türen und enge Flure bewältigen, seinen Akkustand überwachen, bei Bedarf zur Ladestation zurückkehren und für Nutzer verständlich bedienbar sein. Gleichzeitig soll er bezahlbar bleiben, wartbar sein und in eine reale Umgebung passen. Schon dieses kleine Beispiel verbindet Mechanik, Elektronik, Software, künstliche Intelligenz, Energieversorgung, Sicherheitsüberlegungen, Bedienkonzepte und organisatorische Fragen.

Genau an dieser Stelle beginnt Systems Engineering. Systems Engineering ist eine Arbeitsweise für die Entwicklung komplexer Systeme. Es hilft dabei, vom Problem aus zu denken, statt vorschnell einzelne Lösungen zu bauen. Es fragt:

- Welches Problem soll gelöst werden?
- Wer hat Erwartungen an das System?
- Was gehört zum System und was gehört zur Umgebung?
- Welche Funktionen muss das System erfüllen?
- Welche Anforderungen müssen überprüfbar erfüllt werden?

- Welche Komponenten, Schnittstellen und Abläufe sind dafür notwendig?
- Wie wird nachgewiesen, dass das fertige System wirklich funktioniert?

Für Einsteiger ist wichtig: Systems Engineering ist keine zusätzliche Bürokratie, die ein Projekt künstlich schwerer macht. Gut eingesetzt sorgt es für Klarheit. Es verhindert, dass ein Team zu früh in Details springt und später feststellt, dass wichtige Erwartungen übersehen wurden. In diesem Lehrbuch nutzen wir Systems Engineering als Denkwerkzeug. Wir wollen nicht jede Norm auswendig lernen. Wir wollen verstehen, wie man ein technisches System sauber beschreibt, strukturiert und Schritt für Schritt in eine umsetzbare Architektur überführt.

1.2 Was ist ein System?

Ein System ist eine Menge von Elementen, die zusammenwirken, um einen Zweck zu erfüllen. Diese Definition klingt einfach, enthält aber drei wichtige Aussagen.

Erstens besteht ein System aus mehreren Elementen. Beim Indoor-Roboter sind das zum Beispiel Sensoren, Rechner, Softwarekomponenten, Batterie, Motoren, Räder, Gehäuse, Ladkontakte und Benutzerschnittstellen. Auch Menschen können im erweiterten Sinn Teil einer Systembetrachtung sein, etwa Bediener, Wartungspersonal oder Personen im Fahrbereich.

Zweitens wirken diese Elemente zusammen. Ein einzelner Lidar-Sensor (ein Sensor, der Abstände per Laser misst) macht noch keinen autonomen Roboter. Erst wenn der Sensor Messdaten liefert, diese Daten verarbeitet werden, daraus eine Karte entsteht, die Navigation einen Weg plant und die Motorsteuerung passende Fahrbefehle ausführt, entsteht das gewünschte Verhalten.

Drittens hat ein System einen Zweck. Der Zweck unseres Roboters könnte lauten: ?Der Roboter transportiert kleine Gegenstände selbstständig innerhalb eines Bürogebäudes.? Ein anderer Zweck wäre: ?Der Roboter überwacht nachts Flure und meldet ungewöhnliche Ereignisse.? Beide Systeme könnten äußerlich ähnlich aussehen, hätten aber andere Anforderungen, andere Stakeholder und andere technische Schwerpunkte.

Praxisbeispiel

Stellen Sie sich zwei Roboter mit ähnlicher Hardware vor. Der erste Roboter transportiert Medikamente in einem Krankenhaus. Der zweite Roboter liefert Werkzeuge in einer Werkstatt. Beide brauchen Navigation und Hinderniserkennung. Trotzdem unterscheiden sich die Anforderungen stark. Im Krankenhaus sind Hygiene, leiser Betrieb, Patientensicherheit und zuverlässige Dokumentation besonders wichtig. In der Werkstatt zählen Robustheit, Staubschutz und die sichere Bewegung zwischen Maschinen. Das System wird also nicht nur durch seine Bauteile definiert, sondern durch seinen Zweck und seine Einsatzumgebung.

1.2.1 Elemente, Beziehungen und Verhalten

Ein häufiger Anfängerfehler besteht darin, ein System nur als Liste von Teilen zu betrachten:

- Kamera
- Lidar
- Akku
- Motorcontroller
- Computer
- ROS2-Software (eine Robotik-Software-Plattform, siehe Kapitel 14)

Diese Liste ist nützlich, aber sie reicht nicht aus. Entscheidend sind die Beziehungen zwischen den Teilen. Die Kamera liefert Bilder an eine Erkennungssoftware. Die Erkennungssoftware meldet erkannte Personen an die Navigation. Die Navigation berücksichtigt Hindernisse und erzeugt Sollgeschwindigkeiten. Der Motorcontroller setzt diese Sollgeschwindigkeiten in elektrische Ansteuerung der Motoren um. Der Batteriemonitor meldet einen niedrigen Ladezustand, woraufhin die Missionsplanung eine Rückkehr zur Ladestation auslöst.

Das Systemverhalten entsteht also aus dem Zusammenspiel. Man spricht oft von emergentem Verhalten: Das Verhalten des Gesamtsystems ist mehr als die Summe der Einzelteile. Ein Motor kann drehen, ein Sensor kann messen und ein Rechner kann Programme ausführen. Autonome Navigation entsteht aber erst, wenn diese Fähigkeiten sinnvoll verbunden sind.

1.2.2 Offene und geschlossene Systeme

Viele technische Systeme sind offene Systeme. Sie stehen in ständigem Austausch mit ihrer Umgebung. Der Indoor-Roboter nimmt seine Umgebung über Sensoren wahr, gibt Bewegung in die Umgebung ab, erhält Energie von der Ladestation und kommuniziert möglicherweise mit Nutzern oder einem Gebäudemanagementsystem.

Das ist wichtig, weil offene Systeme nicht vollständig im Labor verstanden werden können. Ein Roboter, der in einer Testhalle gut funktioniert, kann in einem echten Bürogebäude Probleme bekommen: spiegelnde Glasflächen, enge Durchgänge, wechselnde Lichtverhältnisse, Menschen mit Rollkoffern, offene Türen, Teppichkanten oder WLAN-Ausfälle. Systems Engineering zwingt uns deshalb, die Umgebung früh mitzudenken.

1.3 Warum Systems Engineering?

In kleinen Projekten kann man oft direkt loslegen. Wer ein einfaches Python-Skript schreibt, kann Anforderungen, Entwurf, Umsetzung und Test im Kopf behalten.

Bei einem komplexeren Produkt funktioniert das nicht mehr zuverlässig. Zu viele Entscheidungen hängen voneinander ab.

Nehmen wir an, ein Team entscheidet sich für eine stärkere Kamera, damit Personen besser erkannt werden. Diese Änderung klingt lokal. In Wahrheit kann sie viele Folgen haben:

- Die Kamera erzeugt mehr Daten.
- Der Rechner benötigt mehr Rechenleistung.
- Die Software muss größere Bilder schneller verarbeiten.
- Der Stromverbrauch steigt.
- Der Akku hält kürzer.
- Das Gehäuse braucht eventuell eine andere Befestigung.
- Die Wärmeentwicklung kann zunehmen.
- Der Preis des Roboters steigt.

Systems Engineering hilft, solche Abhängigkeiten sichtbar zu machen. Es betrachtet nicht nur eine Komponente, sondern das Gesamtsystem und seinen Lebenszyklus: Idee, Anforderungen, Entwurf, Umsetzung, Integration, Test, Betrieb, Wartung und spätere Weiterentwicklung.

1.3.1 Komplexität beherrschbar machen

Komplexität bedeutet nicht nur, dass ein System viele Teile hat. Komplex wird ein System vor allem durch viele Wechselwirkungen, unklare Erwartungen und Änderungen während der Entwicklung. Ein Roboterprojekt wird komplex, weil Hardware, Software und Umgebung gleichzeitig eine Rolle spielen.

Ein Beispiel: Die Navigation soll den kürzesten Weg wählen. Die Sicherheitsanforderung sagt aber, dass der Roboter in der Nähe von Personen langsam fahren muss. Die Batterieanforderung sagt, dass der Roboter rechtzeitig zur Ladestation zurückkehren muss. Die Nutzeranforderung sagt, dass Lieferungen nicht zu lange dauern dürfen. Diese Anforderungen können miteinander in Konflikt geraten. Systems Engineering macht solche Konflikte sichtbar, bevor sie erst beim Test des fertigen Roboters auffallen.

1.3.2 Kommunikation verbessern

Technische Projekte sind Teamarbeit. Mechanik, Elektronik, Software, Test, Projektleitung, Einkauf, Betrieb und Kundenseite verwenden oft unterschiedliche Begriffe. Was für einen Softwareentwickler ein ?Node? ist, ist für einen Systemingenieur vielleicht eine Funktionseinheit. Was ein Nutzer als ?schnell? empfindet, muss für die Entwicklung in messbare Anforderungen übersetzt werden.

Systems Engineering schafft gemeinsame Beschreibungen. Diese Beschreibungen können Texte, Tabellen, Diagramme und Modelle sein. Später nutzen wir dafür SysML. Im ersten Schritt geht es aber um die Denkweise: Wir wollen Begriffe klären, Zusammenhänge sichtbar machen und Entscheidungen nachvollziehbar dokumentieren.

1.3.3 Fehler früher finden

Je später ein Fehler entdeckt wird, desto teurer wird seine Korrektur. Wenn erst kurz vor der Auslieferung auffällt, dass die Batterie für die geplante Fahrzeit zu klein ist, muss vielleicht das Gehäuse geändert, der Schwerpunkt neu bewertet und die Ladeelektronik angepasst werden. Wenn diese Frage früh durch Anforderungen und einfache Berechnungen betrachtet wird, ist die Anpassung deutlich einfacher.

Systems Engineering verlagert Denken und Prüfen nach vorne. Das bedeutet nicht, dass man alles im Voraus perfekt wissen muss. Es bedeutet, dass man Annahmen bewusst macht und gezielt überprüft.

1.4 Eine kurze Geschichte des Systems Engineering

Systems Engineering entstand nicht aus einer einzelnen Erfindung. Es entwickelte sich aus praktischen Problemen großer technischer Vorhaben. Immer dann, wenn viele Fachgebiete zusammenarbeiten mussten, wurde eine übergreifende Sicht notwendig.

1.4.1 Frühe technische Großsysteme

Schon der Bau von Eisenbahnen, Telefonnetzen, Kraftwerken und Flugzeugen erforderte systemisches Denken. Ein Eisenbahnnetz besteht nicht nur aus Schienen und Lokomotiven. Es benötigt Bahnhöfe, Signale, Fahrpläne, Wartung, Energieversorgung, Personal, Sicherheitsregeln und Betriebsprozesse. Wird ein Teil geändert, wirkt sich das auf viele andere Teile aus.

Der Begriff Systems Engineering wurde besonders im 20. Jahrhundert wichtig. Große Telekommunikations-, Luftfahrt- und Raumfahrtprojekte machten deutlich, dass klassische Einzeldisziplinen allein nicht ausreichen. Man brauchte Methoden, um Anforderungen, Architektur, Schnittstellen, Tests und Projektentscheidungen zusammenzuführen.

1.4.2 Luft- und Raumfahrt als Treiber

In der Luft- und Raumfahrt sind die Folgen von Fehlern besonders gravierend. Ein Satellit, eine Rakete oder ein Flugzeug besteht aus vielen Teilsystemen: Struktur, Antrieb, Energie, Kommunikation, Steuerung, Sensorik, Software und Bodeninfrastruktur. Diese Teilsysteme müssen unter extremen Bedingungen zusammen funktionieren.

Solche Projekte prägten viele Grundideen des Systems Engineering: klare Anforderungen, kontrollierte Schnittstellen, Konfigurationsmanagement (der geordnete Umgang mit Änderungen und Versionen), Verifikation (wird weiter unten in diesem Kapitel genauer erklärt), Validierung (die Prüfung, ob das System tatsächlich das richtige Problem

löst) und Nachvollziehbarkeit. Der zentrale Gedanke lautet: Man entwickelt nicht nur Bauteile, sondern ein Gesamtsystem mit nachweisbarer Erfüllung seiner Aufgaben.

1.4.3 Software und vernetzte Systeme

Später wurde Software immer wichtiger. Moderne Systeme sind oft **cyber-physische Systeme**: physische Komponenten werden durch Software gesteuert und sind über Netzwerke verbunden. Ein autonomer Roboter ist ein typisches Beispiel. Er bewegt sich physisch durch die Welt, entscheidet aber mit Hilfe von Software, Sensorfusion und Algorithmen.

Damit verschiebt sich auch die Herausforderung. Mechanische Teile ändern sich meist langsamer als Software. Software kann häufig aktualisiert werden, führt aber auch neue Risiken ein: Versionskonflikte, unerwartete Laufzeitfehler, unvollständige Tests oder schwer nachvollziehbare Entscheidungen von Machine-Learning-Komponenten. Systems Engineering muss deshalb heute Hardware, Software, Daten, **Künstliche Intelligenz (KI)**-Modelle und Betriebsprozesse gemeinsam betrachten.

1.5 Systemdenken

Systemdenken bedeutet, Zusammenhänge wichtiger zu nehmen als isolierte Einzelteile. Es ist eine Denkweise, die fragt: ?Was passiert im Gesamtsystem, wenn ich an einer Stelle etwas ändere??

1.5.1 Vom Bauteil zur Wirkung

Einsteiger denken oft zuerst in Bauteilen. Das ist verständlich, denn Bauteile kann man kaufen, anfassen und einbauen. Systems Engineering beginnt jedoch früher. Es fragt zunächst nach Wirkungen und Funktionen.

Für den Indoor-Roboter könnte eine gewünschte Wirkung lauten: ?Der Roboter vermeidet Zusammenstöße mit Menschen und Gegenständen.? Daraus ergeben sich Funktionen:

- Umgebung erfassen
- Hindernisse erkennen
- Abstand bewerten
- Geschwindigkeit anpassen
- bei Gefahr stoppen
- Nutzer über Störung informieren

Eine Funktion beschreibt dabei eine Fähigkeit, zum Beispiel ?Hindernisse erkennen?, während eine Komponente das konkrete Bauteil ist, das diese Fähigkeit umsetzt, zum Beispiel ein Lidar-Sensor; dieselbe Funktion kann durch unterschiedliche Komponenten

erfüllt werden. Erst danach entscheiden wir, welche Komponenten diese Funktionen erfüllen. Vielleicht braucht der Roboter einen Lidar-Sensor. Vielleicht reicht eine Tiefenkamera nicht aus. Vielleicht ist eine Kombination aus mehreren Sensoren sinnvoll. Die Lösung folgt aus dem Problem, nicht umgekehrt. Abbildung 1.1 zeigt diese Wirkkette vom Sensor bis zu den Motoren sowie, parallel dazu, den Regelkreis der Energieversorgung.

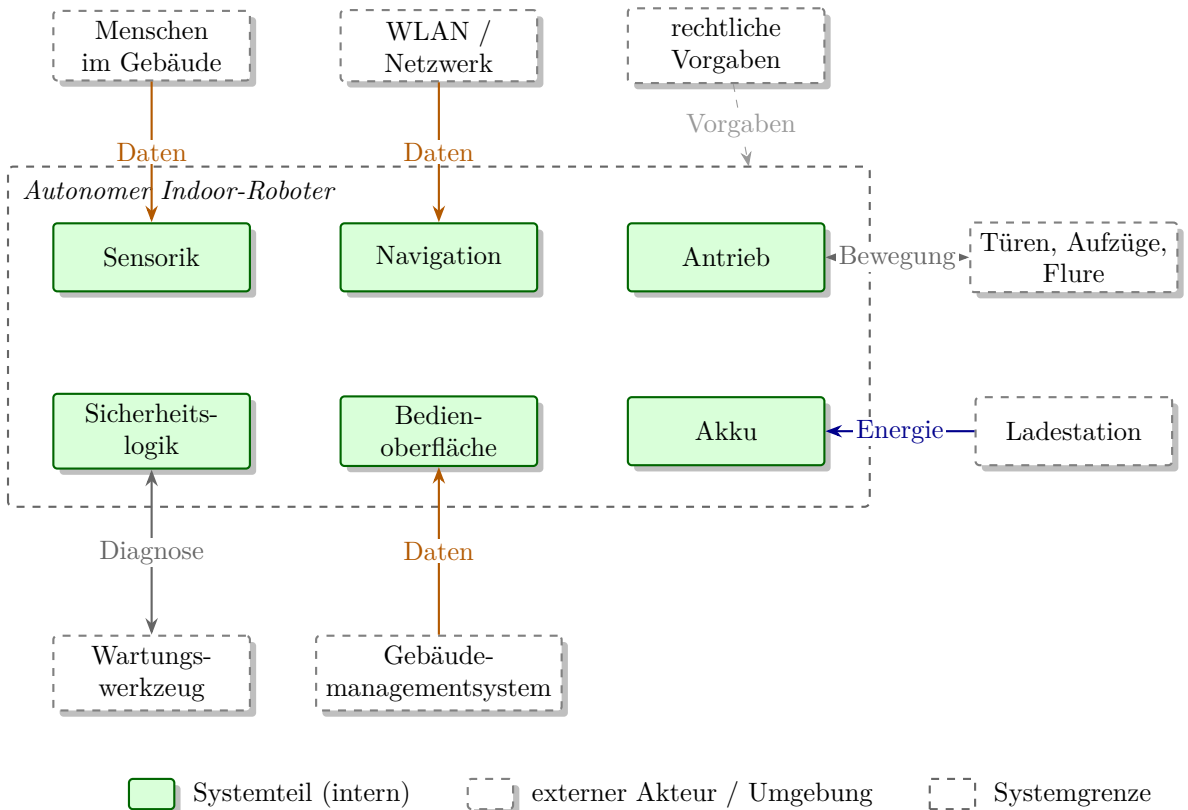


Figure 1.1: Wirkkette der Hinderniserkennung von der Sensorik bis zu den Motoren, darunter der Regelkreis der Energieversorgung.

1.5.2 Rückkopplungen

Viele Systeme enthalten Rückkopplungen. Der Roboter misst seine Umgebung, entscheidet eine Bewegung, bewegt sich, misst erneut und korrigiert seine Entscheidung. Navigation ist ohne Rückkopplung kaum möglich.

Auch organisatorisch gibt es Rückkopplungen. Testergebnisse führen zu neuen Anforderungen oder zu geänderten Entwürfen. Nutzerfeedback verändert Bedienkonzepte. Lieferprobleme beeinflussen die Auswahl von Komponenten. Systems Engineering akzeptiert diese Schleifen. Es versucht nicht, Entwicklung als starre Gerade zu behandeln.

1.5.3 Trade-offs

Ein Trade-off ist ein Zielkonflikt. Man verbessert eine Eigenschaft und verschlechtert dabei möglicherweise eine andere. Beim Roboter gibt es viele typische Trade-offs:

- Höhere Geschwindigkeit verkürzt Lieferzeiten, kann aber Sicherheitsabstände vergrößern.
- Mehr Sensoren verbessern die Wahrnehmung, erhöhen aber Kosten und Energieverbrauch.
- Ein größerer Akku verlängert die Laufzeit, erhöht aber Gewicht und Ladezeit.
- Ein komplexeres KI-Modell erkennt Personen besser, benötigt aber mehr Rechenleistung.
- Eine sehr einfache Bedienung kann technische Diagnosemöglichkeiten verstecken.

Gute Entwicklung bedeutet nicht, jede Eigenschaft zu maximieren. Gute Entwicklung bedeutet, begründete Kompromisse zu finden. Systems Engineering liefert dafür die Struktur: Anforderungen, Randbedingungen, Alternativen, Bewertungen und Entscheidungen werden nachvollziehbar gemacht.

Hinweis

Wenn ein Team sagt: „Wir wollen maximale Reichweite, maximale Geschwindigkeit, niedrigste Kosten, höchste Sicherheit und kleinste Bauform“, dann ist das noch keine realistische Zieldefinition. Systems Engineering hilft, solche Wünsche in priorisierte, überprüfbare Anforderungen zu übersetzen.

1.6 Anforderungen und Lösungen unterscheiden

Ein sehr wichtiger Grundsatz lautet: Anforderungen beschreiben, was erreicht werden soll. Lösungen beschreiben, wie es erreicht werden soll. Diese Unterscheidung klingt einfach, wird aber in Projekten ständig vermischt.

Eine schlechte Anforderung wäre:

Der Roboter muss einen Lidar-Sensor vom Typ X verwenden.

Das kann in manchen Fällen berechtigt sein, etwa wenn ein bestimmter Sensor aus Plattformgründen vorgeschrieben ist. Oft ist es aber bereits eine Lösungsentscheidung. Eine bessere Anforderung könnte lauten:

Der Roboter muss Hindernisse ab einer Breite von 10 cm in einem Abstand von mindestens 1,5 m erkennen.

Diese Formulierung beschreibt die benötigte Fähigkeit. Ob diese Fähigkeit mit Lidar, Kamera, Ultraschall oder Sensorfusion erreicht wird, bleibt zunächst offen. Dadurch kann das Team technische Alternativen vergleichen.

1.6.1 Warum diese Trennung wichtig ist

Wenn Anforderungen zu früh konkrete Lösungen enthalten, wird der Lösungsraum unnötig eingeschränkt. Das Team baut dann vielleicht eine Komponente ein, obwohl eine bessere, günstigere oder zuverlässigere Lösung möglich wäre.

Umgekehrt dürfen Anforderungen nicht so allgemein bleiben, dass sie nicht prüfbar sind. ?Der Roboter soll zuverlässig sein? ist ein Wunsch, aber noch keine testbare Anforderung. Besser wäre:

Der Roboter muss in einem Büroflur mit normaler Beleuchtung 95 von 100 Zielfahrten ohne manuellen Eingriff abschließen.

Diese Anforderung ist noch nicht perfekt, aber sie ist deutlich überprüfbarer. Man kann Testbedingungen definieren, Fahrten zählen und Ergebnisse auswerten.

1.7 Stakeholderanalyse

Stakeholder sind Personen, Gruppen oder Organisationen, die ein Interesse am System haben oder von ihm betroffen sind. Eine Stakeholderanalyse beantwortet die Frage: ?Für wen bauen wir das System eigentlich, und wessen Erwartungen müssen wir berücksichtigen??

Beim autonomen Indoor-Roboter gibt es mehr Stakeholder, als man zunächst denkt:

- Nutzer, die den Roboter beauftragen
- Personen, die sich im Gebäude bewegen
- Betreiber des Gebäudes
- Wartungspersonal
- Entwicklerinnen und Entwickler
- Sicherheitsbeauftragte
- Datenschutzverantwortliche
- Einkauf oder Management
- Reinigungspersonal
- IT-Administration

Jede dieser Gruppen sieht das System anders. Nutzer fragen: ?Ist der Roboter einfach zu bedienen?? Wartungspersonal fragt: ?Wie erkenne ich Fehler?? IT-Administratoren fragen: ?Wie wird der Roboter ins Netzwerk eingebunden?? Datenschutzverantwortliche fragen: ?Was passiert mit Kamerabildern?? Sicherheitsbeauftragte fragen: ?Wie wird verhindert, dass jemand verletzt wird??

1.7.1 Stakeholderbedürfnisse erfassen

Stakeholder äußern oft Bedürfnisse, keine fertigen Anforderungen. Ein Nutzer sagt vielleicht: ?Der Roboter soll nicht nerven.? Das ist Alltagssprachlich verständlich, aber technisch unklar. Dahinter können mehrere Anforderungen stecken:

- Der Roboter soll leise fahren.
- Der Roboter soll nicht mitten im Flur stehen bleiben.
- Der Roboter soll seine Absicht anzeigen.
- Der Roboter soll Menschen nicht dicht auffahren.
- Der Roboter soll keine komplizierte Bedienoberfläche haben.

Die Aufgabe im Systems Engineering besteht darin, solche Aussagen ernst zu nehmen und in überprüfbare Anforderungen zu übersetzen. Dabei darf der ursprüngliche Sinn nicht verloren gehen.

1.7.2 Konflikte zwischen Stakeholdern

Stakeholder haben nicht immer dieselben Ziele. Das Management möchte niedrige Kosten. Das Entwicklungsteam möchte genug Zeit für saubere Tests. Nutzer möchten schnelle Lieferungen. Sicherheitsverantwortliche möchten vorsichtige Fahrstrategien. Datenschutzverantwortliche möchten möglichst wenige personenbezogene Daten erfassen, während das KI-Team gerne viele Kameradaten für Personenerkennung nutzen würde.

Solche Konflikte sind normal. Gefährlich werden sie erst, wenn sie unausgesprochen bleiben. Systems Engineering macht Konflikte sichtbar und dokumentiert Entscheidungen. Nicht jede Gruppe bekommt alles, was sie möchte. Aber gute Entscheidungen sind begründet und nachvollziehbar.

Praxisbeispiel

Für unseren Indoor-Roboter könnte die Stakeholderanalyse ergeben, dass die Kamera zwar für Personenerkennung nützlich ist, aber Datenschutzfragen auslöst. Eine mögliche Systementscheidung wäre, Bilder nur lokal auf dem Roboter zu verarbeiten und nicht dauerhaft zu speichern. Daraus entstehen neue Anforderungen an Softwarearchitektur, Speicherverwaltung und Tests.

Abbildung 1.2 fasst die zehn wichtigsten Stakeholder des Indoor-Roboters noch einmal zusammen und zeigt jeweils ein typisches Interesse am System.

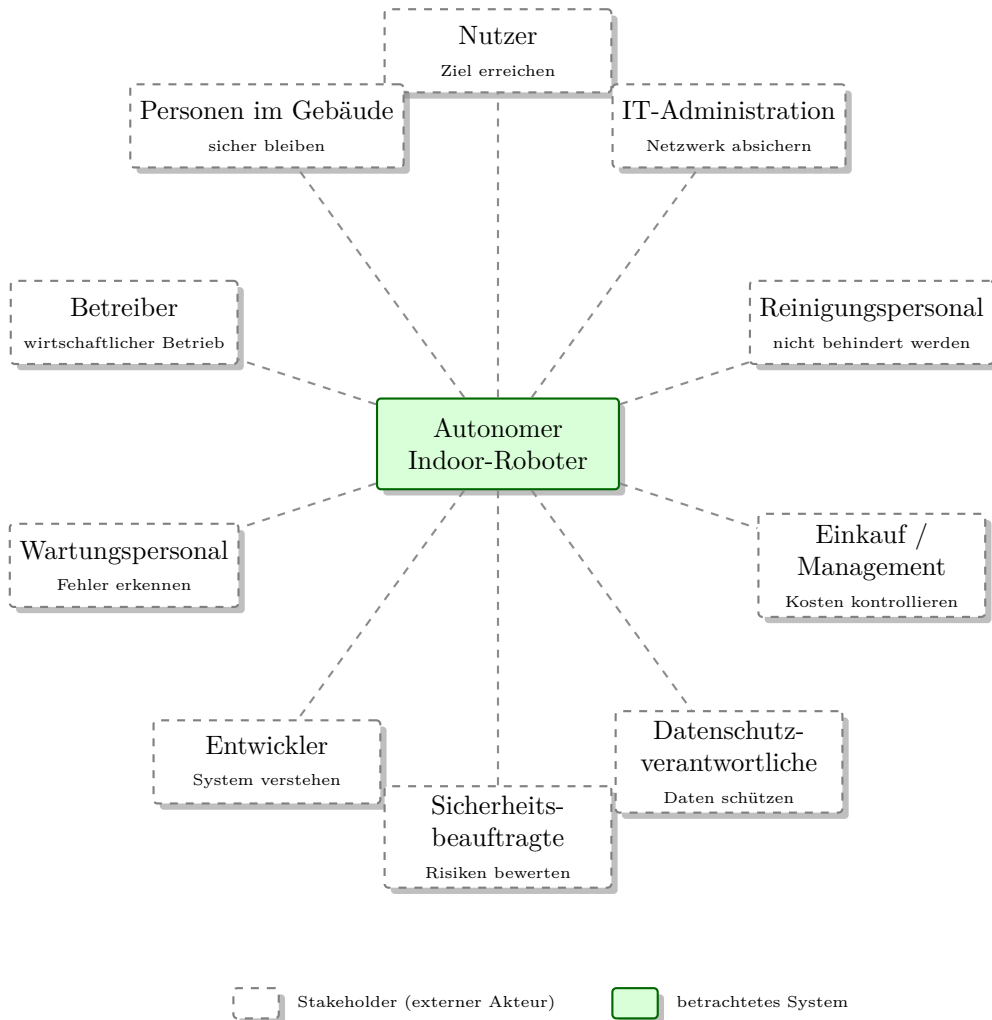


Figure 1.2: Stakeholder des autonomen Indoor-Roboters mit jeweils einem typischen Interesse am System.

1.8 Systemgrenzen

Die Systemgrenze legt fest, was zum betrachteten System gehört und was als Umgebung betrachtet wird. Diese Entscheidung ist nicht nur eine Zeichnung. Sie beeinflusst Anforderungen, Verantwortlichkeiten, Schnittstellen und Tests.

Beim Indoor-Roboter stellt sich zum Beispiel die Frage: Gehört die Ladestation zum System? Es gibt zwei mögliche Antworten.

1.8.1 Variante 1: Ladestation gehört zum System

Wenn die Ladestation Teil des Systems ist, entwickelt oder spezifiziert das Projektteam auch die Ladestation. Dann gehören Ladeelektronik, mechanische Andockhilfe, Statusanzeige

und Kommunikation zwischen Roboter und Station zur Systemarchitektur. Der Vorteil ist Kontrolle. Das Team kann Roboter und Ladestation optimal aufeinander abstimmen. Der Nachteil ist mehr Entwicklungsaufwand.

1.8.2 Variante 2: Ladestation ist ein externes System

Wenn die Ladestation als externes System betrachtet wird, muss der Roboter nur mit ihr kompatibel sein. Dann wird die Schnittstelle wichtig: Spannung, Strom, mechanische Kontakte, Andockposition, Sicherheitsabschaltung und eventuell Kommunikationsprotokoll. Der Vorteil ist weniger eigener Entwicklungsumfang. Der Nachteil ist Abhängigkeit von einem Fremdsystem.

Beide Entscheidungen können richtig sein. Wichtig ist, dass sie bewusst getroffen werden. Eine unklare Systemgrenze führt später fast immer zu Missverständnissen. Wer ist verantwortlich, wenn das Andocken nicht zuverlässig funktioniert? Das Robotikteam? Der Lieferant der Ladestation? Die Gebäudetechnik? Genau solche Fragen klärt die Systemgrenze.

1.8.3 Kontext des Systems

Außerhalb der Systemgrenze liegt der Systemkontext. Dazu gehören alle externen Akteure und Systeme, mit denen der Roboter interagiert:

- Menschen im Gebäude
- Türen, Aufzüge, Flure und Räume
- WLAN oder anderes Netzwerk
- Ladestation, falls extern betrachtet
- Bedienoberfläche auf Tablet oder PC
- Wartungswerkzeug
- Gebäudemanagementsystem
- rechtliche und organisatorische Vorgaben

Der Kontext ist wichtig, weil viele Anforderungen an den Grenzen entstehen. Schnittstellen sind häufig Fehlerquellen. Ein Roboter kann intern sauber funktionieren und trotzdem im Betrieb scheitern, wenn die Kommunikation mit der Umgebung unklar ist. [Abbildung 1.3](#) veranschaulicht die Systemgrenze und den Kontext mit den wichtigsten Energie-, Daten- und Bewegungsflüssen.

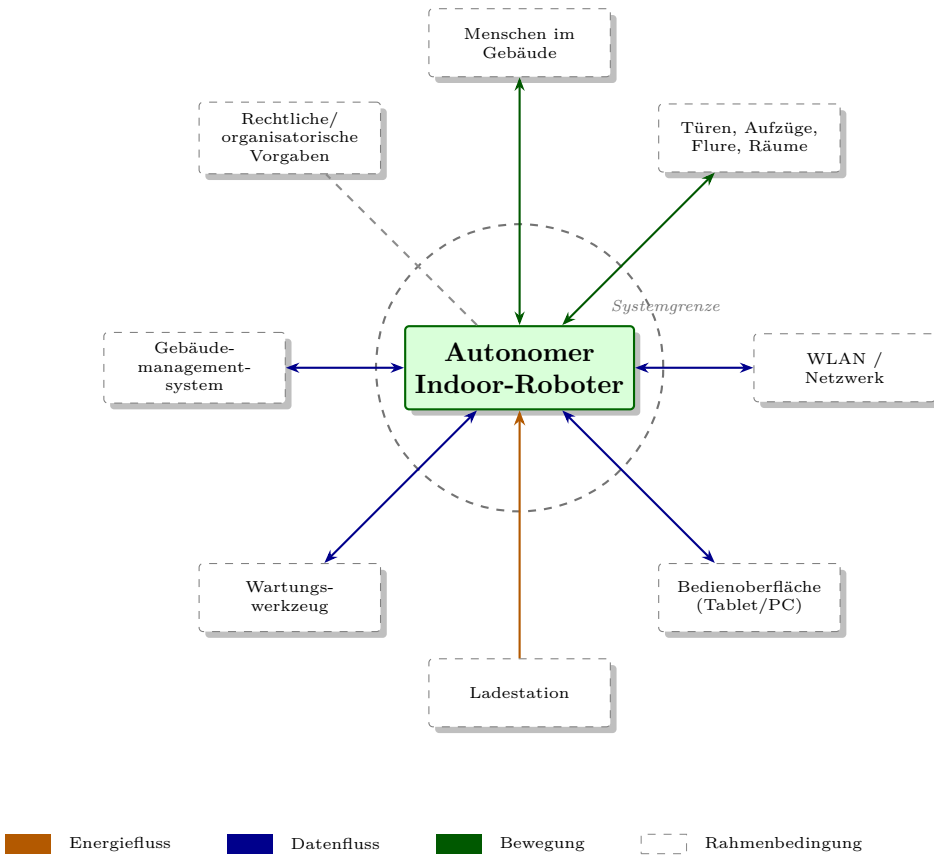


Figure 1.3: Systemgrenze und Kontext des autonomen Indoor-Roboters mit Energie-, Daten- und Bewegungsflüssen über die Grenze hinweg.

1.9 Fallstudie: Autonomer Indoor-Roboter

Wir verwenden in diesem Buch durchgehend einen autonomen Indoor-Roboter als Praxisbeispiel. In diesem Kapitel betrachten wir ihn noch ohne SysML-Details. Wir beschreiben ihn so, wie ein Systems-Engineering-Team am Anfang eines Projekts darüber sprechen könnte.

1.9.1 Ausgangssituation

Ein mittelgroßes Bürogebäude möchte interne Botengänge automatisieren. Kleine Gegenstände wie Dokumentenmappen, Werkzeuge, Laborproben oder Ersatzteile sollen zwischen Räumen transportiert werden. Bisher erledigen Mitarbeitende diese Wege manuell. Das kostet Zeit und unterbricht Arbeitsabläufe.

Der Roboter soll sich selbstständig im Gebäude bewegen. Er soll Ziele anfahren, Hindernisse erkennen, Personen ausweichen, seinen Akkustand überwachen und bei niedrigem Akkustand automatisch zur Ladestation zurückkehren. Bediener sollen einen Transportauftrag einfach starten können.

1.9.2 Erste Systembeschreibung

Eine erste Systembeschreibung könnte lauten:

Das System ?Autonomer Indoor-Roboter? transportiert kleine Gegenstände innerhalb eines definierten Gebäudebereichs. Es navigiert autonom auf Basis einer Karte, erkennt Hindernisse und Personen, vermeidet Kollisionen, überwacht seinen Energiezustand und kehrt bei Bedarf selbstständig zur Ladestation zurück.

Diese Beschreibung ist noch grob, aber sie enthält bereits wichtige Punkte: Zweck, Einsatzbereich, Hauptfunktionen und Sicherheitsaspekt.

1.9.3 Hauptfunktionen

Aus der Beschreibung lassen sich erste Hauptfunktionen ableiten:

- Auftrag entgegennehmen
- aktuelle Position bestimmen
- Umgebung erfassen
- Karte nutzen oder aktualisieren
- Route planen
- Bewegung ausführen
- Hindernisse erkennen und vermeiden
- Personen erkennen und Abstand halten
- Akkustand überwachen
- Ladestation anfahren
- Status an Nutzer melden
- Fehler erkennen und sicher reagieren

Diese Funktionsliste ist noch keine Architektur. Sie sagt noch nicht, welche ROS2-Nodes es geben wird oder welche Sensoren verwendet werden. Sie ist aber ein guter Zwischenschritt zwischen Problem und Lösung.

1.9.4 Erste Randbedingungen

Neben Funktionen gibt es Randbedingungen. Randbedingungen beschreiben Grenzen, unter denen das System arbeiten muss. Beispiele:

- Der Roboter wird ausschließlich in Innenräumen eingesetzt.
- Der Roboter fährt auf ebenen Böden.
- Der Roboter muss durch normale Bürotüren passen.
- Der Roboter darf Fluchtwege nicht dauerhaft blockieren.
- Der Roboter soll während der Arbeitszeit nicht störend laut sein.
- Der Roboter muss ohne Spezialwissen bedienbar sein.
- Die Verarbeitung von Kameradaten muss datenschutzfreundlich erfolgen.

Randbedingungen sind für Einsteiger manchmal weniger spannend als Funktionen. Trotzdem sind sie entscheidend. Viele Fehlentwicklungen entstehen, weil Randbedingungen zu spät erkannt werden.

1.9.5 Vom Wunsch zur Anforderung

Im nächsten Schritt werden Wünsche in Anforderungen übersetzt. Die folgende Tabelle zeigt einfache Beispiele.

Wunsch oder Aussage	Mögliche Anforderung
Der Roboter soll sicher sein.	Der Roboter muss bei einem Hindernis im Abstand von 0,5 m seine Geschwindigkeit reduzieren und bei 0,2 m stoppen.
Der Akku soll lange halten.	Der Roboter muss mindestens 4 Stunden typischen Bürotransportbetrieb ohne Nachladen durchführen.
Die Bedienung soll einfach sein.	Ein Standardauftrag muss mit höchstens drei Benutzereingaben gestartet werden können.
Der Roboter soll Menschen erkennen.	Der Roboter muss Personen im Fahrbereich unter normalen Lichtbedingungen erkennen und seine Route entsprechend anpassen.
Der Roboter soll nicht verloren gehen.	Der Roboter muss seine Position innerhalb des kartierten Bereichs mit ausreichender Genauigkeit für die Navigation bestimmen.

Die Zahlen in dieser Tabelle sind nur Beispielwerte. In einem echten Projekt müssten sie begründet, abgestimmt und getestet werden. Für den Einstieg zeigen sie aber den Unterschied zwischen einem Wunsch und einer überprüfbaren Anforderung.

1.10 Typische Projektfehler

Systems Engineering wird besonders verständlich, wenn man typische Fehler betrachtet. Viele Projekte scheitern nicht an fehlendem Talent, sondern an unklaren Grundlagen.

1.10.1 Fehler 1: Zu früh in Komponenten denken

Ein Team entscheidet am ersten Tag: ?Wir nehmen diesen Lidar, diese Kamera und diesen Rechner.? Das fühlt sich konkret an. Vielleicht entstehen sogar schnell erste Prototypen. Das Problem: Es ist noch nicht klar, welche Fähigkeiten wirklich benötigt werden. Später stellt sich heraus, dass die Kamera in dunklen Fluren schlecht funktioniert oder der Rechner für das KI-Modell zu schwach ist.

Besser ist es, zuerst Funktionen, Anforderungen und Einsatzbedingungen zu beschreiben. Komponenten werden dann gezielt ausgewählt.

1.10.2 Fehler 2: Unklare Begriffe

Ein Stakeholder sagt ?autonom?, ein anderer versteht darunter ?ohne Fernsteuerung?, ein dritter versteht ?ohne menschliche Aufsicht?. Solche Begriffe müssen geklärt werden. Für unser Projekt könnte ?autonom? bedeuten: Der Roboter fährt innerhalb eines kartierten Bereichs selbstständig zu einem Ziel, ohne dass ein Mensch die Fahrt kontinuierlich steuert.

1.10.3 Fehler 3: Anforderungen sind nicht testbar

Formulierungen wie ?schnell?, ?zuverlässig?, ?intuitiv? oder ?robust? sind als erste Wünsche in Ordnung. Als Anforderungen reichen sie nicht aus. Testbare Anforderungen brauchen beobachtbare Kriterien. Sonst kann am Ende niemand eindeutig sagen, ob sie erfüllt wurden.

1.10.4 Fehler 4: Schnittstellen werden unterschätzt

Viele Probleme entstehen an Schnittstellen: zwischen Sensor und Software, zwischen Navigation und Motorsteuerung, zwischen Roboter und Ladestation, zwischen Bedienoberfläche und Missionsplanung. Wenn Schnittstellen nicht klar beschrieben sind, integrieren Teams ihre Teile erst spät und entdecken dann teure Fehler.

1.10.5 Fehler 5: Die Umgebung wird idealisiert

Ein Roboter fährt in der Simulation perfekt. Im echten Gebäude rutscht er auf glattem Boden, erkennt Glaswände schlecht, verliert WLAN oder wird durch Menschen blockiert. Systems Engineering erinnert daran, dass das System im realen Kontext funktionieren muss. Deshalb gehören Einsatzszenarien, Annahmen und Umgebungsbedingungen früh in die Betrachtung.

1.10.6 Fehler 6: Verifikation wird zu spät geplant

Verifikation bedeutet: Wir prüfen, ob das System die spezifizierten Anforderungen erfüllt. Wenn Tests erst am Ende überlegt werden, sind manche Anforderungen vielleicht gar nicht sinnvoll prüfbar. Besser ist es, bei jeder wichtigen Anforderung früh zu fragen: ?Wie würden wir das später nachweisen??

Hinweis

Eine einfache Kontrollfrage für jedes Projekt lautet: ?Woran erkennen wir am Ende, dass wir fertig sind?? Wenn diese Frage nicht beantwortet werden kann, sind Zweck, Anforderungen oder Nachweise noch zu unklar.

1.11 Systems Engineering im Projektablauf

Systems Engineering ist kein einzelner Arbeitsschritt. Es begleitet das Projekt. Für den Einstieg kann man den Ablauf in einfache Schritte zerlegen.

1.11.1 Problem verstehen

Am Anfang steht nicht die Lösung, sondern das Problem. Wer braucht den Roboter? Welche Aufgabe soll er übernehmen? Was ist heute unbefriedigend? Welche Umgebung liegt vor? Welche Risiken gibt es?

Für unser Beispiel: Botengänge kosten Zeit, sollen aber nicht durch ein unsicheres oder störendes System ersetzt werden. Das Problem ist also nicht ?Wir brauchen ROS2?, sondern ?Wir möchten interne Transporte zuverlässig automatisieren?.

1.11.2 Stakeholder und Kontext klären

Danach werden Stakeholder und Systemkontext beschrieben. Wer nutzt das System? Wer betreibt es? Wer wartet es? Welche externen Systeme sind beteiligt? Welche Regeln gelten?

1.11.3 Anforderungen ableiten

Aus Bedürfnissen und Kontext entstehen Anforderungen. Gute Anforderungen sind verständlich, eindeutig und prüfbar. In späteren Kapiteln werden wir Anforderungen ausführlicher behandeln.

1.11.4 Architektur entwickeln

Die Architektur beschreibt, aus welchen Hauptteilen das System besteht und wie diese zusammenarbeiten. Beim Roboter könnten Hauptteile sein: Wahrnehmung, Lokalisierung, Navigation, Antrieb, Energieversorgung, Nutzerschnittstelle und Sicherheitslogik.

1.11.5 Integration und Nachweis planen

Schon während der Architekturarbeit muss klar werden, wie das System integriert und getestet wird. Ein Roboter kann nicht erst ganz am Ende zum ersten Mal als Gesamtsystem ausprobiert werden. Man braucht schrittweise Integration: Sensor einzeln testen, Navigation in Simulation testen, Motorsteuerung abgesichert testen, Gesamtsystem in definierten Szenarien testen.

1.12 Zusammenfassung

Systems Engineering ist die Disziplin, komplexe Systeme ganzheitlich zu verstehen, zu strukturieren und nachweisbar zu entwickeln. Ein System besteht nicht nur aus Teilen, sondern aus dem Zusammenspiel dieser Teile in einem bestimmten Kontext. Beim autonomen Indoor-Roboter entstehen die wichtigen Fragen an den Übergängen: zwischen Sensorik und Software, zwischen Roboter und Mensch, zwischen Batterie und Mission, zwischen KI-Erkennung und Datenschutz, zwischen Navigation und Sicherheit.

Für Einsteiger sind vier Grundgedanken besonders wichtig:

- Zuerst das Problem verstehen, dann Lösungen auswählen.
- Anforderungen und technische Lösungen bewusst unterscheiden.
- Stakeholder, Systemgrenze und Kontext früh klären.
- Nachweise und Tests nicht erst am Ende planen.

Dieses Kapitel bildet die Grundlage für den Rest des Buches. Im nächsten Kapitel geht es um Model-Based Systems Engineering. Dort übertragen wir die hier besprochenen Gedanken in ein Modell: Anforderungen, Funktionen, Strukturen, Verhalten, Schnittstellen und Tests werden nicht nur in Texten beschrieben, sondern systematisch miteinander verbunden.

1.13 Kontrollfragen

1. Was ist ein System? Erklären Sie die Definition mit eigenen Worten.
2. Warum ist ein autonomer Indoor-Roboter mehr als eine Sammlung von Bauteilen?
3. Was ist der Unterschied zwischen einer Funktion und einer Komponente?
4. Warum sollte man Anforderungen und Lösungen unterscheiden?
5. Nennen Sie fünf Stakeholder eines autonomen Indoor-Roboters.
6. Welche Interessen könnten Nutzer, Wartungspersonal und Datenschutzverantwortliche jeweils haben?
7. Was versteht man unter einer Systemgrenze?

8. Warum ist die Frage wichtig, ob die Ladestation Teil des Systems ist?
9. Was ist ein Trade-off? Nennen Sie ein Beispiel aus dem Roboterprojekt.
10. Warum sollten Tests und Nachweise früh geplant werden?
11. Was ist an der Anforderung ?Der Roboter soll zuverlässig sein? problematisch?
12. Nennen Sie drei typische Fehler am Anfang eines technischen Projekts.

1.14 Übungen

Übung

Übung 1: Systembeschreibung formulieren

Beschreiben Sie den autonomen Indoor-Roboter in fünf bis acht Sätzen. Vermeiden Sie dabei konkrete Produktnamen oder Bauteiltypen. Markieren Sie anschließend farblich oder mit Symbolen, welche Wörter Funktionen, Stakeholder, Randbedingungen und mögliche Komponenten beschreiben.

Übung

Übung 2: Stakeholderliste erstellen

Erstellen Sie eine Tabelle mit drei Spalten: Stakeholder, Interesse, mögliche Anforderung. Tragen Sie mindestens acht Stakeholder ein. Beispiel: Wartungspersonal, möchte Fehler schnell erkennen, benötigt Diagnoseinformationen über Sensorstatus und Akkuzustand.

Übung

Übung 3: Systemgrenze diskutieren

Entscheiden Sie, ob die Ladestation in Ihrem Modell Teil des Systems sein soll oder ein externes System ist. Begründen Sie Ihre Entscheidung in fünf Sätzen. Notieren Sie anschließend drei Schnittstellen, die durch diese Entscheidung wichtig werden.

Übung

Übung 4: Wünsche in Anforderungen übersetzen

Übersetzen Sie die folgenden Wünsche in überprüfbare Anforderungen: ?Der Roboter soll sicher sein?, ?Der Roboter soll einfach bedienbar sein?, ?Der Roboter soll lange fahren?, ?Der Roboter soll Menschen erkennen?. Achten Sie darauf, dass jede Anforderung später prinzipiell getestet werden kann.

Übung

Übung 5: Trade-offs erkennen

Beschreiben Sie zwei Zielkonflikte im Roboterprojekt. Wählen Sie zum Beispiel Geschwindigkeit gegen Sicherheit oder Akkulaufzeit gegen Gewicht. Erklären Sie jeweils, welche Entscheidung Sie treffen würden und welche Information Ihnen für eine bessere Entscheidung noch fehlt.

1.15 Literaturhinweise

Für den Einstieg in Systems Engineering lohnt sich zunächst ein praxisorientierter Zugang. Lesen Sie nicht nur Definitionen, sondern achten Sie darauf, wie Anforderungen, Architektur, Schnittstellen und Tests zusammenhängen. Besonders hilfreich sind Einführungen des International Council on Systems Engineering (INCOSE), Grundlagenwerke zum Systems Engineering sowie praxisnahe Darstellungen zu SysML und MBSE.

Für dieses Lehrbuch sind außerdem die folgenden Themen als weiterführende Lektüre sinnvoll:

- Grundlagen des Systems Engineering und des Systemdenkens
- Anforderungsanalyse und Stakeholdermanagement
- Verifikation und Validierung technischer Systeme
- Model-Based Systems Engineering
- SysML als Modellierungssprache für technische Systeme
- Sicherheits- und Datenschutzfragen in autonomen Robotiksystemen

Die in späteren Kapiteln verwendete SysML-Literatur, insbesondere praxisnahe Einführungen in SysML, ist auch für dieses Kapitel nützlich, weil sie die Brücke vom allgemeinen Systems Engineering zur konkreten Modellierung schlägt.

2 Einführung in Model-Based Systems Engineering

Lernziele

Nach diesem Kapitel verstehen Sie den Unterschied zwischen dokumentenzentrierter Entwicklung und modellbasierter Entwicklung. Sie können erklären, was **Model-Based Systems Engineering (MBSE)** bedeutet, welche Informationen in einem Systemmodell abgelegt werden und warum ein zentrales Modell bei komplexen Projekten nützlich ist. Sie können den Nutzen von MBSE am Beispiel eines autonomen Indoor-Roboters beschreiben, typische Missverständnisse erkennen und erste Modellinhalte für ein eigenes Roboterprojekt strukturieren.

2.1 Einleitung

Im vorherigen Kapitel ging es um Systems Engineering als Denkweise: ein technisches System wird nicht nur als Sammlung einzelner Teile betrachtet, sondern als Zusammenspiel von Anforderungen, Funktionen, Komponenten, Schnittstellen, Verhalten, Tests und Einsatzumgebung. Dieses Denken ist bereits hilfreich, auch wenn man nur mit Texten, Tabellen und Skizzen arbeitet.

In realen Projekten entsteht jedoch schnell ein praktisches Problem. Informationen liegen an vielen Orten. Anforderungen stehen in einem Word-Dokument oder in einer Excel-Tabelle. Architekturentscheidungen werden in PowerPoint gezeichnet. Schnittstellen werden in E-Mails diskutiert. Testfälle liegen in einem Testmanagement-Werkzeug. Software wird in Git entwickelt. Aufgaben stehen in Jira oder einem ähnlichen Werkzeug. Protokolle aus Besprechungen liegen irgendwo im Projektordner.

Solange das Projekt klein ist, kann das funktionieren. Sobald aber mehrere Personen beteiligt sind und sich Anforderungen ändern, wird es schwierig. Welche Version ist aktuell? Welche Komponente erfüllt welche Anforderung? Welche Tests prüfen die Hinderniserkennung? Welche ROS2-Node nutzt welche Sensordaten? (Eine Node ist ein Softwarebaustein, ein Topic ein Kommunikationskanal zwischen Nodes; mehr dazu weiter unten in diesem Kapitel.) Welche Dokumente müssen angepasst werden, wenn der Roboter eine neue Kamera erhält?

MBSE setzt genau hier an. Model-Based Systems Engineering bedeutet, dass ein Modell zum zentralen Träger der wichtigsten Systeminformationen wird. Das Modell ersetzt nicht alles. Es ersetzt nicht den Code, nicht jede Berechnung und nicht jedes

Gespräch. Aber es verbindet die entscheidenden Informationen so, dass Zusammenhänge nachvollziehbar bleiben.

Praxisbeispiel

Beim Indoor-Roboter könnte eine Anforderung lauten: ?Der Roboter muss Hindernisse rechtzeitig erkennen und sicher stoppen.? Im MBSE-Modell kann diese Anforderung mit einer Funktion ?Hindernis erkennen?, mit einem Lidar-Sensor, mit einer Sicherheitslogik, mit einer ROS2-Node und mit einem Testfall verbunden werden. Dadurch sieht das Team später, welche Teile betroffen sind, wenn die Anforderung geändert wird.

2.2 Dokumentenzentrierte Entwicklung

Dokumentenzentrierte Entwicklung bedeutet, dass Projektinformationen hauptsächlich in einzelnen Dokumenten gespeichert werden. Das können Textdokumente, Tabellen, Präsentationen, Diagrammbilder, E-Mails oder Tickets sein. Diese Arbeitsweise ist weit verbreitet, weil sie leicht beginnt. Fast jeder kann ein Dokument schreiben, eine Tabelle ausfüllen oder ein Diagramm zeichnen.

Der Anfang ist bequem. Ein Team erstellt eine Anforderungsliste in Excel. Die Architektur wird in PowerPoint skizziert. Ein Entwickler beschreibt eine Schnittstelle in einem Markdown-Dokument. Eine Testerin legt Testfälle in einer Tabelle an. Für ein sehr kleines Projekt reicht das oft aus.

Mit wachsender Komplexität entstehen aber typische Probleme:

- Informationen werden mehrfach abgelegt.
- Unterschiedliche Dokumente widersprechen sich.
- Änderungen werden nicht überall nachgeführt.
- Diagramme sehen aktuell aus, enthalten aber veraltete Inhalte.
- Beziehungen zwischen Anforderungen, Architektur und Tests sind nur schwer erkennbar.
- Neue Teammitglieder brauchen lange, um den Projektstand zu verstehen.
- Entscheidungen sind später kaum noch nachvollziehbar.

2.2.1 Beispiel: Eine geänderte Kamera

Angenommen, der Indoor-Roboter erhält statt einer einfachen Kamera eine Tiefenkamera. In einer dokumentenzentrierten Umgebung kann diese Änderung viele Dokumente betreffen:

- die Stückliste

- das Architekturdiagramm
- die Beschreibung der Sensorik
- die Schnittstellenspezifikation
- die Stromverbrauchsabschätzung
- die Softwarebeschreibung der Bildverarbeitung
- die Datenschutzbewertung
- die Testfälle zur Personenerkennung

Wenn nur eines dieser Dokumente vergessen wird, entsteht ein Widerspruch. Vielleicht zeigt die Architektur noch die alte Kamera, während die Schnittstellenspezifikation bereits die neue Kamera beschreibt. Vielleicht testet das Testteam noch mit alten Bildformaten. Vielleicht merkt niemand, dass die neue Kamera mehr Strom verbraucht und die Akkulaufzeit sinkt.

2.2.2 Der stille Aufwand der Synchronisation

In dokumentenzentrierten Projekten besteht ein großer Teil der Arbeit aus Synchronisation. Menschen müssen Dokumente vergleichen, Änderungen weitergeben, Tabellen aktualisieren und nachfragen, welche Information gilt. Dieser Aufwand ist oft unsichtbar, aber er kostet Zeit und erzeugt Fehler.

Ein typischer Satz in solchen Projekten lautet: „Wo steht denn die aktuelle Version?“ Ein anderer lautet: „Das hatten wir doch geändert.“ Beides sind Warnzeichen. Sie zeigen, dass das Projektwissen nicht zuverlässig organisiert ist.

2.3 Was bedeutet modellbasierte Entwicklung?

Modellbasierte Entwicklung bedeutet, dass wichtige Informationen über das System in einem Modell abgelegt werden. Ein Modell ist dabei mehr als eine Zeichnung. Es besteht aus Modellelementen und Beziehungen zwischen diesen Elementen.

Ein Modellelement kann zum Beispiel sein:

- eine Anforderung
- ein Stakeholder
- eine Funktion
- ein Systemelement (`part def`) oder Teilsystem
- eine Schnittstelle
- ein Datenfluss

- ein Zustand
- ein Testfall
- eine ROS2-Node als spätere Implementierungseinheit

Die Beziehungen sind mindestens genauso wichtig wie die Elemente. Eine Anforderung kann durch eine Funktion erfüllt werden. Eine Funktion kann durch eine Komponente realisiert werden. Eine Komponente kann eine Schnittstelle besitzen. Ein Testfall kann eine Anforderung verifizieren. Eine ROS2-Node kann Teil einer Softwarearchitektur sein und Daten über ein Topic austauschen.

2.3.1 Modell statt Bild

Für Einsteiger ist dieser Punkt besonders wichtig: Ein Diagramm ist nicht automatisch ein Modell. Ein Diagramm kann nur ein Bild sein. Wenn man in PowerPoint Kästen und Pfeile zeichnet, sieht das vielleicht wie ein Architekturmodell aus. Die Kästen wissen aber nichts voneinander. Wenn derselbe Block in drei Diagrammen vorkommt, sind das in PowerPoint drei getrennte grafische Objekte.

In einem echten Modellierungswerkzeug ist ein Diagramm dagegen eine Sicht auf Modellelemente. Das Systemelement `NavigationSystem` existiert im Modell. Er kann in mehreren Diagrammen erscheinen. Wenn sein Name geändert wird, sollte die Änderung überall sichtbar werden. Wenn eine Anforderung mit diesem Block verknüpft ist, kann das Werkzeug diese Beziehung anzeigen.

Hinweis

Ein MBSE-Modell ist nicht gut, weil es hübsche Diagramme enthält. Es ist gut, wenn es wichtige Fragen beantwortet: Was muss das System leisten? Welche Teile erfüllen welche Anforderungen? Welche Schnittstellen gibt es? Was passiert bei einer Änderung? Wie wird die Erfüllung nachgewiesen?

2.3.2 Das Modell als gemeinsame Informationsbasis

Das Ziel eines MBSE-Modells ist eine gemeinsame Informationsbasis. Alle Beteiligten sollen nicht in getrennten Dokumentwelten arbeiten, sondern auf eine konsistente Systembeschreibung zugreifen können. Das bedeutet nicht, dass jeder jedes Detail sehen muss. Ein Projektleiter interessiert sich vielleicht für Anforderungen, Risiken und Meilensteine. Eine Softwareentwicklerin interessiert sich für Schnittstellen, Datenflüsse und ROS2-Nodes. Eine Testerin interessiert sich für Anforderungen und Testfälle. Das Modell kann für verschiedene Rollen verschiedene Sichten anbieten. Abbildung 2.1 stellt diesen Unterschied zwischen loser Dokumentensammlung und zentralem Modell noch einmal bildlich gegenüber.

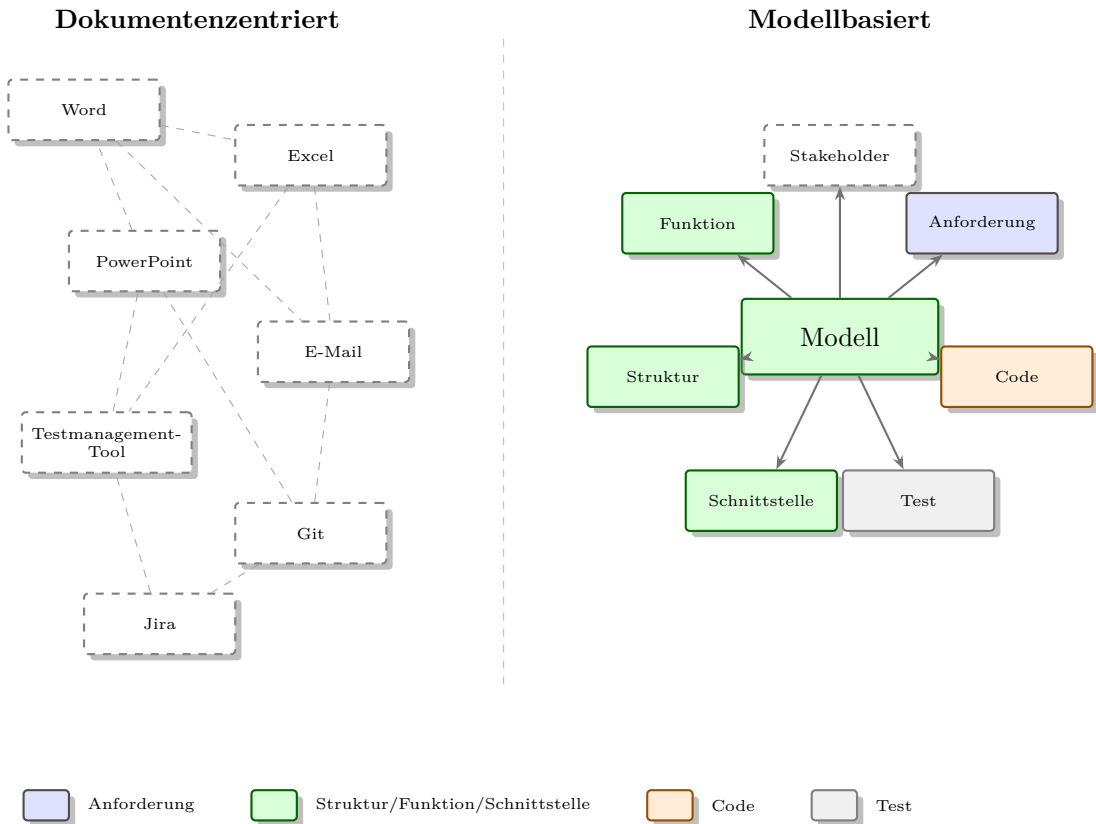


Figure 2.1: Dokumentenzentrierte Entwicklung mit vielen lose verknüpften Dokumenten im Vergleich zu einem zentralen, klar verknüpften Systemmodell.

2.4 Was gehört in ein MBSE-Modell?

Ein MBSE-Modell kann sehr groß werden. Für den Einstieg ist es deshalb wichtig, sich auf zentrale Inhalte zu konzentrieren. Beim autonomen Indoor-Roboter betrachten wir zunächst acht Bereiche.

2.4.1 Stakeholder und Bedürfnisse

Das Modell kann Stakeholder und ihre Bedürfnisse enthalten. Beispiele sind Nutzer, Wartungspersonal, Sicherheitsbeauftragte, Datenschutzverantwortliche und Betreiber. Diese Stakeholder haben Erwartungen, aus denen Anforderungen abgeleitet werden.

Für den Roboter könnte das so aussehen:

- Nutzer wünschen einfache Bedienung.
- Wartungspersonal wünscht verständliche Diagnoseinformationen.
- Sicherheitsbeauftragte wünschen kontrolliertes Verhalten in Personennähe.

- Datenschutzverantwortliche wünschen lokale Verarbeitung von Kameradaten.

2.4.2 Anforderungen

Anforderungen beschreiben, was das System leisten muss. Im Modell werden sie nicht nur als Text gesammelt, sondern mit anderen Elementen verbunden. Eine Anforderung zur Hinderniserkennung kann mit einer Funktion, einer Sensorikkomponente, einem Testfall und später mit einer Softwareeinheit verknüpft werden.

2.4.3 Funktionen

Funktionen beschreiben, was das System tut. Sie liegen zwischen Anforderungen und Lösung. Beispiele für den Roboter sind:

- Umgebung erfassen
- Hindernisse erkennen
- Route planen
- Bewegung ausführen
- Akkustand überwachen
- Ladestation anfahren
- Status melden

2.4.4 Struktur

Die Struktur beschreibt, aus welchen Hauptteilen das System besteht. In SysML werden dafür später Systemteile mit **part def** definiert (wird in Kapitel 3 genauer erklärt). Für den Einstieg kann man an Teilsysteme denken:

- Fahrwerk
- Energieversorgung
- Sensorik
- Recheneinheit
- Navigationssoftware
- Personenerkennung
- Bedienoberfläche
- Ladestation oder Ladeschnittstelle

2.4.5 Schnittstellen

Schnittstellen beschreiben, wie Elemente miteinander verbunden sind. Sie sind entscheidend, weil viele Integrationsprobleme an Schnittstellen entstehen. Beim Roboter gibt es elektrische, mechanische, softwaretechnische und menschliche Schnittstellen.

Beispiele:

- Kamera liefert Bilddaten an die Objekterkennung.
- Lidar liefert Abstandsdaten an die Navigation.
- Batteriemanagement liefert Ladezustand an die Missionsplanung.
- Bedienoberfläche sendet Zielaufträge an das System.
- Roboter und Ladestation tauschen Energie und eventuell Statusinformationen aus.

2.4.6 Verhalten

Verhalten beschreibt Abläufe, Entscheidungen und Zustände. Ein Roboter hat typische Betriebszustände wie ?Bereit?, ?Fährt zum Ziel?, ?Wartet?, ?Kehrt zur Ladestation zurück?, ?Lädt? oder ?Fehler?. Abläufe können beschreiben, wie ein Transportauftrag abgearbeitet wird.

2.4.7 Tests und Nachweise

Ein Modell sollte zeigen, wie Anforderungen überprüft werden. Eine Sicherheitsanforderung benötigt passende Tests. Eine Navigationsanforderung braucht Testszenarien. Eine Datenschutzanforderung braucht vielleicht eine Inspektion der Softwarearchitektur und eine Prüfung der Datenspeicherung.

2.4.8 Entscheidungen und Annahmen

Nicht jede wichtige Information ist eine Anforderung. Viele Projektinformationen sind Annahmen oder Entscheidungen. Beispiel: ?Der Roboter wird nur in Innenräumen mit ebenem Boden eingesetzt.? Oder: ?Kamerabilder werden lokal verarbeitet und nicht dauerhaft gespeichert.? Solche Informationen sollten sichtbar bleiben, weil spätere Änderungen große Auswirkungen haben können.

2.5 Von Dokumenten zu Modellbeziehungen

Der wichtigste Unterschied zwischen klassischer Dokumentation und MBSE liegt in den Beziehungen. Ein Dokument kann eine Anforderung beschreiben. Ein Modell kann zusätzlich zeigen, womit diese Anforderung zusammenhängt. Abbildung ?? zeigt dies am Beispiel von Stakeholderbedarf, Anforderung, Funktion, Systemelement und Testfall.

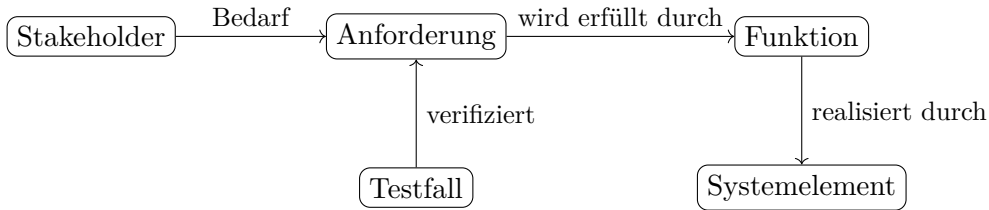


Figure 2.2: Vom Dokument zur Modellbeziehung: Ein Stakeholderbedarf führt zu einer Anforderung, die durch eine Funktion erfüllt und durch ein Systemelement realisiert wird; ein Testfall verifiziert die Anforderung.

Dieses einfache Bild zeigt bereits den Kern von MBSE. Es geht nicht darum, viele Diagramme zu produzieren. Es geht darum, Systeminformationen so zu verknüpfen, dass Fragen beantwortbar werden.

2.5.1 Traceability

Traceability bedeutet Nachvollziehbarkeit. Man kann von einer Information zu verwandten Informationen springen. Von einer Anforderung gelangt man zu Funktionen, Komponenten und Tests. Von einer Komponente erkennt man, welche Anforderungen sie erfüllt. Von einem Test sieht man, welche Anforderung er prüft.

Beim Indoor-Roboter ist Traceability besonders hilfreich. Wenn die Sicherheitsanforderung verschärft wird, möchte das Team wissen:

- Welche Funktionen sind betroffen?
- Welche Sensoren und Softwarekomponenten sind beteiligt?
- Welche Schnittstellen müssen angepasst werden?
- Welche Tests müssen erweitert werden?
- Welche Risiken entstehen für Kosten, Zeitplan und Akkulaufzeit?

Ohne Modell muss man diese Zusammenhänge mühsam aus Dokumenten zusammensuchen. Mit einem gepflegten Modell werden sie zumindest teilweise direkt sichtbar.

2.6 MBSE im Roboterprojekt

Für unser Lehrbuchprojekt modellieren wir den autonomen Indoor-Roboter schrittweise. Das Modell wächst mit dem Verständnis. Am Anfang muss es noch nicht vollständig sein. Es soll die wichtigsten Zusammenhänge sichtbar machen.

2.6.1 Erste Modellstruktur

Eine einfache Startstruktur für das Modell könnte so aussehen:

- Paket ?Stakeholder und Kontext?
- Paket ?Anforderungen?
- Paket ?Funktionen?
- Paket ?Systemstruktur?
- Paket ?Schnittstellen?
- Paket ?Verhalten?
- Paket ?ROS2-Architektur?
- Paket ?Tests und Verifikation?

Ein Paket ist hier zunächst nur ein Ordnungsbereich. Es hilft, das Modell übersichtlich zu halten. Später werden wir sehen, wie solche Pakete als SysML-v2-Dateien und Namespaces konkret angelegt werden können.

2.6.2 Beispielkette: Hinderniserkennung

Betrachten wir eine typische Modellkette:

1. Stakeholderbedarf: Personen im Gebäude sollen nicht gefährdet werden.
2. Anforderung: Der Roboter muss Hindernisse im Fahrweg erkennen und rechtzeitig stoppen.
3. Funktion: Hindernis erkennen.
4. Funktion: Bewegung sicher begrenzen.
5. Systemelement: Lidar-Sensor.
6. Systemelement: Sicherheitslogik.
7. ROS2-Element: Topic für Sensordaten.
8. ROS2-Element: Node zur Hindernisauswertung.
9. Testfall: Stoppverhalten bei Hindernis in definiertem Abstand prüfen.

Diese Kette zeigt, wie MBSE die Brücke von einem menschlichen Bedürfnis bis zu einem technischen Test schlägt. Genau diese Brücke ist in komplexen Projekten wertvoll. Abbildung 2.2 stellt diese Modellkette farbkodiert dar, von der Anforderung (blau) über Funktionen und Systemelemente (grün) bis zu ROS2-Elementen (orange) und dem Testfall (grau).

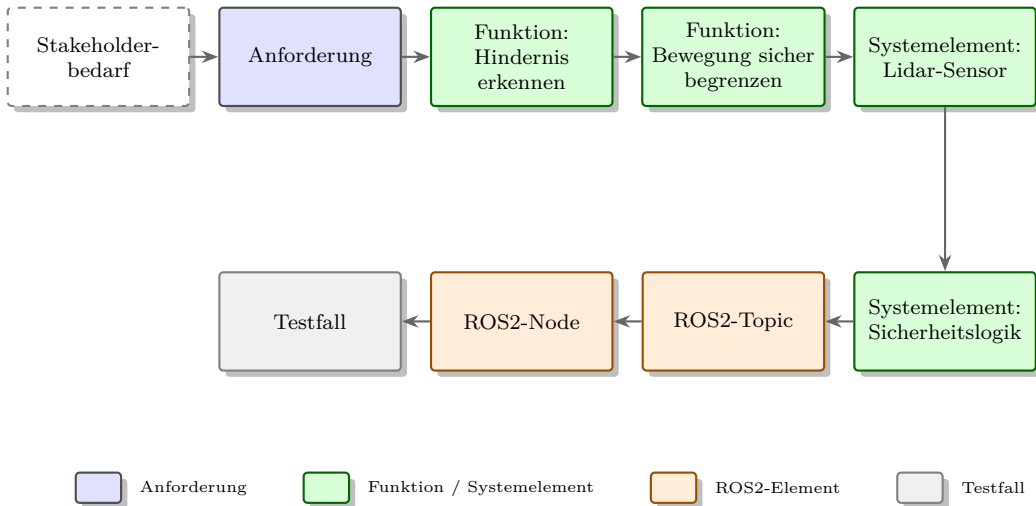


Figure 2.3: Farbkodierte Modellkette von einem Stakeholderbedarf über Anforderung, Funktionen und Systemelemente bis zu ROS2-Node und Testfall.

2.6.3 Beispielkette: Batteriemanagement

Eine zweite Kette betrifft den Akku:

1. Stakeholderbedarf: Der Roboter soll nicht mitten im Flur stehen bleiben.
2. Anforderung: Der Roboter muss bei niedrigem Akkustand selbstständig zur Ladestation zurückkehren.
3. Funktion: Akkustand überwachen.
4. Funktion: Rückkehr zur Ladestation planen.
5. Systemelement: Batterie.
6. Systemelement: Batteriemanagement.
7. Systemelement: Navigation.
8. ROS2-Element: Battery-State-Nachricht.
9. Testfall: Rückkehr zur Ladestation bei unterschrittenem Schwellwert prüfen.

Auch hier sieht man: Die Anforderung betrifft nicht nur eine einzelne Komponente. Sie verbindet Energieversorgung, Software, Navigation, Ladeinfrastruktur und Test.

2.7 Nutzen von MBSE

MBSE ist kein Selbstzweck. Der Nutzen entsteht nur, wenn das Modell echte Projektfragen beantwortet. Die wichtigsten Vorteile sind Nachvollziehbarkeit, Konsistenz, Kommunikation, Änderungsanalyse und bessere Vorbereitung der Umsetzung.

2.7.1 Nachvollziehbarkeit

Nachvollziehbarkeit bedeutet, dass Entscheidungen und Zusammenhänge erkennbar bleiben. Wenn jemand fragt, warum der Roboter einen Lidar-Sensor besitzt, sollte das Modell nicht nur „weil wir das so entschieden haben“ zeigen, sondern den Zusammenhang mit Anforderungen und Funktionen.

2.7.2 Konsistenz

Ein Modell kann helfen, Widersprüche zu vermeiden. Wenn ein Systemelement umbenannt wird, erscheint es in allen Sichten mit demselben Namen. Wenn eine Anforderung keinen Testfall besitzt, kann das auffallen. Wenn eine Funktion keiner Komponente zugeordnet ist, entsteht eine offene Frage.

2.7.3 Kommunikation

Diagramme und strukturierte Modelle helfen Teams, über dasselbe System zu sprechen. Gerade bei einem Roboterprojekt ist das wichtig, weil unterschiedliche Fachgebiete beteiligt sind. Eine gemeinsame Modellstruktur kann Missverständnisse reduzieren.

2.7.4 Änderungsanalyse

Änderungen sind normal. MBSE hilft zu verstehen, welche Folgen eine Änderung hat. Wenn die maximale Geschwindigkeit erhöht wird, betrifft das nicht nur die Motoren. Es betrifft auch Bremsweg, Hinderniserkennung, Sicherheitslogik, Akkulaufzeit, Tests und eventuell Zulassungsfragen.

2.7.5 Brücke zur Umsetzung

In diesem Lehrbuch ist besonders wichtig: Das Modell soll später zur ROS2-Architektur passen. Wir wollen nicht modellieren und anschließend alles neu erfinden. Wir wollen verstehen, wie Anforderungen, Funktionen und Systemelemente auf Nodes, Topics, Services, Parameter und Tests abgebildet werden können.

Praxisbeispiel

Eine SysML-Funktion „Person erkennen“ kann später auf eine ROS2-Node `object_detector_node` abgebildet werden. Die Schnittstelle dieser Funktion kann zu Topics für Kamerabilder und erkannte Objekte führen. Dadurch entsteht eine nachvollziehbare Verbindung zwischen Systemmodell und Softwarestruktur.

2.8 Grenzen und Missverständnisse

MBSE hilft, Komplexität zu beherrschen. Es löst aber nicht automatisch alle Probleme. Ein Modell kann falsch, veraltet oder unnötig kompliziert sein. Dann erzeugt es mehr Aufwand als Nutzen.

2.8.1 Missverständnis 1: MBSE bedeutet, alles zu modellieren

Ein gutes Modell enthält nicht jedes Detail. Es enthält die Details, die für Verständnis, Entscheidungen, Schnittstellen, Risiken und Nachweise wichtig sind. Ein Modell, das jeden kleinen Softwareparameter abbildet, kann unübersichtlich werden. Ein Modell, das nur drei hübsche Bilder enthält, ist zu oberflächlich.

2.8.2 Missverständnis 2: Das Modell ersetzt Kommunikation

Ein Modell unterstützt Kommunikation, ersetzt sie aber nicht. Menschen müssen weiterhin Fragen stellen, Entscheidungen treffen und Annahmen klären. Das Modell dokumentiert und verknüpft die Ergebnisse.

2.8.3 Missverständnis 3: Das Werkzeug macht das Engineering

Ein SysML-v2-Werkzeug kann Beziehungen prüfen, Sichten erzeugen und Konsistenzprüfungen unterstützen. Es entscheidet aber nicht, welche Anforderungen sinnvoll sind oder welche Architektur gut ist. Das bleibt Ingenieursarbeit.

2.8.4 Missverständnis 4: MBSE ist nur für große Konzerne

MBSE wird oft mit großen Luftfahrt-, Automobil- oder Raumfahrtprojekten verbunden. Die Grundidee ist aber auch für kleine Projekte nützlich. Gerade ein Lernprojekt profitiert davon, weil Zusammenhänge sichtbar werden. Man muss nur den Umfang passend wählen.

Hinweis

Für dieses Lehrbuch gilt: Wir modellieren so viel, dass die Zusammenhänge zwischen SysML v2, MBSE und ROS2 verständlich werden. Wir modellieren nicht jedes denkbare Detail des Roboters.

2.9 Ein pragmatischer Einstieg in MBSE

Einsteiger sollten MBSE nicht mit dem Anspruch beginnen, sofort ein perfektes Modell zu erstellen. Besser ist ein schrittweiser Einstieg.

2.9.1 Schritt 1: Zweck und Kontext beschreiben

Zuerst wird beschrieben, wofür das System gebaut wird und in welcher Umgebung es arbeitet. Beim Roboter: Innenräume, definierter Gebäudebereich, Menschen im Umfeld, Ladestation, Bedienoberfläche und mögliche Netzwerkverbindung.

2.9.2 Schritt 2: Anforderungen sammeln

Danach werden wichtige Anforderungen gesammelt. Sie müssen am Anfang noch nicht perfekt sein. Wichtig ist, sie sichtbar zu machen und später zu verbessern. Gute Anforderungen sind verständlich, eindeutig und prüfbar.

2.9.3 Schritt 3: Funktionen ableiten

Aus Anforderungen werden Funktionen abgeleitet. Wenn der Roboter automatisch zur Ladestation zurückkehren soll, braucht er Funktionen wie ?Akkustand überwachen?, ?Ladestation lokalisieren? und ?Rückkehr planen?.

2.9.4 Schritt 4: Struktur entwerfen

Dann wird überlegt, welche Systemelemente die Funktionen realisieren. Hier entstehen erste Systemteile (`part def`, wird in Kapitel 3 genauer erklärt), Teilsysteme oder Komponenten. Beim Roboter sind das zum Beispiel Sensorik, Navigation, Energieversorgung und Bedienung.

2.9.5 Schritt 5: Schnittstellen beschreiben

Schnittstellen sollten früh beschrieben werden. Wer liefert welche Daten? Wer benötigt welche Information? Welche Signale, Nachrichten oder Energieflüsse gibt es?

2.9.6 Schritt 6: Tests zuordnen

Zu wichtigen Anforderungen werden Tests oder andere Nachweise zugeordnet. Dadurch bleibt klar, wie das Team später prüfen will, ob das System funktioniert.

2.10 Qualitätsmerkmale eines guten Modells

Ein MBSE-Modell ist nicht automatisch gut, nur weil es groß ist. Gute Modelle besitzen einige praktische Eigenschaften.

2.10.1 Verständlichkeit

Ein neues Teammitglied sollte sich im Modell orientieren können. Namen sollten sprechend sein. Diagramme sollten nicht überladen sein. Die Struktur sollte nachvollziehbar sein.

2.10.2 Konsistenz

Begriffe sollten einheitlich verwendet werden. Wenn eine Komponente einmal ?Battery Monitor?, einmal ?Akkumodul? und einmal ?Power Node? heißt, entstehen Missverständnisse. Für ein deutsches Lehrbuch können deutsche Begriffe im Systemmodell sinnvoll sein, während spätere ROS2-Paketnamen technisch englisch bleiben.

2.10.3 Angemessene Tiefe

Das Modell sollte nicht zu grob und nicht zu detailliert sein. Eine gute Frage lautet: ?Hilft dieses Detail bei einer Entscheidung, Schnittstelle, Umsetzung oder Prüfung?? Wenn nicht, gehört es vielleicht nicht ins Einsteigermodell.

2.10.4 Pflegefähigkeit

Ein Modell muss gepflegt werden können. Wenn jede kleine Änderung viel manuelle Nacharbeit erfordert, wird das Modell schnell veraltet. Deshalb ist eine einfache, klare Modellstruktur wichtig.

2.10.5 Nachweisbarkeit

Wichtige Anforderungen sollten mit Nachweisen verbunden sein. Nicht jede Anforderung braucht sofort einen vollständig ausgearbeiteten Test, aber es sollte erkennbar sein, wie die Erfüllung später geprüft wird.

2.11 Sichten und Rollen im Modell

Ein Systemmodell enthält viele Informationen. Trotzdem soll nicht jede Person ständig alles sehen müssen. Deshalb arbeitet MBSE mit Sichten. Eine Sicht zeigt einen bestimmten Ausschnitt des Modells für eine bestimmte Fragestellung oder Rolle.

Das ist vergleichbar mit einer Gebäudekarte. Eine Architektin betrachtet Grundrisse, ein Elektriker betrachtet Stromleitungen, die Feuerwehr betrachtet Fluchtwege und eine Besucherin betrachtet den Lageplan. Alle Sichten beziehen sich auf dasselbe Gebäude, zeigen aber unterschiedliche Informationen.

2.11.1 Sicht der Projektleitung

Die Projektleitung interessiert sich vor allem für Fortschritt, Risiken, offene Entscheidungen und Auswirkungen von Änderungen. Im MBSE-Modell könnten dafür folgende Fragen beantwortet werden:

- Welche Anforderungen sind noch ungeklärt?
- Welche Anforderungen haben noch keinen Nachweis?
- Welche Teilsysteme sind besonders stark von Änderungen betroffen?

- Welche Schnittstellen sind noch nicht beschrieben?
- Welche Risiken betreffen Sicherheit, Kosten oder Termine?

Für die Projektleitung ist das Modell also kein technisches Bilderbuch, sondern ein Werkzeug zur Steuerung. Es zeigt, wo Unsicherheit liegt.

2.11.2 Sicht der Systemarchitektur

Die Systemarchitektur betrachtet die großen Bausteine und ihre Beziehungen. Beim Roboter geht es zum Beispiel um die Aufteilung in Sensorik, Lokalisierung, Navigation, Antrieb, Energieversorgung, Bedienung und Sicherheitsfunktionen. Diese Sicht beantwortet Fragen wie:

- Welche Hauptsysteme gibt es?
- Welche Daten und Signale fließen zwischen ihnen?
- Wo liegen kritische Schnittstellen?
- Welche Funktionen sind welchem Teilsystem zugeordnet?
- Welche Komponenten sind für sicherheitsrelevantes Verhalten verantwortlich?

2.11.3 Sicht der Softwareentwicklung

Die Softwareentwicklung interessiert sich für Datenflüsse, Zustände, Abläufe, Verantwortlichkeiten und technische Schnittstellen. In unserem Projekt wird diese Sicht später besonders wichtig, weil wir SysML mit ROS2 verbinden wollen.

Eine Softwareentwicklerin möchte wissen:

- Welche ROS2-Nodes werden benötigt?
- Welche Topics transportieren Sensordaten, Statusinformationen oder Steuerbefehle?
- Welche Services oder Actions passen zu Aufträgen und Navigation?
- Welche Parameter müssen konfigurierbar sein?
- Welche Fehlerzustände muss die Software erkennen?

Das MBSE-Modell liefert dafür noch keinen fertigen Code. Es liefert aber eine strukturierte Grundlage, damit die Softwarearchitektur nicht zufällig entsteht.

2.11.4 Sicht des Testteams

Das Testteam möchte wissen, was geprüft werden muss und wie Tests mit Anforderungen zusammenhängen. Besonders wichtig ist die Frage, ob jede zentrale Anforderung einen Nachweis besitzt.

Beim Indoor-Roboter könnten Testsichten zeigen:

- Anforderungen an Hinderniserkennung und passende Testszenarien
- Anforderungen an Akkulaufzeit und Messverfahren
- Zustände des Roboters und Tests für Zustandsübergänge
- Schnittstellen zwischen ROS2-Nodes und Integrationstests
- Fehlersituationen und erwartete sichere Reaktionen

Diese Sicht verhindert, dass Tests erst am Ende erfunden werden. Gute Tests entstehen parallel zum Systemverständnis.

Abbildung 2.3 fasst die vier beschriebenen Sichten noch einmal zusammen: Alle greifen auf dasselbe Modell zu, betrachten es aber aus unterschiedlichen Blickwinkeln.

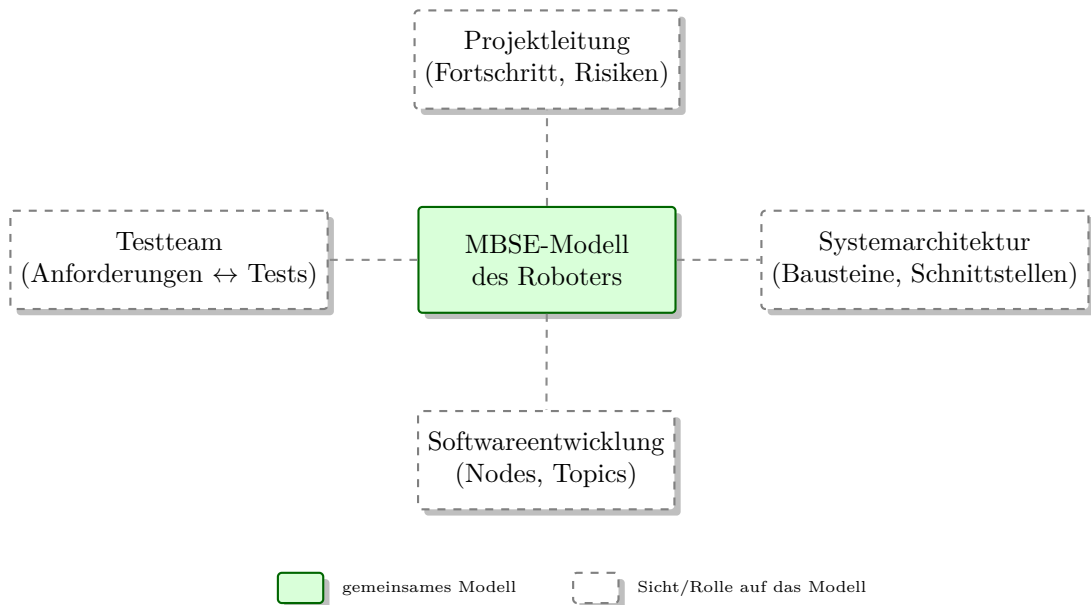


Figure 2.4: Vier Rollen betrachten dasselbe MBSE-Modell des Roboters aus unterschiedlichen Blickwinkeln.

2.12 Vom MBSE-Modell zur ROS2-Architektur

Ein zentrales Ziel dieses Lehrbuchs ist die Verbindung von SysML v2, MBSE und ROS2. Deshalb ist es wichtig, früh zu verstehen, wie ein Systemmodell zur späteren

Softwarestruktur führen kann.

Der Übergang erfolgt nicht automatisch. Ein SysML-Systemelement wird nicht durch Zauberei zu einer ROS2-Node. Aber das Modell kann systematisch vorbereiten, welche Softwareeinheiten sinnvoll sind und wie sie miteinander kommunizieren.

2.12.1 Funktionen als Ausgangspunkt

Am Anfang stehen Funktionen. Funktionen beschreiben, was das System tun muss. Für den Roboter sind das zum Beispiel ?Umgebung erfassen?, ?Position bestimmen?, ?Route planen?, ?Hindernisse vermeiden?, ?Personen erkennen? und ?Akkustand überwachen?.

Diese Funktionen können später auf Softwarebausteine abgebildet werden. Nicht jede Funktion wird automatisch eine eigene Node. Manchmal werden mehrere Funktionen in einer Node zusammengefasst. Manchmal wird eine Funktion auf mehrere Nodes verteilt. Die Modellierung hilft aber, diese Entscheidung bewusst zu treffen.

2.12.2 Datenflüsse als Vorbereitung auf Topics

ROS2-Systeme kommunizieren häufig über Topics. Ein Topic transportiert Nachrichten von einem Sender zu einem oder mehreren Empfängern. Im MBSE-Modell können Datenflüsse vorbereitet werden, bevor konkrete Topic-Namen festgelegt werden.

Beispiele:

- Kamera liefert Bilddaten an Personenerkennung.
- Lidar liefert Abstandsdaten an Hindernisauswertung.
- Lokalisierung liefert aktuelle Pose an Navigation.
- Navigation liefert Geschwindigkeitsvorgaben an Antriebssteuerung.
- Batteriemanagement liefert Ladezustand an Missionsplanung.

Später können daraus ROS2-Topics wie `/camera/image`, `/scan`, `/robot_pose`, `/cmd_vel` oder `/battery_state` entstehen. Die genauen Namen sind technische Entscheidungen. Die fachliche Kommunikationsstruktur kann aber schon im Modell sichtbar werden.

2.12.3 Zustände als Grundlage für Steuerlogik

Ein autonomer Roboter besitzt Betriebszustände. Beispiele sind ?Bereit?, ?Auftrag aktiv?, ?Navigiert?, ?Hindernis erkannt?, ?Kehrt zurück?, ?Lädt? und ?Fehler?. Diese Zustände können in SysML modelliert werden und helfen später bei der Softwareentwicklung.

Ohne Zustandsmodell entsteht Steuerlogik oft verteilt im Code. Dann weiß nach einiger Zeit niemand mehr genau, wann der Roboter fahren darf, wann er stoppen muss oder wie er aus einem Fehlerzustand zurückkehrt. Ein Zustandsmodell schafft eine gemeinsame Grundlage.

2.12.4 Tests als Verbindung zwischen Modell und Code

Tests bilden eine wichtige Brücke zwischen Modell und Umsetzung. Wenn eine Anforderung im Modell mit einem Testfall verbunden ist, kann der spätere Code gezielt gegen diese Erwartung geprüft werden.

Beispiel:

1. Anforderung: Der Roboter muss bei niedrigem Akkustand zur Ladestation zurückkehren.
2. Modellfunktion: Rückkehr zur Ladestation auslösen.
3. ROS2-Node: `battery_monitor_node` meldet niedrigen Ladezustand.
4. ROS2-Node: `navigation_node` erhält Ziel ?Ladestation?.
5. Test: Simulierter Akkustand unterschreitet Schwellwert, anschließend wird ein Rückkehrziel gesetzt.

So wird aus einer fachlichen Erwartung eine überprüfbare technische Umsetzung.

2.13 Typische Fehler beim Einstieg in MBSE

Auch MBSE kann falsch eingesetzt werden. Gerade am Anfang helfen einige Warnsignale.

2.13.1 Fehler 1: Diagramme ohne Modellinhalt

Das Team zeichnet schöne Diagramme, aber die Elemente sind nicht sauber benannt, nicht wiederverwendet und nicht verknüpft. Dann entsteht nur eine hübschere Form dokumentenzentrierter Entwicklung.

2.13.2 Fehler 2: Zu viele Details zu früh

Einsteiger versuchen manchmal, sofort jedes Kabel, jede Softwareklasse und jeden Parameter zu modellieren. Dadurch wird das Modell groß, bevor die Grundstruktur verstanden ist. Besser ist es, mit Anforderungen, Hauptfunktionen, Hauptsystemen und kritischen Schnittstellen zu beginnen.

2.13.3 Fehler 3: Keine Modellpflege

Ein Modell ist nur nützlich, wenn es gepflegt wird. Wenn Architekturentscheidungen im Code geändert werden, aber nie ins Modell zurückfließen, verliert das Modell seinen Wert. Deshalb muss im Projekt geklärt sein, welche Modellteile verbindlich gepflegt werden.

2.13.4 Fehler 4: Unklare Modellierungsregeln

Wenn jedes Teammitglied anders modelliert, entsteht Unordnung. Schon einfache Regeln helfen: einheitliche Namen, klare Paketstruktur, definierte Beziehungstypen und begrenzte Diagrammgrößen.

2.13.5 Fehler 5: Modell und Implementierung werden getrennte Welten

Das Modell darf nicht neben der Entwicklung herlaufen wie eine Pflichtübung. Es muss Fragen beantworten, die im Projekt wirklich vorkommen. In unserem Lehrbuch ist deshalb die Verbindung zur ROS2-Architektur wichtig. Das Modell soll nicht nur erklären, sondern die Umsetzung vorbereiten.

2.14 Zusammenfassung

MBSE überträgt die Gedanken des Systems Engineering in eine modellbasierte Arbeitsweise. Statt Projektwissen über viele lose Dokumente zu verteilen, werden wichtige Informationen in einem Systemmodell strukturiert und miteinander verknüpft. Das Modell enthält Anforderungen, Funktionen, Struktur, Schnittstellen, Verhalten, Tests, Annahmen und Entscheidungen.

Der wichtigste Punkt ist nicht das Zeichnen von Diagrammen, sondern das Herstellen von Beziehungen. Eine Anforderung steht nicht isoliert im Raum. Sie hängt mit Stakeholdern, Funktionen, Systemelementen, Schnittstellen und Tests zusammen. Genau diese Nachvollziehbarkeit macht MBSE wertvoll.

Für den autonomen Indoor-Roboter bedeutet das: Wir können vom Bedarf eines Nutzers über Anforderungen und Funktionen bis zur späteren ROS2-Architektur denken. Das Modell wird zur Brücke zwischen Problemverständnis, SysML-v2-Modellierung im Syside Editor und technischer Umsetzung.

2.15 Kontrollfragen

1. Was bedeutet Model-Based Systems Engineering?
2. Worin unterscheidet sich ein Modell von einem einfachen Diagrammbild?
3. Warum wird dokumentenzentrierte Entwicklung bei komplexeren Projekten problematisch?
4. Welche Informationen können in einem MBSE-Modell enthalten sein?
5. Was versteht man unter Traceability?
6. Warum ist Traceability für Anforderungen und Tests wichtig?
7. Nennen Sie drei typische Modellbeziehungen im Roboterprojekt.
8. Warum ersetzt MBSE nicht den Quellcode?

9. Warum ersetzt MBSE nicht die Kommunikation im Team?
10. Welche Vorteile bietet MBSE bei Änderungen?
11. Was wäre ein Beispiel für eine zu detaillierte Modellierung?
12. Welche Pakete würden Sie für ein erstes Modell des Indoor-Roboters anlegen?

2.16 Übungen

Übung

Übung 1: Dokumentenchaos erkennen

Erstellen Sie eine Liste mit fünf Dokumenten oder Informationsquellen, die in einem klassischen Roboterprojekt entstehen könnten. Beschreiben Sie für jede Quelle, welches Risiko entsteht, wenn sie nicht aktuell gehalten wird.

Übung

Übung 2: Modellkette erstellen

Erstellen Sie eine einfache Kette aus Stakeholderbedarf, Anforderung, Funktion, Systemelement und Testfall für die automatische Rückkehr zur Ladestation.

Übung

Übung 3: Anforderungen verknüpfen

Wählen Sie drei Anforderungen für den Indoor-Roboter. Ordnen Sie jeder Anforderung mindestens eine Funktion, ein mögliches Systemelement und einen Testfall zu.

Übung

Übung 4: Modellumfang begrenzen

Notieren Sie zehn mögliche Informationen über den Roboter. Entscheiden Sie für jede Information, ob sie in ein Einsteiger-MBSE-Modell gehört oder zunächst außerhalb bleiben kann. Begründen Sie Ihre Entscheidung kurz.

Übung

Übung 5: Änderungsanalyse

Angenommen, der Roboter soll nicht mehr 0,8 m/s, sondern 1,2 m/s fahren dürfen. Beschreiben Sie, welche Anforderungen, Funktionen, Komponenten, Schnittstellen und Tests davon betroffen sein könnten.

2.17 Literaturhinweise

Für den Einstieg in MBSE sind praxisnahe SysML-Einführungen besonders hilfreich, weil sie Modellierung nicht abstrakt behandeln, sondern an Anforderungen, Architektur und Verhalten erklären. Empfehlenswert sind insbesondere die in diesem Projekt bereits geführten Werke zu SysML, zum Beispiel Delligatti [?] sowie Friedenthal, Moore und Steiner [?].

Darüber hinaus lohnt sich weiterführende Literatur zu Systems Engineering, Anforderungsmanagement, Verifikation und Validierung sowie modellbasierter Entwicklung cyber-physischer Systeme. Für die spätere technische Umsetzung ist außerdem die ROS2-Dokumentation [?] wichtig, weil sie zeigt, wie Softwarearchitekturen mit Nodes, Topics, Services, Actions und Parametern konkret aufgebaut werden.

3 SysML-Grundlagen

Lernziele

Nach diesem Kapitel kennen Sie die Grundidee von [Systems Modeling Language \(SysML\)](#) v2, die wichtigsten Modellierungskonzepte und deren Rolle in einem MBSE-Projekt. Sie können erklären, warum ein SysML-Diagramm nur eine Sicht auf ein Modell ist, welche Konzepte für den Einstieg besonders wichtig sind und wie diese Konzepte beim autonomen Indoor-Roboter zusammenspielen.

3.1 Einleitung

In den ersten beiden Kapiteln haben wir geklärt, warum Systems Engineering und MBSE nützlich sind. Jetzt benötigen wir eine Sprache, mit der wir Anforderungen, Struktur, Verhalten, Schnittstellen und Tests präzise beschreiben können. Diese Sprache ist in diesem Buch [SysML v2](#).

[SysML](#) steht für Systems Modeling Language. Die Sprache wurde entwickelt, um technische Systeme zu modellieren. SysML v2 ist eine vollständige Neuentwicklung gegenüber SysML v1 mit einer präzisen textuellen Notation und einer klaren, modernen Semantik. Sie ist stärker auf Systeme aus Hardware, Software, Menschen, Prozessen, Daten und physikalischen Zusammenhängen ausgerichtet als UML. Das passt gut zu unserem autonomen Indoor-Roboter: Er besteht nicht nur aus Software, sondern auch aus Sensorik, Antrieb, Batterie, Gehäuse, Ladestation, Bedienung und realer Umgebung.

Für Einsteiger ist wichtig: SysML ist kein Zeichenprogramm. SysML ist eine Modellierungssprache. Ein Diagramm ist nur eine sichtbare Darstellung von Modellelementen. Der eigentliche Wert entsteht im Modell: Anforderungen sind mit Systemteilen verbunden, Systemteile besitzen Schnittstellen, Verhalten beschreibt Abläufe, Tests verifizieren Anforderungen.

3.2 Warum eine Modellierungssprache?

Ohne gemeinsame Sprache entstehen in Projekten schnell Missverständnisse. Ein Teammitglied zeichnet Kästen für Funktionen, ein anderes für Softwaremodule, ein drittes für Hardwarekomponenten. Alle Diagramme sehen ähnlich aus, meinen aber unterschiedliche Dinge. SysML hilft, solche Bedeutungen zu klären.

SysML v2 bietet vordefinierte Konzepte:

- `requirement` für Anforderungen
- `part def` und `part` für Systemelemente und deren Instanzen
- `port` und `connection` für Schnittstellen
- `action def` und `action` für Abläufe
- `state def` und `state` für Zustände
- `calc def` und `constraint` für mathematische Zusammenhänge
- `satisfy`, `verify`, `derive` für Nachvollziehbarkeit

Damit kann ein Team über dasselbe System sprechen, ohne jedes Symbol neu erklären zu müssen.

Praxisbeispiel

Wenn im Modell eine `part def PerceptionSystem` existiert, kann sie in einer Struktursicht als Teil des Roboters erscheinen, in einer Verbindungssicht mit Kamera und Lidar verknüpft werden und über eine `satisfy`-Beziehung Anforderungen zur Hinderniserkennung erfüllen. Es ist derselbe Modellinhalt, nur aus verschiedenen Blickwinkeln.

3.3 Wichtige Konzepte in SysML v2

SysML v2 kennt keine festen Diagrammtypen mehr wie in v1. Stattdessen gibt es Sichten (*views*) und Sichtpunkte (*viewpoints*) auf ein gemeinsames Modell. Ein Sichtpunkt legt fest, welche Art von Frage eine Sicht beantworten soll, zum Beispiel Struktur oder Verhalten; die Sicht selbst ist die daraus erzeugte konkrete Darstellung. Für den Einstieg konzentrieren wir uns auf diejenigen Konzepte, die für das Roboterprojekt den größten Nutzen bringen. Abbildung 3.1 gibt einen Überblick über die sieben wichtigsten Konzepte, bevor wir sie einzeln vorstellen.

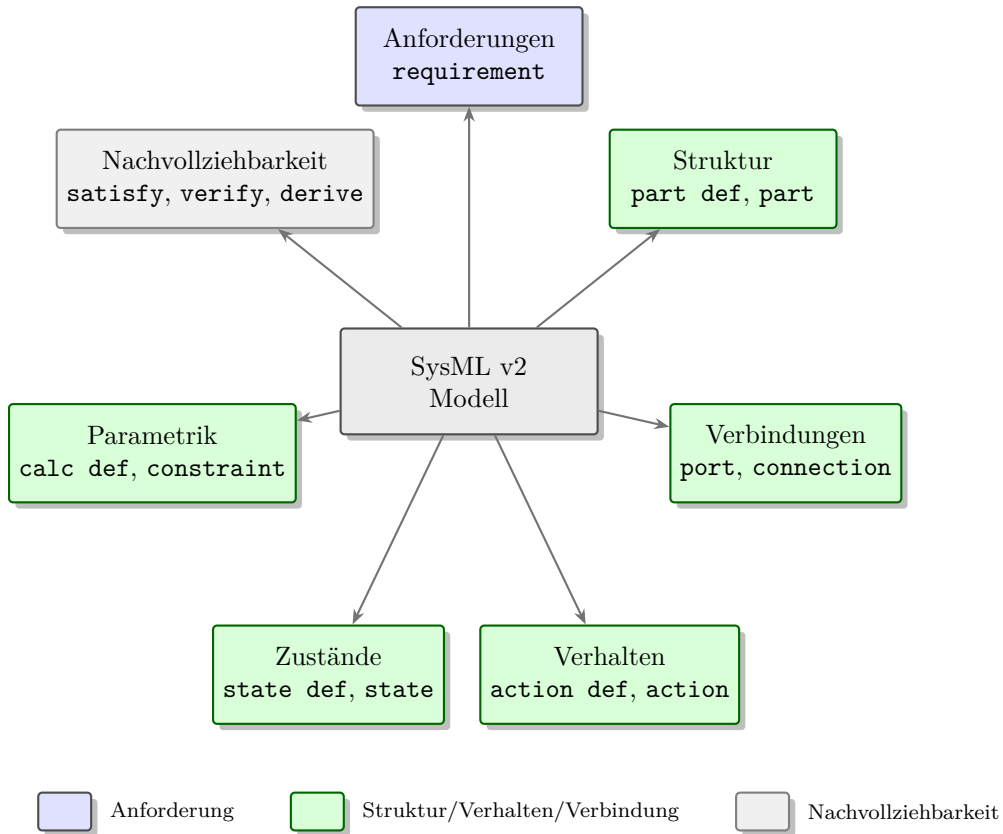


Figure 3.1: Die sieben für dieses Buch wichtigsten SysML-v2-Konzepte rund um das gemeinsame Modell.

3.3.1 Anforderungen (requirement)

Anforderungen beschreiben, was das System leisten muss. In SysML v2 werden sie textuell mit **requirement** definiert. Beziehungen wie **satisfy**, **verify** und **derive** verbinden Anforderungen mit Systemteilen und Testfällen.

Typische Fragen:

- Was muss das System leisten?
- Welche Anforderungen hängen zusammen?
- Welche Anforderung wurde aus welcher höheren abgeleitet?
- Welche Teile erfüllen eine Anforderung?
- Welche Tests überprüfen eine Anforderung?

3.3.2 Strukturmodellierung (part def, part)

In SysML v2 beschreibt **part def** den Typ eines Systemelements. **part** erzeugt eine Instanz davon. Zusammen ersetzen sie den Block-Begriff aus SysML v1.


```
1 package RobotStructure {  
2   part def Robot {  
3     part perceptionSystem : PerceptionSystem;  
4     part navigationSystem : NavigationSystem;  
5     part batterySystem : BatterySystem;  
6     part motionControl : MotionControl;  
7   }  
8   part def PerceptionSystem;  
9   part def NavigationSystem;  
10  part def BatterySystem;  
11  part def MotionControl;  
12 }
```

Diese Darstellung beantwortet die Frage: Aus welchen Hauptbausteinen besteht das System?

3.3.3 Verbindungsmodellierung (port, connection)

port beschreibt die Schnittstelle eines Systemteils nach außen. **connection** verbindet Ports verschiedener Teile miteinander. Damit wird sichtbar, wie Teile eines Systems intern zusammenarbeiten.

```
1 part def PerceptionSystem {  
2   port sensorData : ~SensorDataPort;  
3   part camera : Camera;  
4   part lidar : Lidar;  
5   part detector : ObjectDetector;  
6 }
```

Hinweis

Das Tilde-Zeichen (~) kennzeichnet einen sogenannten konjugierten Port; was das genau bedeutet, erklären wir ausführlich in Kapitel 9.

3.3.4 Verhaltensmodellierung (action def, action)

Aktivitäten beschreiben Abläufe. Sie eignen sich für Prozesse wie ?Transportauftrag ausführen?, ?Zur Ladestation zurückkehren? oder ?Hindernis behandeln?. In SysML v2 werden Abläufe mit **action def** und **action** modelliert.

3.3.5 Zustandsmodellierung (state def, state)

Zustandsmodelle beschreiben Zustände und Übergänge. Ein Roboter besitzt Betriebszustände wie **Idle**, **Navigating**, **AvoidingObstacle** und **Charging**. In SysML v2 werden diese mit **state def** definiert.

3.3.6 Parametrische Modellierung (`calc` `def`, `constraint`)

Parametrische Modelle beschreiben mathematische Beziehungen. Sie helfen, technische Entscheidungen früh zu prüfen. In SysML v2 werden Berechnungen mit `calc` `def` und Bedingungen mit `constraint` ausgedrückt.

Verhalten, Zustände und Parametrik lernen Sie ab Kapitel 11 im Detail kennen. Für den ersten Einstieg in Kapitel 4 genügen zunächst Anforderungen und Struktur.

3.4 Modell statt Bild

Ein SysML-Diagramm ist nur eine Sicht auf ein Modell. Diese Unterscheidung ist zentral. Wenn Sie in einem Zeichenprogramm drei Kästen mit dem Namen `NavigationSystem` zeichnen, sind das drei unabhängige grafische Objekte. In einem Modellierungswerkzeug wie dem Syside Editor ist `NavigationSystem` ein einziges Modellelement, das in mehreren Sichten dargestellt werden kann.

Das hat praktische Vorteile:

- Namen bleiben konsistent.
- Beziehungen können ausgewertet werden.
- Änderungen sind leichter nachvollziehbar.
- Verschiedene Sichten zeigen denselben Inhalt.
- Traceability wird möglich.

3.5 Wie die Konzepte zusammenarbeiten

SysML-v2-Modellelemente sollten nicht isoliert entstehen. Sie ergänzen sich.

Beim Indoor-Roboter kann eine Modellkette so aussehen:

1. Im Anforderungsmodell steht: ?Der Roboter muss Hindernisse erkennen.?
2. `part def PerceptionSystem` wird als Systemteil definiert.
3. Ports und Verbindungen zeigen, wie Kamera und Lidar Daten liefern.
4. Eine `action def` beschreibt, wie ein Hindernis während der Fahrt behandelt wird.
5. Ein `state def` modelliert, dass der Roboter bei Gefahr in einen Stopp- oder Ausweichzustand wechselt.
6. Ein Testfall wird über `verify` mit der Anforderung verbunden.

So entsteht ein zusammenhängendes Modell statt einer losen Sammlung schöner Bilder. Abbildung 3.2 zeigt beispielhaft, wie Struktur- und Verbindungssicht des `PerceptionSystem` zusammenspielen: Die Übersicht zeigt die Subsysteme von `Robot`, die Detailsicht zeigt `PerceptionSystem` mit seinen Teilen und dem Port `sensorData`.

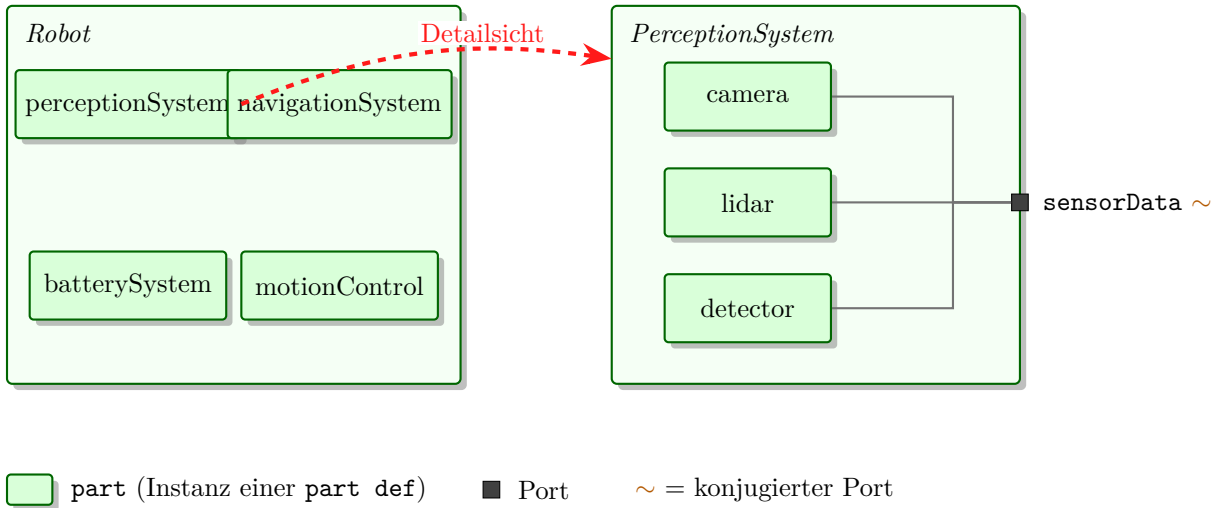


Figure 3.2: Struktursicht von `Robot` mit seinen Subsystemen und Detailsicht des `PerceptionSystem` mit Port `sensorData`.

3.6 SysML v1 und SysML v2 im Vergleich

Einsteiger stoßen in der Literatur häufig noch auf SysML v1. Die wichtigsten Begriffsänderungen sind:

SysML v1	SysML v2
Block (block)	<code>part def</code>
Block Definition Diagram (BDD)	Struktursicht (<code>part def</code> , <code>part</code>)
Internal Block Diagram (IBD)	Verbindungssicht (<code>port</code> , <code>connection</code>)
Activity Diagram	<code>action def</code> , <code>action</code>
State Machine Diagram	<code>state def</code> , <code>state</code>
Parametric Diagram	<code>calc def</code> , <code>constraint</code>
<code>deriveRqt</code>	<code>derive</code>

Dieses Buch verwendet durchgehend SysML v2. Wo alte Begriffe aus Kompatibilitätsgründen erwähnt werden, ist das explizit gekennzeichnet.

3.7 Typische Anfängerfehler

3.7.1 Zu viele Elemente zu früh

Einsteiger möchten oft sofort alle Konzepte verwenden. Das führt schnell zu Überforderung. Beginnen Sie mit Anforderungen, `part def` und einfachen Verbindungen. Verhalten und Parametrik können danach folgen.

3.7.2 Unklare Bedeutung von Systemteilen

Manchmal werden Funktionen, Softwaremodule, Hardwareteile und Rollen wild gemischt. Das ist nicht grundsätzlich verboten, muss aber bewusst geschehen. Eine `part def Camera` ist ein physisches Element. Eine `part def ObjectDetector` ist eher eine Software- oder Funktionskomponente.

3.7.3 Diagramme ohne Text

Ein Diagramm ohne Beschreibung ist oft schwer verständlich. Nutzen Sie Dokumentationsfelder (`doc /* ... */`). Schreiben Sie kurz, welchen Zweck ein Modellelement hat.

3.7.4 Keine Wiederverwendung

Wenn dasselbe Element mehrfach neu angelegt wird, verliert das Modell seinen Wert. Definieren Sie Typen mit `part def` einmal und instanziiieren Sie sie mit `part`.

3.8 Zusammenfassung

SysML v2 ist eine Modellierungssprache für technische Systeme. Sie hilft, Anforderungen, Struktur, Verhalten, Schnittstellen und Nachweise einheitlich zu beschreiben. Für dieses Buch sind besonders `requirement`, `part def`, `port`, `connection`, `action def`, `state def` und `calc def` wichtig. Entscheidend ist nicht das Zeichnen einzelner Diagramme, sondern der Aufbau eines zusammenhängenden Modells.

3.9 Kontrollfragen

1. Wofür steht SysML?
2. Was unterscheidet SysML v2 von SysML v1?
3. Was ist der Unterschied zwischen einem Diagramm und einem Modell?
4. Welche Fragen beantwortet ein Anforderungsmodell?
5. Was ist der Unterschied zwischen `part def` und `part`?
6. Wozu dienen `port` und `connection`?

7. Wann verwenden Sie `action def`?
8. Wann verwenden Sie `state def`?
9. Warum sollte man Modellelemente wiederverwenden?
10. Nennen Sie einen typischen Anfängerfehler bei SysML.

3.10 Übungen

Übung

Übung 1: Konzept zuordnen

Ordnen Sie folgende Fragen einem SysML-v2-Konzept zu: Was muss das System leisten? Aus welchen Teilen besteht es? Welche Daten fließen zwischen Komponenten? Welche Zustände kann der Roboter haben? Wie läuft ein Transportauftrag ab?

Übung

Übung 2: Erste Modellkette

Wählen Sie die Funktion `?Akkustand überwachen?`. Beschreiben Sie dazu eine Anforderung, eine `part def`, einen Port, einen möglichen Zustand und einen Testfall.

Übung

Übung 3: v1 vs. v2

Nennen Sie für BDD, IBD und Constraint Block jeweils das entsprechende SysML-v2-Konzept und erklären Sie in einem Satz den Unterschied.

3.11 Literaturhinweise

Für den Einstieg in SysML v2 sind die offizielle Spezifikation und das SysML-v2-Pilotprojekt grundlegend [?]. Der Syside Editor bietet eigene Dokumentation und Beispiele [?]. Ältere Einführungen wie Delligatti [?] und Friedenthal et al. [?] beziehen sich auf SysML v1 und sind nur noch als Hintergrundlektüre empfehlenswert.

4 Werkzeuge für SysML v2 einrichten

Lernziele

Nach diesem Kapitel können Sie erklären, warum dieses Lehrbuch eine textuelle SysML-v2-Arbeitsweise verwendet. Sie können ein einfaches Projekt in VS Code mit dem Syside Editor anlegen und SysML-v2-Dateien strukturieren. Außerdem können Sie Ziel, Umfang und Systemgrenze, Anforderungen, Part Definitions, Testfälle und ROS2-Topic-Zuordnungen in der SysML-v2-Textnotation formulieren.

4.1 Einleitung

SysML v2 kann grafisch und textuell verwendet werden. Für dieses Lehrbuch steht die textuelle Notation im Vordergrund. Das passt gut zu modernen Entwicklungsabläufen: Modelle liegen in Dateien, können versioniert, verglichen, überprüft und gemeinsam mit Code gepflegt werden.

Als kostenloser Einstieg eignet sich der Syside Editor für VS Code. Er unterstützt SysML-v2-Dateien mit Syntaxhervorhebung, Autovervollständigung, Formatierung, Gliederung und Validierung. Für grafische Sichten kann zusätzlich Eclipse SysON genutzt werden. SysON ist ein webbasiertes Open-Source-Werkzeug für SysML-v2-Modelle.

Hinweis

Ein Werkzeug löst keine Modellierungsaufgabe automatisch. Es hilft beim Schreiben, Prüfen und Anzeigen des Modells. Die fachliche Entscheidung bleibt Ihre Aufgabe: Welche Elemente gibt es? Welche Beziehungen sind wichtig? Welche Anforderungen müssen nachverfolgbar sein?

4.2 Warum textuelle Modellierung?

Viele Einsteiger erwarten bei SysML zuerst Diagramme. Diagramme bleiben wichtig, aber SysML v2 macht den Modellinhalt stärker als präzisen Text zugänglich. Ein Modell kann dadurch ähnlich behandelt werden wie Quellcode.

Textuelle Modellierung hat mehrere Vorteile:

- Modelle lassen sich in Git versionieren.

- Änderungen sind als Textvergleich sichtbar.
- Beispiele können direkt im Lehrbuch gezeigt werden.
- Modellteile können leichter wiederverwendet werden.
- Validierung und Automatisierung werden einfacher.
- ROS2-Code und SysML-v2-Modell können im selben Projekt liegen.

Das bedeutet nicht, dass Diagramme unwichtig werden. Diagramme sind weiterhin wertvolle Sichten. Der Unterschied ist: Das Modell entsteht nicht nur durch Zeichnen, sondern durch präzise definierte Modellelemente.

4.3 Empfohlene Werkzeugkette

Für dieses Lehrbuch verwenden wir eine einfache Werkzeugkette:

- VS Code als Editor
- Syside Editor als SysML-v2-Erweiterung
- Git für Versionsverwaltung
- optional Eclipse SysON für grafische Sichten
- optional das SysML-v2-Pilotprojekt als Referenz für fortgeschrittene Experimente

Diese Auswahl ist bewusst pragmatisch. Sie ist kostenlos zugänglich, passt zu textueller Modellierung und überfordert Einsteiger weniger als große industrielle Modellierungsumgebungen. Abbildung 4.1 zeigt diese Werkzeugkette im Zusammenhang, einschließlich des parallelen Pfads zum ROS2-Code.

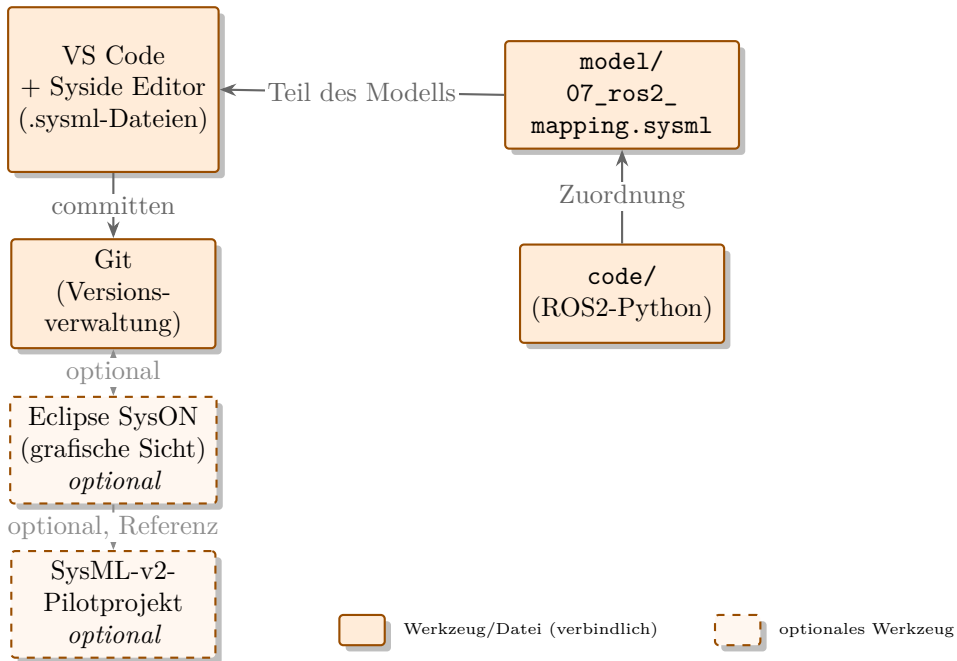


Figure 4.1: Empfohlene Werkzeugkette: VS Code mit Syside Editor und Git als Kern, optional ergänzt um Eclipse SysON und das SysML-v2-Pilotprojekt; parallel dazu der ROS2-Code.

4.4 Projekt anlegen

Für das Lehrbuch verwenden wir ein Projekt mit dem Namen `AutonomousIndoorRobot`. Der Name ist bewusst englisch, weil spätere ROS2-Pakete, Nodes und Topics ebenfalls englische technische Bezeichnungen verwenden. Die erklärenden Texte im Modell können trotzdem deutsch sein.

Eine einfache Projektstruktur kann so aussehen:

```

AutonomousIndoorRobot/
  model/
    00_overview.sysml
    01_context.sysml
    02_requirements.sysml
    03_structure.sysml
    04_behavior.sysml
    05_interfaces.sysml
    06_tests.sysml
    07_ros2_mapping.sysml
  code/
    status_node.py
    battery_monitor_node.py

```



```
docs/
  decisions.md
```

Die Dateien im Ordner `model` enthalten das SysML-v2-Modell. Der Ordner `code` enthält spätere ROS2-Beispiele. Der Ordner `docs` kann kurze Architekturentscheidungen aufnehmen. Abbildung 4.2 stellt diese Projektstruktur grafisch dar und ordnet jeder Modell-Datei ein kurzes Stichwort zu.

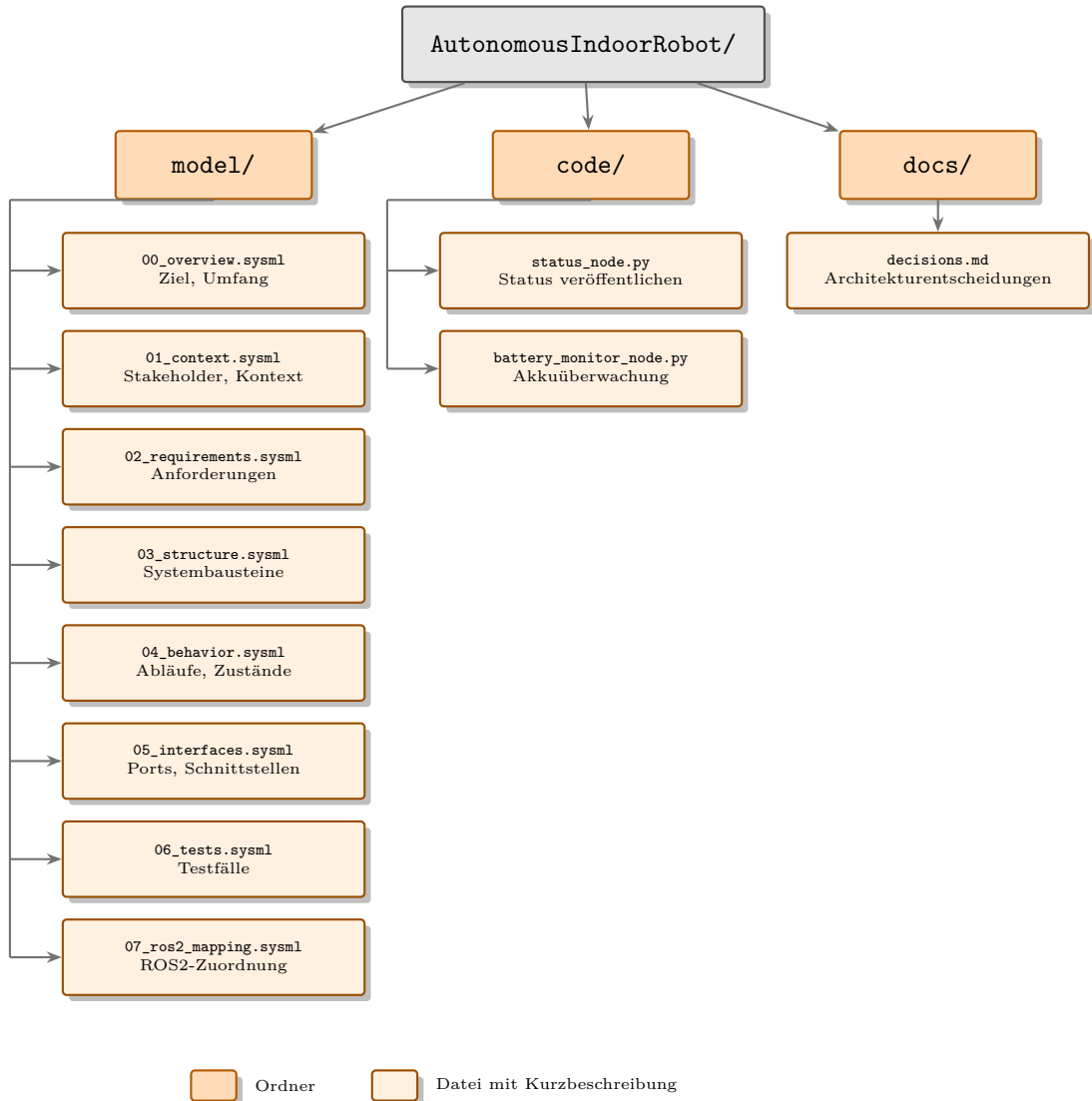


Figure 4.2: Projektstruktur `AutonomousIndoorRobot` mit den Ordnern `model/`, `code/` und `docs/` sowie den wichtigsten Dateien.

4.5 Ziel, Umfang und Systemgrenze beschreiben

Die erste Datei `00_overview.sysml` dient dazu, das Projekt zu rahmen. Sie beantwortet drei grundlegende Fragen:

- **Ziel:** Wofür ist das System da?
- **Umfang:** Was gehört dazu, was nicht?
- **Systemgrenze:** Wo endet das System und wo beginnt die Umgebung?

In SysML v2 beginnt jede Datei mit einem **package**. Das Package ist der benannte Modellraum. Innerhalb davon beschreiben wir Ziel und Umfang als Dokumentation mit `doc /* ... */`. Die Systemgrenze zeigt sich darin, was wir als **part def** definieren und was wir als externen Akteur behandeln.

Beachten Sie: In `doc`-Kommentaren schreiben wir Umlaute bewusst aus (`ae`, `oe`, `ue`), um Encoding-Probleme zwischen unterschiedlichen Werkzeugen und Betriebssystemen zu vermeiden.

```

1 package AutonomousIndoorRobot {
2   doc
3   /* Ziel: Ein autonomer Indoor-Roboter transportiert kleine
4      Gegenstaende innerhalb eines Gebaeudes zu ausgewaehlten Zielen.
5      Er faehrt selbststaendig, erkennt Hindernisse und kehrt bei
6      niedrigem Akkustand zur Ladestation zurueck.
7
8      Umfang: Dieses Modell beschreibt Navigation, Wahrnehmung,
9      Energieversorgung, Bedienung und Sicherheit des Roboters.
10
11     Systemgrenze: Zur Ladestation und zum Gebaeude gehoert die
12     physische Infrastruktur, die der Roboter nutzt, aber nicht
13     selbst steuert. Personen sind externe Akteure. */
14
15   part def Robot;
16   part def ChargingStation; // extern, aber modellrelevant
17 }
```

Die `part def Robot` zeigt: Das System ist der Roboter. Die `ChargingStation` taucht als externe Komponente auf, weil der Roboter sie anfahren muss, sie aber nicht Teil des Roboters ist. Diese Entscheidung muss bewusst getroffen und dokumentiert werden.

Praxisbeispiel

Häufiger Anfängerfehler: Systemgrenze unklar lassen. Wenn Sie nicht entscheiden, ob die Ladestation ?im System? oder ?außen? ist, entstehen später Widersprüche in Anforderungen und Tests. Eine kurze `doc`-Notiz am Anfang verhindert das.

Diese Darstellung ist für den Einstieg vereinfacht: **ChargingStation** steht hier als **part def** auf derselben Ebene wie **Robot**. Sauberer wäre es, externe Systeme in einem eigenen Kontext-Package zu modellieren, um die Systemgrenze auch strukturell sichtbar zu machen.

4.6 Anforderungen formulieren

Anforderungen beschreiben, was das System leisten muss, nicht wie. In SysML v2 werden sie in der Datei `02_requirements.sysml` mit dem Schlüsselwort **requirement** definiert.

Jede Anforderung bekommt einen eindeutigen Namen und eine Beschreibung als **doc-Block**. Anforderungen können geschachtelt werden, um Hierarchien auszudrücken.

```

1 package Requirements {
2
3   requirement REQ_Navigate {
4     doc /* Der Roboter muss in einem bekannten Innenraumbereich
5         autonom ein ausgewaehltes Ziel anfahren koennen. */
6   }
7
8   requirement REQ_Safety {
9     doc /* Sicherheitsanforderungen fuer den Betrieb
10        in der Naehе von Personen. */
11
12     requirement REQ_DetectObstacles {
13       doc /* Der Roboter muss Hindernisse im Fahrweg erkennen. */
14     }
15
16     requirement REQ_StopSafely {
17       doc /* Der Roboter muss bei einem Hindernis in 0,5 m
18          Abstand bremsen und bei 0,2 m stoppen. */
19     }
20   }
21
22   requirement REQ_Battery {
23     doc /* Akkumanagement und Energieversorgung. */
24
25     requirement REQ_MonitorBattery {
26       doc /* Der Roboter muss seinen Akkustand kontinuierlich
27          ueberwachen. */
28     }
29
30     requirement REQ_ReturnToDock {
31       doc /* Der Roboter muss bei einem Akkustand unter 20 Prozent
32          selbststaendig zur Ladestation fahren. */
33     }
34   }
35 }
```

Diese Schachtelung ist zunächst eine reine Namensraum-Gruppierung: Sie ordnet verwandte Anforderungen thematisch unter einer übergeordneten Anforderung ein, erzeugt aber keine automatische **derive**-Beziehung zwischen übergeordneter und untergeordneter Anforderung; eine solche Ableitungsbeziehung müssten Sie bei Bedarf zusätzlich explizit mit **derive** modellieren.

Lesen Sie jede Anforderung laut vor und fragen Sie: Kann ich das testen? Wenn nicht, ist sie zu vage. *Der Roboter muss navigieren?* ist nicht testbar. *Der Roboter muss bei 20 Prozent Akkustand die Ladestation anfahren?* ist testbar.

4.7 Part Definitions formulieren

Part Definitions beschreiben die Bausteine des Systems. In SysML v2 definiert **part def** einen Typ. **part** erzeugt eine Instanz davon. Die Datei `03_structure.sysml` enthält die Strukturmodellierung.

Beginnen Sie mit dem Hauptelement **Robot** und listen Sie seine Hauptteile auf. Jeder Teil wird zuerst grob benannt. Die Details kommen schrittweise.

```

1 package Structure {
2
3   part def Robot {
4     doc /* Das Gesamtsystem. Enthaelte alle funktionalen Subsysteme. */
5
6     part perception : PerceptionSystem;
7     part navigation : NavigationSystem;
8     part motion : MotionControl;
9     part battery : BatterySystem;
10    part ui : UserInterface;
11    part safety : SafetyLogic;
12  }
13
14  part def PerceptionSystem {
15    doc /* Nimmt Sensordaten auf und verarbeitet sie zu
16        Objektlisten und Hindernissignalen. */
17
18    part frontCamera : Camera;
19    part lidar : Lidar;
20    part detector : ObjectDetector;
21  }
22
23  part def BatterySystem {
24    doc /* Ueberwacht den Akkustand und meldet kritische Werte. */
25
26    part cell : Battery;
27    part monitor : BatteryMonitor;
28    part charger : ChargingInterface;
29  }
30
31  part def NavigationSystem;
```

```

32 part def MotionControl;
33 part def UserInterface;
34 part def SafetyLogic;
35 part def Camera;
36 part def Lidar;
37 part def ObjectDetector;
38 part def Battery;
39 part def BatteryMonitor;
40 part def ChargingInterface;
41 }

```

Abbildung 4.3 stellt diesen Strukturbaum grafisch dar.

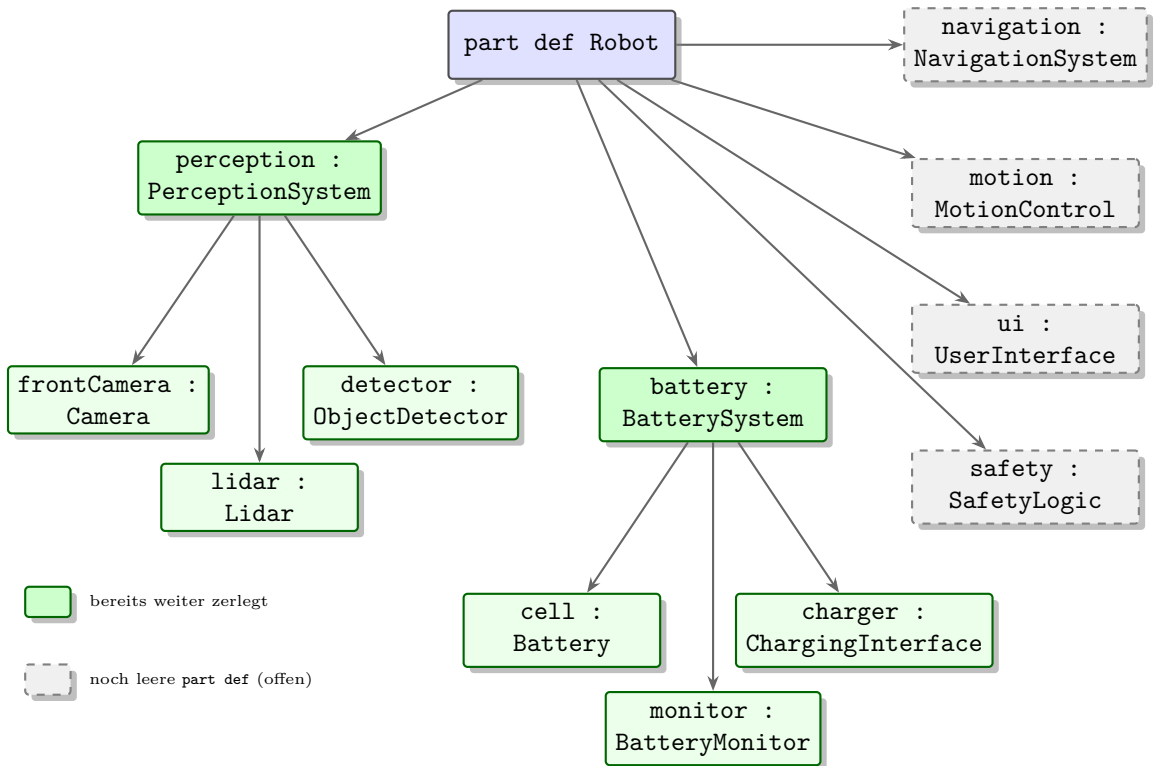


Figure 4.3: Strukturbaum des Roboters mit `part def Robot` und seinen Subsystemen. Grün hinterlegte Kästen sind im Modell bereits weiter zerlegt, grau hinterlegte Kästen sind an dieser Stelle noch leere `part defs`.

Der `doc`-Block bei jeder `part def` ist optional, aber empfohlen. Er hält fest, welche Verantwortung dieser Baustein trägt. Wenn Sie das nicht in einem Satz beschreiben können, ist die Verantwortung noch unklar.

4.8 Testfälle anlegen

Testfälle überprüfen, ob das System eine Anforderung erfüllt. Mit `verification def` wird der Testfall selbst definiert, mit dem Schlüsselwort `verify` wird anschließend die Verbindung zwischen diesem Testfall und einer Anforderung ausgedrückt. Die Datei `06_tests.sysml` enthält die Testmodellierung.

Ein Testfall beschreibt: Was wird getestet? Welche Vorbedingungen gelten? Was ist die erwartete Reaktion? Welche Anforderung wird damit geprüft?

```

1 package Tests {
2
3   verification def TC_BatteryLowReturn {
4     doc /* Test: Roboter kehrt bei niedrigem Akkustand zurueck.
5        Vorbedingung: Roboter navigiert, Akkustand betraegt 25 Prozent.
6        Aktion: Akkustand sinkt auf 18 Prozent.
7        Erwartung: Roboter bricht Navigation ab und faehrt Ladestation an.
8        Akzeptanzkriterium: Ladestation erreicht innerhalb von 120 Sekunden.
9        */
10
11   }
12
13   verification def TC_ObstacleStop {
14     doc /* Test: Roboter haelte vor Hindernis an.
15        Vorbedingung: Roboter faehrt mit 0,3 m/s.
16        Aktion: Hindernis erscheint in 0,4 m Abstand.
17        Erwartung: Roboter bremst und stoppt innerhalb von 0,2 m
18           Sicherheitsabstand.
19        Akzeptanzkriterium: Keine Kollision, Stopp in weniger als 2 Sekunden.
20        */
21
22   }
23
24   part robot : Robot {
25     verify REQ_ReturnToDock by tc1 : TC_BatteryLowReturn;
26     verify REQ_StopSafely by tc2 : TC_ObstacleStop;
27   }
28 }

```

Die `verify`-Beziehung verbindet den Testfall mit der Anforderung. Das ist der Kern der Nachvollziehbarkeit: Jede Anforderung sollte mindestens einen Testfall haben, der sie prüft. Anforderungen ohne Test bleiben im Projekt unkontrolliert.

Die genaue Syntax von `verify ... by` kann sich zwischen SysML-v2-Werkzeugversionen leicht unterscheiden, da die Sprache noch in Entwicklung ist. Prüfen Sie im Zweifel gegen Ihre installierte Werkzeugversion.

4.9 ROS2-Topics zuordnen

Die Datei `07_ros2_mapping.sysml` verbindet das SysML-Modell mit der späteren ROS2-Implementierung. Für jeden Systemteil wird dokumentiert, welcher ROS2-Node ihn umsetzt und welche Topics er verwendet.

In SysML v2 können solche Zuordnungen als Attribute oder als Dokumentation modelliert werden. Das Schlüsselwort **attribute** definiert dabei eine benannte, typisierte Eigenschaft eines Modellelements, hier jeweils vom Typ **String**. Für den Einstieg reicht eine klare Textnotation:

```

1 package ROS2Mapping {
2
3   part def BatterySystem_ROS2 {
4     doc /* ROS2-Umsetzung des BatterySystem.
5
6       Node: battery_monitor_node
7       Paket: autonomous_indoor_robot/battery_monitor
8
9       Publizierte Topics:
10        /battery_state (sensor_msgs/BatteryState) -- Akkustand, 1 Hz
11
12       Abonnierte Topics:
13        keine
14
15       Verbindung zum Modell:
16        BatterySystem.monitor -> battery_monitor_node
17        REQ_MonitorBattery wird durch diesen Node erfuehlt. */
18
19     attribute rosNode : String = "battery_monitor_node";
20     attribute topicOut : String = "/battery_state";
21   }
22
23   part def NavigationSystem_ROS2 {
24     doc /* ROS2-Umsetzung des NavigationSystem.
25
26       Nodes: nav2_stack (extern), path_planner_node
27       Abonnierte Topics:
28        /battery_state -- triggert Rueckkehr bei niedrigem Akku
29        /scan (sensor_msgs/LaserScan) -- Lidardaten
30        /map (nav_msgs/OccupancyGrid) -- Karte
31
32       Publizierte Topics:
33        /cmd_vel (geometry_msgs/Twist) -- Fahrbefehle
34        /robot_status (std_msgs/String) -- Betriebszustand
35
36       Verbindung zum Modell:
37        REQ_Navigate und REQ_ReturnToDock werden durch diese Nodes erfuehlt.
38        */
39
40     attribute rosNode : String = "nav2_stack";
41     attribute topicIn : String = "/battery_state, /scan, /map";
42     attribute topicOut : String = "/cmd_vel, /robot_status";
43   }
44 }
```

Für den Einstieg vereinfacht: Mehrere Topics in einem String zusammenzufassen ist praktisch, aber unsauber. Sauberer wäre ein eigenes Attribut je Topic (z. B. `topicScan`, `topicMap`), damit jedes Topic einzeln typisiert und nachverfolgt werden kann.

Diese Datei wird im Laufe des Projekts wachsen. Beginnen Sie mit den Nodes, die Sie als Erstes implementieren. Die Verbindung zwischen SysML-Systemelement und ROS2-Node sollte immer explizit dokumentiert sein.

4.10 Alles zusammen: Ein Minimalmodell

Nach den ersten Schritten sieht ein Minimalmodell für den Indoor-Roboter so aus:

Datei	Inhalt	Schlüsselwörter
00_overview.sysml	Ziel, Umfang, Systemgrenze	package, doc, part def
02_requirements.sysml	Anforderungen	requirement, doc
03_structure.sysml	Systembausteine	part def, part
06_tests.sysml	Testfälle	verification def, verify
07_ros2_mapping.sysml	ROS2-Zuordnung	attribute, doc

Dieses Minimalmodell ist kein perfektes Systemmodell. Es ist ein Ausgangspunkt. Jede Datei kann mit wenigen Zeilen begonnen und schrittweise erweitert werden.

4.11 Empfohlene Modellstruktur

Die Modellstruktur orientiert sich an den wichtigsten Fragen des Projekts:

- 00_overview.sysml: Ziel, Umfang und Systemgrenze
- 01_context.sysml: Stakeholder, externe Systeme und Umgebung
- 02_requirements.sysml: Anforderungen und Beziehungen
- 03_structure.sysml: Systemteile, Subsysteme und Komponenten
- 04_behavior.sysml: Aktionen, Abläufe und Zustände
- 05_interfaces.sysml: Ports, Verbindungen und Datenflüsse
- 06_tests.sysml: Testfälle und Verifikation
- 07_ros2_mapping.sysml: Zuordnung zu ROS2-Nodes, Topics, Services und Actions

Diese Struktur ist bewusst schlicht. Sie trennt die wichtigsten Modellbereiche, ohne schon zu Beginn ein kompliziertes Metamodell einzuführen.

4.12 Namenskonventionen

Klare Namen machen Modelle lesbar. Für dieses Lehrbuch verwenden wir einfache Regeln:

- Packages: `AutonomousIndoorRobot`
- Part Definitions: `Robot`, `BatterySystem`, `NavigationSystem`
- Parts (Instanzen): `battery`, `navigation`, `motion`
- Anforderungen: `REQ_DetectObstacles`, `REQ_ReturnToDock`
- Testfälle: `TC_BatteryLowReturn`, `TC_ObstacleStop`
- ROS2-Nodes: `battery_monitor_node`, `path_planner_node`
- ROS2-Topics: `/battery_state`, `/cmd_vel`, `/robot_status`

Vermeiden Sie wechselnde Namen für dasselbe Element. Wenn ein Element einmal `BatterySystem` heißt, sollte es nicht an anderer Stelle `PowerModule` genannt werden, außer Sie modellieren bewusst unterschiedliche Dinge.

4.13 Grafische Sichten

Grafische Sichten sind besonders hilfreich, wenn ein Team schnell über Struktur oder Schnittstellen sprechen möchte. In SysML v2 sollte man sie als Darstellung des Modells verstehen, nicht als Ersatz für das Modell.

Eclipse SysON kann für solche Sichten interessant sein. Es ist besonders dann nützlich, wenn Sie aus dem textuellen Modell heraus Struktur-, Verbindungs- oder Anforderungssichten visualisieren wollen. Für den Einstieg in diesem Buch reicht es aber, zunächst die textuelle Notation sauber zu verstehen.

Wichtig ist dabei die Blickrichtung: Der Text bleibt die Quelle der Wahrheit, die Grafik ist eine automatisch erzeugte Sicht darauf. Wenn Sie im Syside Editor eine neue `part` in `Robot` ergänzen und speichern, kann SysON daraus die passende Struktursicht neu erzeugen, ohne dass Sie selbst einen Kasten zeichnen müssen. [Abbildung 4.4](#) veranschaulicht diesen Zusammenhang am Beispiel der `part def Robot` aus dem vorherigen Abschnitt.

**VS Code mit Syside
Editor (Text)**

**Eclipse SysON
(generierte Sicht)**

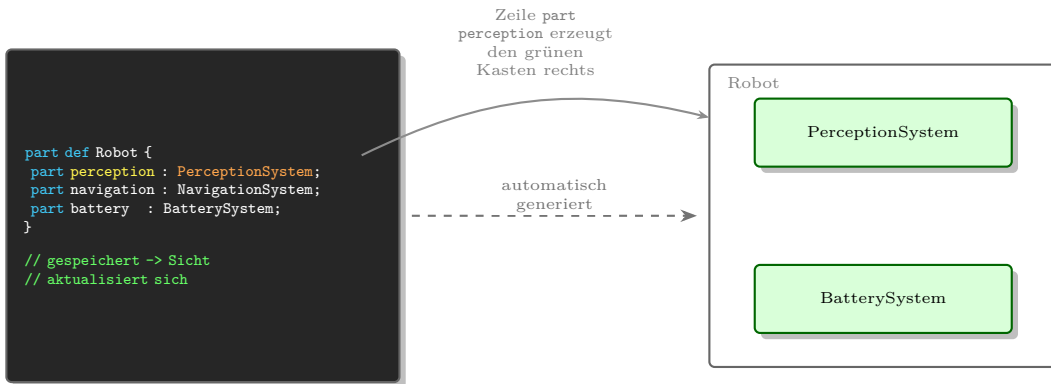


Figure 4.4: Der SysML-v2-Text im Syside Editor (links) ist die Quelle der Wahrheit; Eclipse SysON erzeugt daraus automatisch die grafische Strukturansicht (rechts), ohne dass diese von Hand gezeichnet werden muss.

Hinweis

Aus diesem Grund lohnt es sich bei Konflikten zwischen Text und Grafik immer, zuerst den Text zu prüfen. Ein Diagramm, das von Hand *korrigiert* wurde, weicht sonst unbemerkt vom eigentlichen Modell ab.

4.14 Typische Anfängerfehler

4.14.1 Systemgrenze nicht klären

Viele Modelle beginnen ohne eine klare Antwort auf die Frage: Was gehört zum System? Diese Entscheidung prägt alle späteren Anforderungen und Tests. Dokumentieren Sie die Systemgrenze explizit in der Übersichtsdatei.

4.14.2 Anforderungen nicht testbar formulieren

Anforderungen wie *Der Roboter soll gut navigieren* sind nicht prüfbar. Formulieren Sie immer mit konkreten Bedingungen und messbaren Erwartungen.

4.14.3 Part Definitions ohne Verantwortung

Eine `part def` ohne `doc`-Block lässt offen, wozu dieser Baustein da ist. Schreiben Sie für jeden Baustein mindestens einen Satz über seine Aufgabe.

4.14.4 Testfälle ohne Verbindung zu Anforderungen

Ein Testfall, der keine Anforderung prüft, ist nutzlos für die Nachvollziehbarkeit. Jede `verification def` sollte mindestens eine `verify`-Beziehung haben.

4.14.5 ROS2-Topics nicht dokumentieren

Der Name eines Topics allein reicht nicht. Welcher Node sendet? Welcher empfängt? Welches Nachrichtenformat? Diese Angaben gehören in die Mapping-Datei.

4.14.6 Text wie Programmcode ohne Bedeutung schreiben

SysML-v2-Text sieht teilweise wie Code aus. Trotzdem modellieren Sie kein Programm, sondern ein System. Fragen Sie bei jedem Element: Welche fachliche Aussage steckt dahinter?

4.15 Zusammenfassung

Dieses Kapitel hat gezeigt, wie ein SysML-v2-Projekt von Anfang an strukturiert wird. Ziel, Umfang und Systemgrenze werden im Übersichtspaket als `doc`-Blöcke festgehalten. Anforderungen werden als `requirement` mit prüfbarem Text formuliert. Part Definitions beschreiben die Systembausteine mit `part def`. Testfälle verbinden sich über `verify` mit Anforderungen. ROS2-Topics werden in einer Mapping-Datei als `attribute` und Dokumentation zugeordnet. VS Code mit dem Syside Editor ist die empfohlene kostenlose Arbeitsumgebung. Entscheidend bleibt nicht das Werkzeug, sondern ein verständliches, konsistentes und nachverfolgbares Modell.

4.16 Kontrollfragen

1. Warum eignet sich textuelle Modellierung gut für SysML v2?
2. Welche drei Fragen beantwortet die Datei `00_overview.sysml`?
3. Was ist der Unterschied zwischen `part def` und `part`?
4. Was macht eine Anforderung testbar?
5. Wie verbindet SysML v2 einen Testfall mit einer Anforderung?
6. Welche Informationen gehören in die ROS2-Mapping-Datei?
7. Warum sollte jede `part def` eine `doc`-Notiz haben?
8. Warum ersetzt Werkzeugvalidierung keine fachliche Modellprüfung?

4.17 Übungen

Übung

Übung 1: Systemgrenze beschreiben

Erstellen Sie die Datei `00_overview.sysml`. Beschreiben Sie Ziel, Umfang und Systemgrenze des autonomen Indoor-Roboters im `doc`-Block des Hauptpakets. Entscheiden Sie bewusst: Gehört die Ladestation zum System oder ist sie extern? Begründen Sie Ihre Entscheidung in einem Satz.

Übung

Übung 2: Anforderungen formulieren

Erstellen Sie die Datei `02_requirements.sysml` mit mindestens fünf Anforderungen. Verwenden Sie die Namen `REQ_Navigate`, `REQ_DetectObstacles`, `REQ_StopSafely`, `REQ_MonitorBattery` und `REQ_ReturnToDock`. Formulieren Sie jeden `doc`-Block so, dass die Anforderung messbar und testbar ist.

Übung

Übung 3: Part Definitions anlegen

Erstellen Sie die Datei `03_structure.sysml` mit einer `part def Robot`, die mindestens vier Subsysteme als `part` enthält. Zerlegen Sie das `PerceptionSystem` weiter in Kamera, Lidar und Detektor. Schreiben Sie für jeden Baustein eine kurze `doc`-Notiz.

Übung

Übung 4: Testfälle anlegen

Erstellen Sie die Datei `06_tests.sysml` mit mindestens zwei Testfällen. Beschreiben Sie für jeden Testfall Vorbedingung, Aktion und erwartetes Ergebnis im `doc`-Block. Verbinden Sie jeden Testfall über `verify` mit der zugehörigen Anforderung.

Übung

Übung 5: ROS2-Topics zuordnen

Erstellen Sie die Datei `07_ros2_mapping.sysml`. Ordnen Sie dem `BatterySystem` den Node `battery_monitor_node` und das Topic `/battery_state` zu. Ordnen Sie dem `NavigationSystem` mindestens zwei Topics zu. Dokumentieren Sie Sender, Empfänger und Nachrichtenformat.

4.18 Literaturhinweise

Für die praktische Arbeit mit SysML v2 sind die Dokumentation des Syside Editors, das Eclipse-SysON-Projekt und das SysML-v2-Pilotprojekt hilfreich [?, ?, ?]. Für die fachliche Einordnung bleiben klassische SysML- und MBSE-Grundlagen nützlich, müssen aber bewusst auf SysML v2 übertragen werden.

5 Das Praxisprojekt: Autonomer Indoor-Roboter

Lernziele

Nach diesem Kapitel kennen Sie das durchgängige Beispielsystem dieses Buches. Sie können Ziel, Systemkontext, Hauptfunktionen, Annahmen und Randbedingungen des autonomen Indoor-Roboters beschreiben. Außerdem verstehen Sie, warum dieses Beispiel gut geeignet ist, um SysML v2, MBSE, ROS2 und Machine Learning gemeinsam zu lernen.

5.1 Einleitung

Dieses Lehrbuch verwendet ein durchgängiges Praxisbeispiel: einen autonomen Indoor-Roboter. An diesem Beispiel lernen Sie, wie Systems Engineering, MBSE, SysML v2 und ROS2 zusammenhängen. Das Beispiel ist bewusst realistisch, aber didaktisch begrenzt.

Der Roboter soll sich in Innenräumen bewegen, Ziele anfahren, Hindernisse erkennen, Personen erkennen, seinen Akkustand überwachen und bei niedrigem Akkustand automatisch zur Ladestation zurückkehren. Damit verbindet das Projekt typische Themen moderner Robotik: Sensorik, Aktorik, Navigation, Softwarearchitektur, KI, Schnittstellen, Tests und Systemmodellierung.

5.2 Ziel des Systems

Das Zielsystem kann in einem Satz beschrieben werden:

Der autonome Indoor-Roboter transportiert kleine Gegenstände innerhalb eines definierten Gebäudebereichs selbstständig, sicher und nachvollziehbar zu ausgewählten Zielpunkten.

Dieser Satz enthält mehrere wichtige Begriffe. **„Autonom“** bedeutet hier nicht grenzenlose Selbstständigkeit. Der Roboter arbeitet innerhalb eines bekannten oder kartierten Innenbereichs. **„Sicher“** bedeutet, dass Personen und Gegenstände nicht gefährdet werden sollen. **„Nachvollziehbar“** bedeutet, dass Zustände, Fehler und Entscheidungen im Modell und später im System beobachtbar bleiben.

5.3 Warum dieses Beispiel?

Das Beispiel ist für den Einstieg besonders geeignet, weil es mehrere Disziplinen verbindet, ohne sofort zu groß zu werden.

- Es enthält Hardware: Sensoren, Antrieb, Batterie, Gehäuse.
- Es enthält Software: ROS2-Nodes, Topics, Services und Actions (diese Begriffe werden in Kapitel 14 näher erklärt).
- Es enthält Verhalten: Navigation, Rückkehr zur Ladestation, Fehlerbehandlung.
- Es enthält Schnittstellen: Datenflüsse, Energiefluss, Bedienung.
- Es enthält KI: Personenerkennung mit einem ML-Modell.
- Es enthält Tests: Anforderungen müssen nachgewiesen werden.

Gleichzeitig bleibt der Umfang überschaubar. Wir modellieren keinen kompletten industriellen Lieferroboter mit Zulassung, Produktvarianten und Serienfertigung. Wir modellieren ein Lernsystem, an dem die wichtigsten Konzepte sichtbar werden.

5.4 Systemkontext

Der Roboter interagiert mit seiner Umgebung. Zum Kontext gehören:

- Nutzer, die Transportaufträge starten
- Personen im Gebäude
- Räume, Flure, Türen und Hindernisse
- eine Ladestation
- Wartungspersonal
- eventuell ein WLAN oder Gebäudenetzwerk
- ein Bediengerät oder eine Benutzeroberfläche

Nicht alle diese Elemente gehören automatisch zum System. Die Systemgrenze muss bewusst festgelegt werden. Für das Lehrbuch betrachten wir den Roboter als Hauptsystem. Die Ladestation kann je nach Kapitel als externes System oder als Teil der Gesamtarchitektur betrachtet werden, weil gerade diese Entscheidung ein gutes Beispiel für Schnittstellen ist. Eine bewusst gezogene Systemgrenze ist außerdem die Voraussetzung dafür, dass die Anforderungen (Kapitel 6) und die Struktur (Kapitel 8) des Roboters später sauber modelliert werden können.

Abbildung ?? fasst diesen Systemkontext grafisch zusammen: Der Roboter steht im Zentrum, alle genannten Elemente umgeben ihn als externe Kontextelemente.

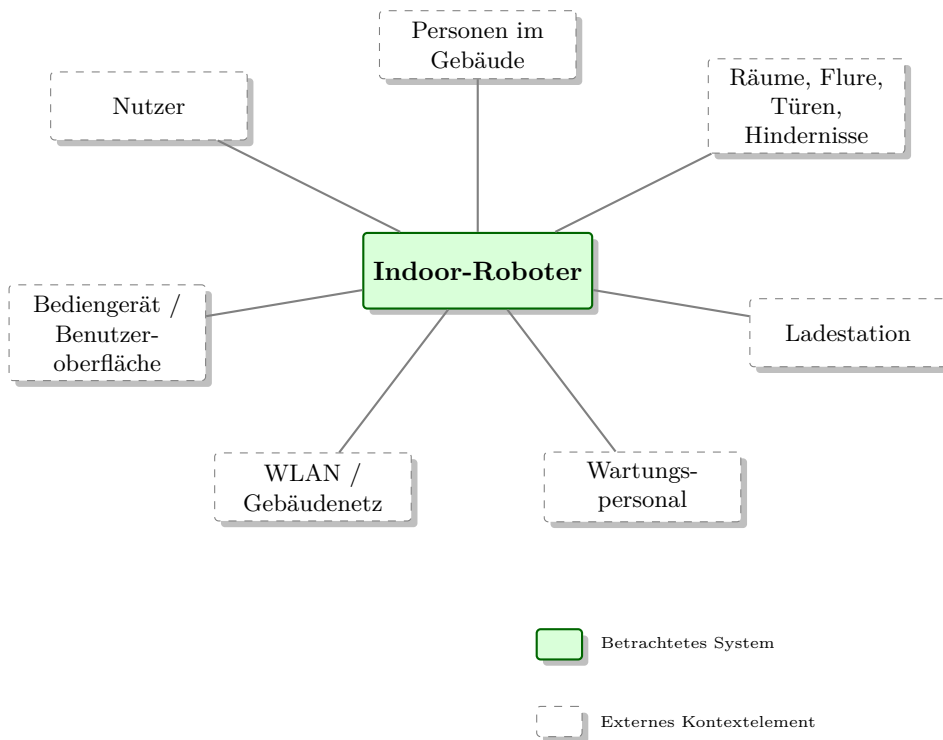


Figure 5.1: Kontextdiagramm des Indoor-Roboters mit den wichtigsten externen Elementen seiner Betriebsumgebung.

5.5 Hauptfunktionen

Die wichtigsten Funktionen des Roboters sind:

- Transportauftrag entgegennehmen
- Karte erstellen oder laden
- eigene Position bestimmen
- Navigationsziel empfangen
- Pfad planen
- Bewegung ausführen
- Hindernisse erkennen und vermeiden
- Personen erkennen und berücksichtigen
- Batterie überwachen
- Ladestation anfahren

- Betriebszustand melden
- Fehler erkennen und sicher reagieren

Diese Liste ist noch keine technische Architektur. Sie beschreibt zunächst, was das System leisten muss. Erst später entscheiden wir, welche Systemteile (**part def**, ausführlich in Kapitel 8), ROS2-Nodes und Schnittstellen diese Funktionen realisieren.

5.6 Annahmen für das Lehrbuch

Damit das Beispiel beherrschbar bleibt, treffen wir einige Annahmen:

- Der Roboter fährt ausschließlich in Innenräumen.
- Der Boden ist im Wesentlichen eben.
- Der Roboter bewegt sich mit niedriger Geschwindigkeit.
- Die Umgebung kann kartiert werden.
- Der Roboter besitzt Kamera, Lidar, mobilen Rechner, Motorcontroller und Batterie.
- Als Softwarebasis verwenden wir ROS2.
- Für Personenerkennung wird später ein ML-Modell eingesetzt.
- Sicherheitsbetrachtungen werden didaktisch behandelt, aber nicht normgerecht vollständig ausgearbeitet.

Hinweis

Das Projekt ist didaktisch vereinfacht. Ein echter Roboter, der in öffentlichen oder sicherheitskritischen Bereichen eingesetzt wird, benötigt deutlich umfangreichere Risikoanalysen, Sicherheitskonzepte, Normenbetrachtungen und Zulassungsprozesse.

5.7 Erste Systemstruktur

Eine erste grobe Struktur könnte aus folgenden Teilsystemen bestehen:

- **PerceptionSystem**: Kamera, Lidar, Objekterkennung, Hinderniserkennung
- **NavigationSystem**: Lokalisierung, Karte, Pfadplanung
- **MotionControl**: Umsetzung von Bewegungsbefehlen

- **BatterySystem:** Akku, Ladezustand, Energiemanagement
- **UserInterface:** Auftragseingabe und Statusanzeige
- **CommunicationSystem:** interne und externe Kommunikation
- **SafetyLogic:** sichere Reaktion auf kritische Situationen

Diese Struktur wird in den folgenden Kapiteln schrittweise verfeinert. Abbildung ?? zeigt diese erste, noch flache Gliederung als einfachen Baum.

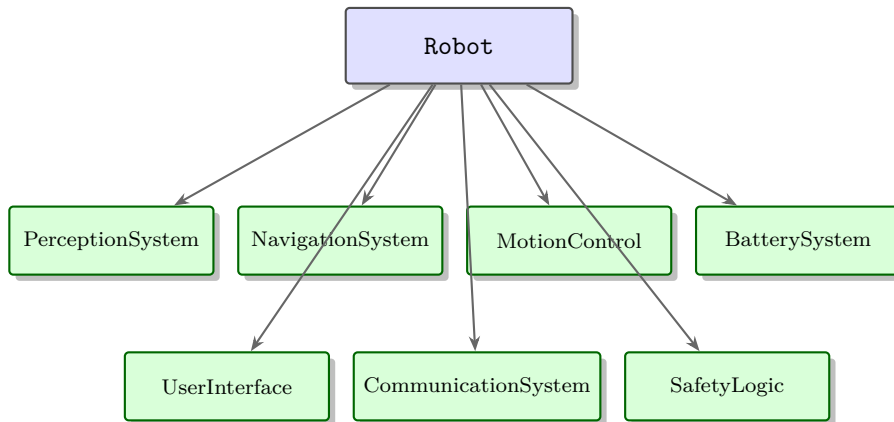


Figure 5.2: Erste, noch grobe Systemstruktur des Roboters mit sieben gleichrangigen Teilsystemen.

5.8 Erste Szenarien

5.8.1 Szenario 1: Ziel anfahren

Ein Nutzer wählt ein Ziel aus. Der Roboter prüft, ob eine Karte verfügbar ist, plant einen Pfad und fährt los. Während der Fahrt verarbeitet er Sensorinformationen, erkennt Hindernisse und passt seine Bewegung an. Am Ziel meldet er den erfolgreichen Abschluss.

5.8.2 Szenario 2: Hindernis im Fahrweg

Während der Navigation erkennt der Roboter ein Hindernis. Er bewertet, ob Ausweichen möglich ist. Falls ja, plant er lokal eine neue Bewegung. Falls nicht, stoppt er und meldet eine Blockade.

5.8.3 Szenario 3: Niedriger Akkustand

Der Batteriemonitor erkennt einen niedrigen Ladezustand. Der Roboter bricht seine aktuelle Mission ab, informiert den Nutzer und plant die Rückkehr zur Ladestation.

5.8.4 Szenario 4: Personenerkennung

Die Kamera liefert Bilder an einen ML-basierten Erkennungs-Node. Erkennt das System eine Person im Fahrbereich, reduziert der Roboter seine Geschwindigkeit oder wählt einen anderen Pfad.

5.9 Was wir im Buch daraus machen

Das Praxisprojekt dient als roter Faden:

- Kapitel 6 formuliert Anforderungen.
- Kapitel 7 betrachtet Stakeholder und Use Cases.
- Kapitel 8 und 9 modellieren Struktur und interne Verbindungen.
- Kapitel 10 beschreibt Schnittstellen.
- Kapitel 11 und 12 modellieren Verhalten.
- Kapitel 14 bis 18 übertragen das Modell in ROS2 und ML.
- Kapitel 19 und 20 verbinden Anforderungen, Tests und Traceability (die Nachvollziehbarkeit von Anforderungen über das gesamte Modell hinweg).

5.10 Zusammenfassung

Der autonome Indoor-Roboter ist klein genug für den Einstieg und realistisch genug, um Systems Engineering sinnvoll zu machen. Er verbindet Anforderungen, Struktur, Verhalten, Schnittstellen, ROS2-Architektur, KI und Tests. Die in diesem Kapitel beschriebenen Annahmen und Funktionen bilden die Grundlage für alle folgenden Modellierungsschritte.

5.11 Kontrollfragen

1. Was ist das Ziel des autonomen Indoor-Roboters?
2. Warum eignet sich dieses Beispiel für ein SysML-Lehrbuch?
3. Welche externen Elemente gehören zum Systemkontext?
4. Nennen Sie fünf Hauptfunktionen des Roboters.
5. Welche Annahmen werden für das Lehrbuch getroffen?
6. Warum ist die Ladestation ein interessantes Beispiel für Systemgrenzen?
7. Welche Rolle spielt ROS2 im Praxisprojekt?
8. Welche Rolle spielt Machine Learning im Praxisprojekt?

5.12 Übungen

Übung

Übung 1: Kontextskizze

Zeichnen Sie eine einfache Kontextskizze: Roboter in der Mitte, außen Nutzer, Hindernis, Ladestation, Raum, Wartungspersonal und Netzwerk.

Übung

Übung 2: Funktionsliste erweitern

Ergänzen Sie die Hauptfunktionsliste um fünf weitere Funktionen, die für einen realen Indoor-Roboter nützlich wären.

Übung

Übung 3: Annahmen prüfen

Wählen Sie drei Annahmen aus diesem Kapitel. Beschreiben Sie jeweils, was sich am System ändern würde, wenn die Annahme nicht mehr gilt.

5.13 Literaturhinweise

Für das Praxisprojekt sind neben SysML-Grundlagen [?, ?] besonders ROS2-Grundlagen [?] relevant. Für reale Robotikprojekte sollten zusätzlich Sicherheitsnormen, Datenschutzanforderungen und domänenspezifische Robotikstandards betrachtet werden.

6 Anforderungen modellieren

Lernziele

Nach diesem Kapitel können Sie Anforderungen verständlich, eindeutig und prüfbar formulieren. Sie können Anforderungen nummerieren, strukturieren, in funktionale und nicht-funktionale Anforderungen unterscheiden und mit SysML-Beziehungen verbinden. Außerdem verstehen Sie, wie Anforderungen im Roboterprojekt von Stakeholderbedürfnissen bis zu Testfällen nachverfolgt werden.

6.1 Einleitung

Anforderungen sind eine der wichtigsten Grundlagen eines Systemmodells. Sie beschreiben, was das System leisten muss und unter welchen Bedingungen es dies tun soll. Ohne gute Anforderungen ist unklar, welche Architektur sinnvoll ist und woran der Projekterfolg gemessen wird.

Beim autonomen Indoor-Roboter klingen viele Wünsche zunächst einfach: Der Roboter soll sicher fahren, Hindernisse erkennen, Personen erkennen, lange genug fahren und leicht bedienbar sein. Für die Entwicklung reicht das nicht. Solche Wünsche müssen in Anforderungen übersetzt werden, die eindeutig und später überprüfbar sind.

6.2 Was ist eine gute Anforderung?

Eine gute Anforderung ist:

- verständlich
- eindeutig
- notwendig
- realistisch
- prüfbar
- möglichst lösungsneutral

„Der Roboter soll gut navigieren?“ ist keine gute Anforderung. Was bedeutet „gut“? Schnell? Genau? Sicher? Ohne Kollision? In unbekannten Räumen? Eine bessere Formulierung lautet:

Der Roboter muss in einem bekannten Innenraum ein ausgewähltes Ziel autonom anfahren und dabei statische Hindernisse im Fahrweg vermeiden.

Auch diese Anforderung kann später noch präziser werden, aber sie ist deutlich verständlicher.

6.3 Vom Bedürfnis zur Anforderung

Stakeholder äußern häufig Bedürfnisse. Diese Bedürfnisse sind wichtig, aber noch keine technischen Anforderungen.

Bedürfnis	Mögliche Anforderung
Der Roboter soll niemanden gefährden.	Der Roboter muss bei einem Hindernis im Abstand von 0,5 m seine Geschwindigkeit reduzieren und bei 0,2 m stoppen.
Der Roboter soll einfach bedienbar sein.	Ein Transportauftrag muss mit höchstens drei Eingaben gestartet werden können.
Der Roboter soll nicht liegen bleiben.	Der Roboter muss bei niedrigem Akkustand automatisch zur Ladestation zurückkehren.
Wartung soll einfach sein.	Der Roboter muss Sensorstatus, Akkustatus und Fehlermeldungen anzeigen.

6.4 Beispielanforderungen

ID	Anforderung
REQ-001	Der Roboter muss autonom in einem bekannten Innenraumbereich navigieren können.
REQ-002	Der Roboter muss Hindernisse im Fahrbereich erkennen.
REQ-003	Der Roboter muss erkannte Hindernisse umfahren oder rechtzeitig stoppen.
REQ-004	Der Roboter muss Personen im Kamerabild erkennen können.
REQ-005	Der Roboter muss seinen Akkustand überwachen.
REQ-006	Der Roboter muss bei niedrigem Akkustand selbstständig zur Ladestation fahren.
REQ-007	Der Roboter muss seinen Betriebszustand melden.
REQ-008	Der Roboter muss einen sicheren Zustand einnehmen, wenn ein kritischer Sensor ausfällt.
REQ-009	Der Roboter muss Transportaufträge über eine einfache Benutzerschnittstelle entgegennehmen.
REQ-010	Der Roboter muss relevante Fehler für Wartungspersonal nachvollziehbar protokollieren.

6.5 Funktionale und nicht-funktionale Anforderungen

Funktionale Anforderungen beschreiben, was das System tun soll. Beispiele sind Navigation, Hinderniserkennung, Akkustandsüberwachung oder Statusmeldung.

Nicht-funktionale Anforderungen beschreiben Eigenschaften des Systems. Beispiele sind Geschwindigkeit, Genauigkeit, Verfügbarkeit, Sicherheit, Wartbarkeit, Datenschutz oder Energieverbrauch.

Beide Arten sind wichtig. Ein Roboter, der alle Funktionen besitzt, aber zu langsam, unsicher oder unwartbar ist, erfüllt das Projektziel nicht.

6.6 Anforderungen strukturieren

In größeren Projekten werden Anforderungen hierarchisch strukturiert. Eine oberste Anforderung kann in mehrere konkrete Anforderungen zerlegt werden.

Beispiel:

- REQ-SAFE-001: Der Roboter muss sicher in der Nähe von Menschen betrieben werden.
- REQ-SAFE-002: Der Roboter muss Hindernisse im Fahrweg erkennen.
- REQ-SAFE-003: Der Roboter muss bei Gefahr stoppen.
- REQ-SAFE-004: Der Roboter muss bei Sensorausfall einen sicheren Zustand einnehmen.

Die allgemeine Sicherheitsanforderung wird dadurch handhabbarer.

6.7 SysML-Beziehungen

Wichtige Beziehungen in SysML sind:

- **derive**: Eine Anforderung wird aus einer anderen abgeleitet.
- **satisfy**: Ein Modellelement erfüllt eine Anforderung.
- **verify**: Ein Testfall überprüft eine Anforderung.
- **trace**: Eine allgemeine Nachverfolgungsbeziehung.
- **refine**: Ein Element beschreibt eine Anforderung genauer.

Praxisbeispiel

REQ-002 ?Hindernisse erkennen? wird durch das PerceptionSystem erfüllt. Der Testfall TC-002 Hinderniserkennung verifiziert diese Anforderung. Eine trace-Beziehung kann zusätzlich zeigen, dass der ROS2-Node obstacle_detector_node zur Umsetzung gehört.

Abbildung ?? stellt diese Traceability-Kette grafisch dar: **satisfy** und **verify** zeigen als zwei getrennte, eingehende Beziehungen auf das **PerceptionSystem**, von dort führt **trace** weiter zum ROS2-Node.

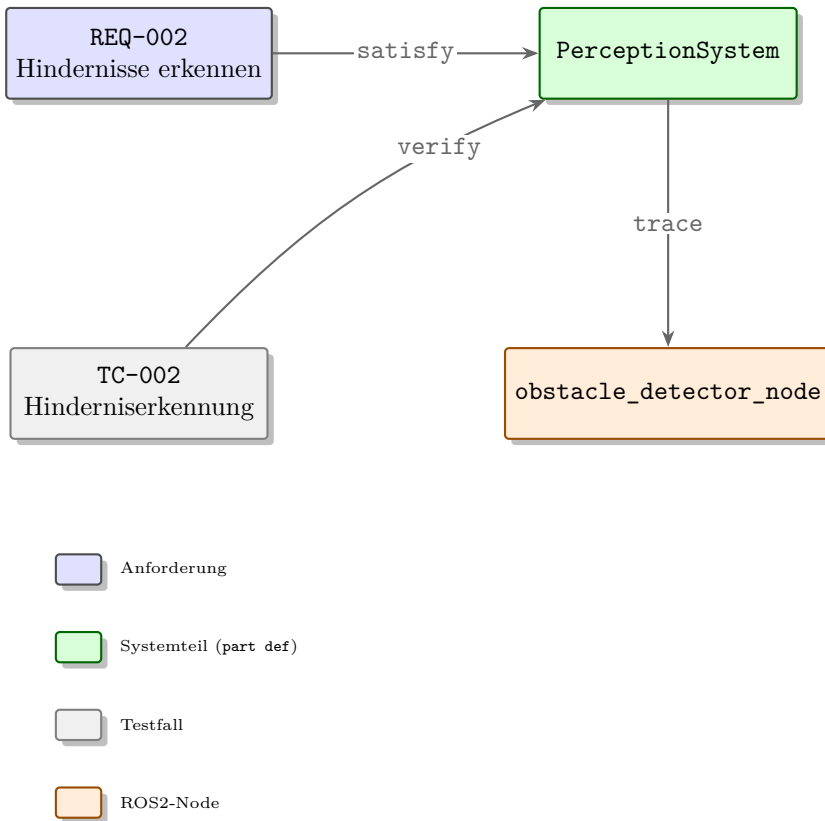


Figure 6.1: Traceability-Kette von REQ-002 über **satisfy** und **verify** zum **PerceptionSystem** und per **trace** weiter zum ROS2-Node.

6.8 Anforderungen in SysML v2

In SysML v2 werden Anforderungen – eingebettet in ein **package** wie bereits aus Kapitel 3 bekannt – textuell mit dem Schlüsselwort **requirement** definiert:

```

1 package Requirements {
2
3   requirement REQ_001 {
4     doc /* Der Roboter muss in einem bekannten Innenraumbereich
5        autonom navigieren koennen. */
6   }
7
8   requirement REQ_SAFE {
9     doc /* Sicherheitsanforderungen. */
10    requirement REQ_SAFE_001 {
11      doc /* Der Roboter muss Hindernisse im Fahrweg erkennen. */

```



```
12     derive REQ_001;  
13 }  
14 }  
15 }
```

Anders als bei `part def` und `part` in Kapitel 8 unterscheidet SysML v2 bei `requirement` im Grundfall nicht zwischen Typ und Nutzung: Das Schlüsselwort `requirement` definiert die Anforderung im Beispiel gleichzeitig als solche.

Erstellen Sie in der Datei `02_requirements.sysml` erste Requirement-Elemente. Verwenden Sie klare Namen, eine stabile ID und einen prüfbaren Text. Achten Sie darauf, dass Anforderungen nicht nur als Liste existieren, sondern später mit Systemteilen, Verhalten und Tests verbunden werden.

Nutzen Sie Beschreibungsfelder (`doc /* ... */`), um Annahmen, Begründungen oder offene Fragen zu dokumentieren. Wenn eine Anforderung noch unklar ist, markieren Sie sie bewusst, statt sie stillschweigend als fertig zu behandeln – zum Beispiel mit einem kurzen `doc /* TBD: ... */`-Vermerk im betroffenen Requirement-Element.

6.9 Typische Fehler

6.9.1 Zu allgemeine Anforderungen

?Der Roboter soll zuverlässig sein? ist nicht prüfbar. Besser sind konkrete Kriterien.

6.9.2 Lösungen als Anforderungen

?Der Roboter muss Sensor X verwenden? ist häufig bereits eine Designentscheidung. Besser ist eine lösungsneutrale Fähigkeit, sofern kein konkreter Sensor vorgeschrieben ist.

6.9.3 Keine Akzeptanzkriterien

Eine Anforderung sollte erkennen lassen, wie sie später geprüft wird. Ein Akzeptanzkriterium legt fest, woran sich die Erfüllung konkret zeigt, zum Beispiel: ?Testfall: Bei einem simulierten Hindernis in 0,5 m Abstand muss die Geschwindigkeit innerhalb 1 s reduziert werden.? Ohne solche Kriterien entstehen Diskussionen am Projektende.

6.9.4 Keine Pflege

Anforderungen ändern sich. Wichtig ist, Änderungen nachvollziehbar zu halten und betroffene Modellelemente zu prüfen.

6.10 Zusammenfassung

Anforderungen verbinden Stakeholderbedürfnisse mit Architektur und Tests. Gute Anforderungen sind verständlich, eindeutig und prüfbar. In SysML v2 werden Anforderungen

nicht nur gesammelt, sondern über Beziehungen mit Systemteilen, Aktionen, Schnittstellen und Testfällen verbunden.

6.11 Kontrollfragen

1. Was macht eine gute Anforderung aus?
2. Warum ist ?Der Roboter soll gut navigieren? problematisch?
3. Was ist der Unterschied zwischen funktionalen und nicht-funktionalen Anforderungen?
4. Wozu dient `derive`?
5. Wozu dient `satisfy`?
6. Wozu dient `verify`?
7. Warum sollten Anforderungen möglichst lösungsneutral sein?
8. Welche Anforderung könnte aus dem Bedürfnis ?Wartung soll einfach sein? entstehen?

6.12 Übungen

Übung

Übung 1: Anforderungen formulieren

Formulieren Sie fünf zusätzliche Anforderungen: zwei zur Sicherheit, eine zur Bedienung, eine zur Wartung und eine zur Leistung.

Übung

Übung 2: Anforderungen verbessern

Verbessern Sie die Aussagen ?Der Roboter soll schnell sein?, ?Der Roboter soll zuverlässig sein? und ?Die Bedienung soll intuitiv sein?, sodass daraus prüfbare Anforderungen werden.

Übung

Übung 3: Beziehungen anlegen

Wählen Sie drei Anforderungen aus der Tabelle. Ordnen Sie jeder Anforderung ein Systemelement (`part def`) und einen Testfall zu.

6.13 Literaturhinweise

Anforderungsmodellierung ist ein zentraler Bestandteil von SysML und MBSE. Für den Einstieg eignen sich Delligatti [?] sowie Friedenthal, Moore und Steiner [?]. Für reale Projekte sollte zusätzlich Literatur zum Requirements Engineering genutzt werden.

7 Stakeholder und Use Cases

Lernziele

Nach diesem Kapitel können Sie Stakeholder identifizieren, ihre Bedürfnisse beschreiben und daraus Use Cases sowie erste Anforderungen ableiten. Sie verstehen, warum Use Cases keine Algorithmen sind, sondern Nutzungsszenarien, die helfen, Systemgrenzen, Akteure und Erwartungen sichtbar zu machen.

7.1 Einleitung

Technische Projekte beginnen nicht mit Komponenten, sondern mit Menschen und Interessen. Ein Roboter wird nicht gebaut, weil es Kamera, Lidar und ROS2 gibt, sondern weil bestimmte Personen ein Problem lösen möchten. Stakeholderanalyse und Use Cases helfen, dieses Problem verständlich zu beschreiben.

7.2 Stakeholder identifizieren

Stakeholder sind nicht nur Endnutzer. Auch Betreiber, Wartung, Entwicklung, Management, Datenschutz und Sicherheitsverantwortliche haben Interessen. Ein Roboter, der technisch funktioniert, aber schlecht wartbar ist oder Datenschutzprobleme erzeugt, kann im Betrieb trotzdem scheitern.

7.3 Stakeholder des Roboters

Stakeholder	Interesse	Mögliche Anforderung
Nutzer	Ziel einfach vorgeben	Auftrag mit höchstens drei Eingaben starten
Betreiber	Zuverlässiger Betrieb	Betriebszustand jederzeit anzeigen
Wartung	Fehler schnell finden	Diagnoseinformationen bereitstellen
Sicherheitsverantwortliche	Personen schützen	Bei Hindernis sicher stoppen
Datenschutz	Bilddaten schützen	Kameradaten lokal verarbeiten
Entwicklung	Klare Architektur	Schnittstellen dokumentieren

Management	Kosten kontrollieren	Komponenten auswählen	begründet
------------	----------------------	-----------------------	-----------

7.4 Use Cases

Use Cases beschreiben, wie ein Akteur – eine Person oder ein externes System, das mit dem betrachteten System interagiert – dieses System nutzt. Sie sind keine detaillierten Algorithmen. Sie beschreiben Ziele, Vorbedingungen, einen typischen Ablauf, Alternativen und Ergebnisse.

Ein guter Use Case beantwortet:

- Wer nutzt das System?
- Welches Ziel verfolgt der Akteur?
- Welche Vorbedingungen gelten?
- Was passiert im Normalfall?
- Welche Ausnahmen können auftreten?
- Welches Ergebnis wird erwartet?

7.5 Beispiel: Ziel anfahren

Bewusst gewählt wurde hier **?Ziel anfahren?** und nicht etwa **?Pfad planen?**: Ein Use Case wird immer aus Sicht eines Akteurs formuliert, während **?Pfad planen?** eher eine interne Funktion beschreibt (mehr dazu im Abschnitt **?Typische Fehler?** weiter unten).

Akteur: Nutzer

Ziel: Roboter fährt zu einem ausgewählten Zielpunkt.

Vorbedingung: Roboter ist eingeschaltet, lokalisiert und besitzt ausreichend Akkuladung.

Ablauf:

1. Nutzer wählt Ziel.
2. Roboter prüft Verfügbarkeit der Karte.
3. Roboter prüft Akkustand.
4. Roboter plant Pfad.
5. Roboter fährt los.
6. Roboter weicht Hindernissen aus oder stoppt sicher.
7. Roboter erreicht Ziel.
8. Roboter meldet Abschluss.

Alternativen: Ziel nicht erreichbar, Akkustand zu niedrig, Hindernis blockiert Pfad.

Ergebnis: Auftrag ist abgeschlossen oder nachvollziehbar abgebrochen.

7.6 Use Cases und Anforderungen

Aus einem Use Case lassen sich Anforderungen ableiten. Der Use Case ?Ziel anfahren? führt zum Beispiel zu Anforderungen an Zielauswahl, Kartennutzung, Lokalisierung, Pfadplanung, Hinderniserkennung, Statusmeldung und Fehlerbehandlung.

Praxisbeispiel

Wenn im Use Case steht, dass der Roboter den Akkustand vor der Fahrt prüft, entsteht daraus eine Anforderung an das Batteriemanagement. Gleichzeitig entsteht ein Testfall: Wird ein Auftrag bei zu niedrigem Akkustand abgelehnt oder in eine Rückkehr zur Ladestation umgewandelt?

7.7 Use Cases in SysML

SysML kann Use-Case-Diagramme nutzen, um Akteure und Systemfunktionen grob darzustellen. Für Einsteiger ist der Text oft wichtiger als das Diagramm. Das Diagramm zeigt die Übersicht, die Use-Case-Beschreibung liefert die Details. Dieses Buch konzentriert sich bei Use Cases bewusst auf die textuelle Beschreibung (Akteur, Ziel, Ablauf) und das grafische Use-Case-Diagramm; die formale SysML-v2-Textsyntax für Use Cases wird hier nicht vertieft, da für den Einstieg die inhaltliche Beschreibung wichtiger ist.

7.8 Typische Fehler

7.8.1 Use Cases mit Funktionen verwechseln

?Pfad planen? ist eher eine Funktion. ?Ziel anfahren? ist ein Use Case, weil ein Akteur ein Ziel verfolgt.

7.8.2 Nur den Normalfall beschreiben

Gerade Alternativen sind wertvoll. Was passiert bei leerem Akku, fehlender Karte oder blockiertem Flur?

7.8.3 Stakeholder vergessen

Wenn nur der Endnutzer betrachtet wird, fehlen Wartung, Betrieb, Sicherheit und Datenschutz.

7.9 Praxisleitfaden für das Roboterprojekt

Für das Roboterprojekt empfiehlt sich ein einfacher Arbeitsablauf. Beginnen Sie nicht mit dem Use-Case-Diagramm, sondern mit einer Liste echter Nutzungssituationen.

Fragen Sie: Wer steht vor dem Roboter? Wer sieht nur die Betriebsdaten? Wer ist betroffen, obwohl er den Roboter nie bewusst bedient?

Danach gruppieren Sie die Situationen. Typische Gruppen sind Bedienung, Betrieb, Wartung, Sicherheit und Datenschutz. Für jede Gruppe wird mindestens ein Use Case beschrieben. Ein gutes Einsteigermodell könnte folgende Use Cases enthalten:

- Transportauftrag starten
- Ziel autonom anfahren
- Hindernis behandeln
- Zur Ladestation zurückkehren
- Fehler diagnostizieren
- Roboter außer Betrieb nehmen

Aus jedem Use Case entstehen Anforderungen. Der Use Case ?Fehler diagnostizieren? führt zum Beispiel zu Anforderungen an Statusmeldungen, Logdaten und Wartungszugriff. Der Use Case ?Roboter außer Betrieb nehmen? führt zu Anforderungen an sichere Abschaltung und klare Bedienrückmeldung.

Abbildung ?? zeigt diese sechs Use Cases zusammen mit den beteiligten Akteuren Nutzer, Sicherheitsverantwortliche und Wartungspersonal.

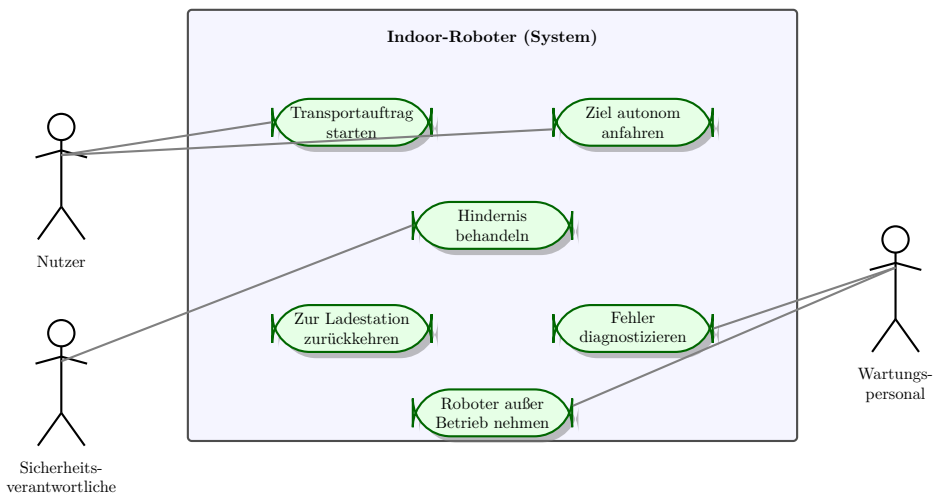


Figure 7.1: Use-Case-Diagramm des Indoor-Roboters mit den Akteuren Nutzer, Sicherheitsverantwortliche und Wartungspersonal.

7.10 Zusammenfassung

Stakeholder und Use Cases helfen, das System aus Nutzungssicht zu verstehen. Sie bilden eine Brücke zwischen Bedürfnissen und Anforderungen. Für den Roboter sind

Use Cases wie ?Ziel anfahren?, ?Zur Ladestation zurückkehren?, ?Hindernis behandeln? und ?Wartung durchführen? besonders wichtig.

7.11 Kontrollfragen

1. Was ist ein Stakeholder?
2. Warum sind Wartung und Datenschutz Stakeholder?
3. Was beschreibt ein Use Case?
4. Warum ist ein Use Case kein Algorithmus?
5. Welche Vorbedingungen hat der Use Case ?Ziel anfahren??
6. Wie können aus Use Cases Anforderungen entstehen?
7. Warum sollten Alternativabläufe beschrieben werden?

7.12 Übungen

Übung

Übung 1: Use Case schreiben

Schreiben Sie einen Use Case ?Zur Ladestation zurückkehren?. Leiten Sie daraus mindestens drei Anforderungen ab.

Übung

Übung 2: Stakeholder erweitern

Ergänzen Sie die Stakeholdertabelle um drei weitere Stakeholder und beschreiben Sie deren Interessen.

Übung

Übung 3: Ausnahmefälle

Ergänzen Sie zum Use Case ?Ziel anfahren? drei Ausnahmefälle und jeweils eine erwartete Systemreaktion.

7.13 Literaturhinweise

Use Cases und Stakeholderanalyse sind Grundlagen des Requirements Engineering und werden in MBSE durch SysML-Beziehungen mit Anforderungen, Struktur und Tests verbunden. Vertiefend helfen praxisnahe SysML-Einführungen [?, ?].

8 Strukturmodellierung mit `part def`

Lernziele

Nach diesem Kapitel können Sie die Hauptstruktur des Roboters als SysML-v2-Strukturmodell beschreiben. Sie verstehen `part def`, `part`, Teil-Ganzes-Beziehungen und Dekomposition. Außerdem kennen Sie den Unterschied zwischen logischen, physischen und softwarebezogenen Systemteilen.

8.1 Einleitung

Die Strukturmodellierung ist einer der wichtigsten Einstiegspunkte in ein MBSE-Projekt. Sie zeigt, aus welchen Bausteinen ein System besteht und wie diese strukturell zusammenhängen. Beim autonomen Indoor-Roboter hilft die Strukturmodellierung, Ordnung in Sensorik, Navigation, Antrieb, Energieversorgung, Bedienung und Software zu bringen.

In SysML v2 erfolgt die Strukturmodellierung textuell mit `part def` und `part`. `part def` definiert den Typ eines Systemelements. `part` erzeugt eine benannte Instanz dieses Typs als Teil eines übergeordneten Elements. Wer bereits programmiert hat, kann sich `part def` wie eine Klassendefinition vorstellen und `part` wie ein konkretes Objekt dieser Klasse.

Hinweis

SysML v1 verwendete den Begriff `?Block?` und das Block Definition Diagram (BDD) – ein grafisches Diagramm mit kastenförmigen Blöcken zur Darstellung von Bausteinen und ihren Beziehungen. In SysML v2 wird beides durch `part def` (Typdefinition) und `part` (Instanz) ersetzt. Das Konzept bleibt ähnlich, die Syntax ist präziser und textuell.

8.2 Zweck der Strukturmodellierung

Eine Struktursicht beantwortet Fragen wie:

- Aus welchen Hauptteilen besteht das System?
- Welche Teilsysteme gehören zum Roboter?
- Welche Systemteile sind spezialisiert oder zusammengesetzt?

- Welche Struktur ist logisch, welche physisch?

8.3 Hauptstruktur des Roboters

Eine erste Strukturbeschreibung für den autonomen Indoor-Roboter kann so aussehen:

```

1 package RobotStructure {
2
3   part def Robot {
4     part perceptionSystem : PerceptionSystem;
5     part navigationSystem : NavigationSystem;
6     part motionControl : MotionControl;
7     part batterySystem : BatterySystem;
8     part userInterface : UserInterface;
9     part communicationSystem : CommunicationSystem;
10    part safetyLogic : SafetyLogic;
11  }
12
13  part def PerceptionSystem;
14  part def NavigationSystem;
15  part def MotionControl;
16  part def BatterySystem;
17  part def UserInterface;
18  part def CommunicationSystem;
19  part def SafetyLogic;
20 }
```

Diese Darstellung ist noch grob. Sie eignet sich aber hervorragend, um den Systemumfang zu verstehen. Jedes **part** innerhalb von **Robot** ist eine Instanz des angegebenen Typs. Der Typ wird durch **part def** definiert.

8.4 Komposition

Wenn ein **part** innerhalb einer **part def** definiert ist, beschreibt das eine starke Teil-Ganzes-Beziehung. Das **perceptionSystem** gehört zum **Robot**. Das bedeutet: In dieser Systembetrachtung ist das Wahrnehmungssystem fester Bestandteil des Roboters.

Komposition ist besonders nützlich, um die Systemgrenze sichtbar zu machen. Gehört die Ladestation zum System oder ist sie extern? Beide Varianten können modelliert werden, aber die Entscheidung muss bewusst sein.

8.5 Dekomposition

Dekomposition bedeutet, ein Systemelement in kleinere Teile zu zerlegen. Die zuvor leer deklarierte **part def PerceptionSystem** wird dabei mit Inhalt gefüllt – es handelt sich um dieselbe Definition, nicht um eine zweite, gleichnamige. Das **PerceptionSystem** kann zum Beispiel weiter zerlegt werden:

```
1 part def PerceptionSystem {  
2   part camera : Camera;  
3   part lidar : Lidar;  
4   part objectDetector : ObjectDetector;  
5   part obstacleDetector : ObstacleDetector;  
6 }
```

Das NavigationSystem könnte folgende Teile enthalten:

```
1 part def NavigationSystem {  
2   part localization : Localization;  
3   part mapping : Mapping;  
4   part pathPlanner : PathPlanner;  
5   part pathFollower : PathFollower;  
6 }
```

Abbildung ?? zeigt die vollständige Dekomposition des Gesamtsystems **Robot** über zwei Ebenen: die sieben Hauptsysteme und ihre jeweils wichtigsten Teile.

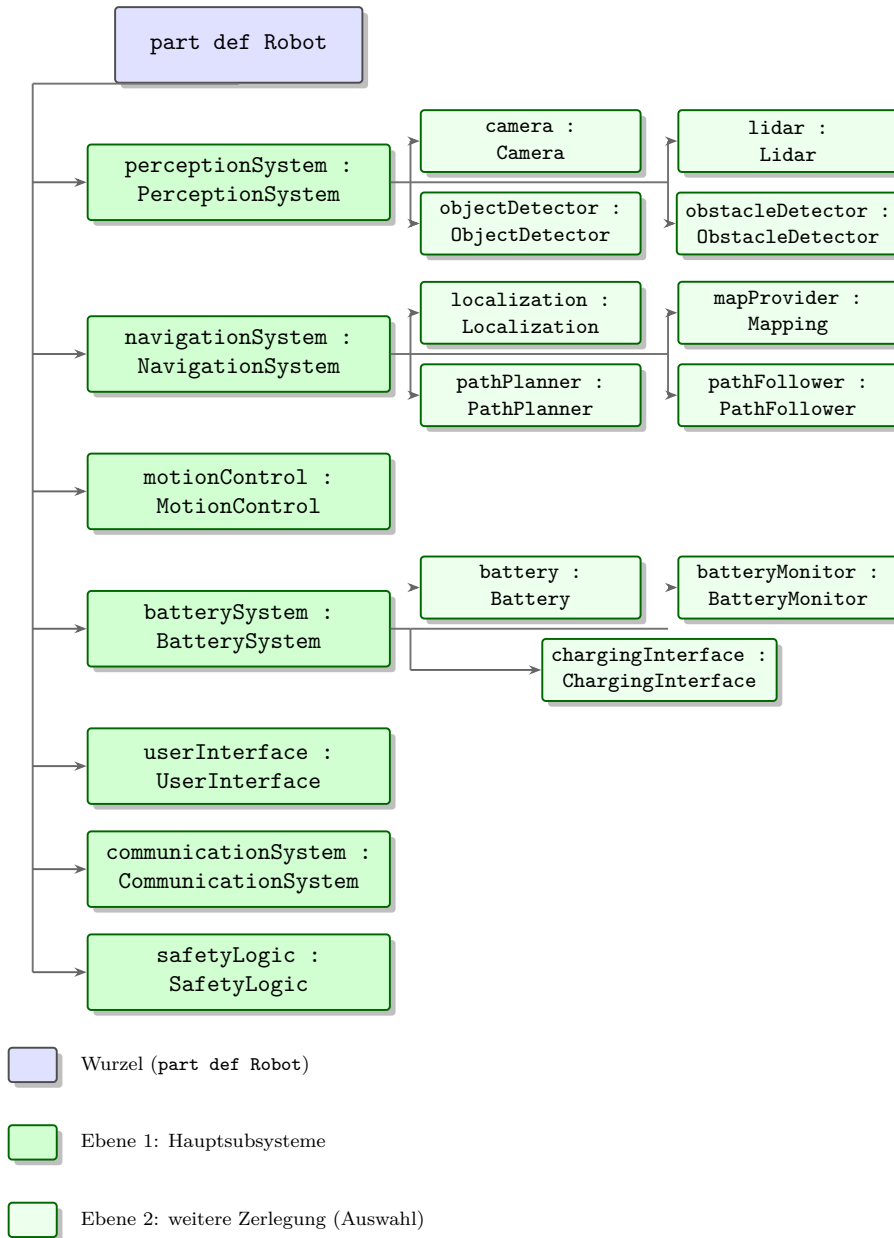


Figure 8.1: Vollständige Dekomposition von Robot in sieben Hauptsubsysteme und deren wichtigste Teile der zweiten Ebene.

8.6 Logische und physische Systemteile

Ein häufiger Anfängerfehler besteht darin, logische Funktionen und physische Bauteile unklar zu vermischen. **Camera** ist eher physisch. **ObjectDetector** ist eher softwarebezogen. **Hindernis erkennen?** ist eher eine Funktion.

Alle drei Sichtweisen dürfen im Modell vorkommen. Sie sollten aber bewusst getrennt

oder klar benannt werden. Eine einfache Regel lautet:

- Physische Systemteile beschreiben greifbare Bauteile.
- Logische Systemteile beschreiben Verantwortlichkeiten oder Teilsysteme.
- Softwareteile beschreiben spätere Implementierungseinheiten.

8.7 Strukturmodell und Anforderungen

Systemteile stehen nicht isoliert im Modell. Sie erfüllen Anforderungen über **satisfy**-Beziehungen. Das **BatterySystem** kann Anforderungen zur Akkustandsüberwachung erfüllen. Das **PerceptionSystem** erfüllt Anforderungen zur Hindernis- und Personenerkennung.

Praxisbeispiel

REQ-006 ?Der Roboter muss bei niedrigem Akkustand zur Ladestation fahren? wird nicht durch ein einzelnes Systemelement erfüllt. Beteiligt sind **BatterySystem**, **NavigationSystem**, **MotionControl** und möglicherweise **UserInterface**. Die Strukturmodellierung hilft, diese beteiligten Systemteile sichtbar zu machen.

8.8 Typen wiederverwenden

Ein wichtiger Vorteil von **part def** ist die Wiederverwendbarkeit. Wenn ein Roboter zwei Kameras besitzt, wird **Camera** nur einmal definiert und zweimal instanziiert:

```
1 part def PerceptionSystem {  
2   part frontCamera : Camera;  
3   part rearCamera : Camera;  
4   part lidar : Lidar;  
5 }
```

Ändert sich die Definition von **Camera**, gilt die Änderung automatisch für beide Instanzen.

8.9 Typische Fehler

8.9.1 Zu tiefe Dekomposition

Ein Strukturmodell muss nicht jedes Kabel und jede Schraube enthalten. Modellieren Sie so tief, wie es für Verständnis, Schnittstellen und Entscheidungen nötig ist. Eine einfache Faustregel: Dekomponieren Sie so lange, bis sich jede relevante Schnittstelle benennen lässt – tiefer muss ein Einsteigermodell in der Regel nicht gehen.

8.9.2 Unklare Namen

`System1` oder `ModuleA` sind schlechte Namen. Namen sollten die Verantwortung des Systemteils ausdrücken.

8.9.3 Funktionen als Hardware modellieren

`ObstacleDetector` kann eine Softwarekomponente sein, aber ?Hindernis erkennen? ist zunächst eine Funktion. Klare Benennung verhindert Verwirrung.

8.10 Modellierungsstrategie für Einsteiger

Ein gutes Strukturmodell entsteht schrittweise. Beginnen Sie mit einer einzigen *part def* `Robot`. Darunter modellieren Sie fünf bis sieben Hauptsysteme. Mehr sollte die erste Ebene nicht enthalten.

Danach wählen Sie ein Teilsystem aus und zerlegen es weiter. Für den Einstieg eignet sich das `PerceptionSystem`, weil es Hardware, Software und Datenverarbeitung verbindet. Modellieren Sie Kamera und Lidar als physische Teile, `ObjectDetector` und `ObstacleDetector` als softwarebezogene Teile und dokumentieren Sie kurz, warum diese Trennung sinnvoll ist.

Für das `BatterySystem` reicht zunächst eine einfachere Dekomposition:

```
1 part def BatterySystem {  
2   part battery : Battery;  
3   part batteryMonitor : BatteryMonitor;  
4   part chargingInterface : ChargingInterface;  
5 }
```

Damit wird sichtbar, dass Akkuzelle, Überwachung und Ladeschnittstelle unterschiedliche Verantwortlichkeiten haben. Diese Unterscheidung hilft später beim Mapping auf ROS2 und beim Testen.

8.11 Zusammenfassung

Die Strukturmodellierung mit *part def* und *part* beschreibt die strukturelle Sicht auf das System. Sie zeigt Hauptsystemteile, Teil-Ganzes-Beziehungen und Dekomposition. Für den Indoor-Roboter ist sie der Einstieg in die Architektur: Wahrnehmung, Navigation, Antrieb, Energieversorgung, Bedienung und Sicherheit werden als Systemteile sichtbar.

8.12 Kontrollfragen

1. Wozu dient die Strukturmodellierung?
2. Was ist der Unterschied zwischen *part def* und *part*?
3. Was beschreibt eine Komposition?

4. Was bedeutet Dekomposition?
5. Warum sollte man logische und physische Systemteile unterscheiden?
6. Welche Hauptteile besitzt der Indoor-Roboter?
7. Wie hängt die Strukturmodellierung mit Anforderungen zusammen?
8. Welchen Vorteil bietet die Wiederverwendung von `part def`?

8.13 Übungen

Übung

Übung 1: Strukturmodell erstellen

Erstellen Sie eine `part def Robot` mit mindestens fünf Subsystemen. Zerlegen Sie das `PerceptionSystem` weiter in Kamera, Lidar und Objekterkennung.

Übung

Übung 2: Systemteile klassifizieren

Ordnen Sie folgende Begriffe als physisch, logisch oder softwarebezogen ein: Kamera, NavigationSystem, `object_detector_node`, Hindernis erkennen, Batterie, PathPlanner.

Übung

Übung 3: Wiederverwendung

Modellieren Sie ein `PerceptionSystem` mit zwei Kameras (`frontCamera` und `rearCamera`). Erklären Sie, warum `Camera` nur einmal als `part def` definiert werden muss.

8.14 Literaturhinweise

Die Strukturmodellierung in SysML v2 ist im SysML-v2-Pilotprojekt dokumentiert [?]. Der Syside Editor bietet praktische Beispiele direkt in der Entwicklungsumgebung [?]. Für den Vergleich mit SysML v1 und dem dort verwendeten Block-Begriff sind Delligatti [?] und Friedenthal et al. [?] hilfreich, müssen aber bewusst auf v2 übertragen werden.

9 Verbindungsmodellierung mit port und connection

Lernziele

Nach diesem Kapitel können Sie interne Verbindungen zwischen Systemteilen in SysML v2 beschreiben. Sie verstehen **port**, **connection** und Datenflüsse und können diese Konzepte auf ROS2-nahe Kommunikation im Roboterprojekt übertragen.

9.1 Einleitung

Die Strukturmodellierung aus dem vorherigen Kapitel zeigt, welche Systemteile existieren. Die Verbindungsmodellierung zeigt, wie diese Teile zusammenarbeiten. Sie ist der nächste Schritt von der Struktur zur Zusammenarbeit.

In SysML v2 erfolgt die Verbindungsmodellierung mit **port** und **connection**. Ports beschreiben die Schnittstellen eines Systemteils nach außen. Verbindungen verknüpfen Ports verschiedener Teile miteinander.

Hinweis

SysML v1 verwendete das Internal Block Diagram (IBD) für diese Sicht. In SysML v2 sind Ports und Verbindungen direkte Modellelemente innerhalb von **part def**. Das Konzept bleibt ähnlich, die textuelle Notation ist präziser.

9.2 Zweck der Verbindungsmodellierung

Eine Verbindungssicht beantwortet Fragen wie:

- Welche Teile arbeiten innerhalb eines Systems zusammen?
- Welche Daten oder Signale fließen zwischen ihnen?
- Welche Ports besitzen die Teile?
- Welche Schnittstellen müssen beschrieben werden?
- Wo entstehen Integrationsrisiken?

9.3 Ports

Ein Port beschreibt einen Ein- oder Ausgang eines Systemteils. Ports machen Schnittstellen explizit und erzwingen, dass Daten- oder Signalflüsse benannt werden.

```

1 port def ImagePort;
2 port def ScanPort;
3 port def ObjectListPort;
4
5 part def Camera {
6   port imageOut : ImagePort;
7 }
8
9 part def Lidar {
10  port scanOut : ScanPort;
11 }
12
13 part def ObjectDetector {
14  port imageIn : ~ImagePort;
15  port objectsOut : ObjectListPort;
16 }
17
18 part def ObstacleDetector {
19  port scanIn : ~ScanPort;
20 }

```

Das Tilde-Zeichen ~ markiert einen [konjugierten Port](#), der empfängt statt sendet.

9.4 Verbindungen

`connection` verknüpft kompatible Ports miteinander. Eine Verbindung sollte immer dokumentieren, was über sie fließt. Die Grundform lautet: `connection <Name> connect <Teil1>.<Port1> to <Teil2>.<Port2>;`

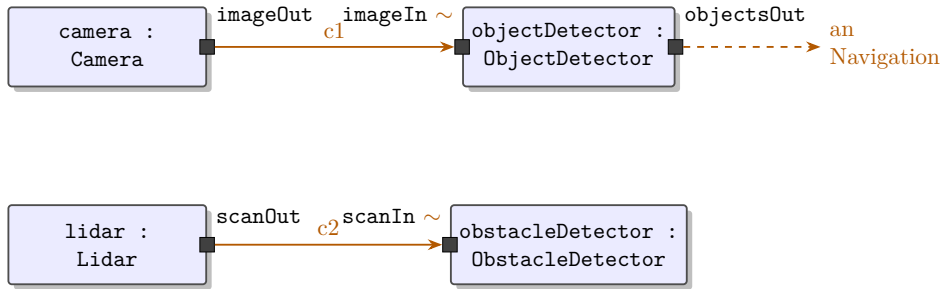
```

1 part def PerceptionSystem {
2   part camera : Camera;
3   part lidar : Lidar;
4   part objectDetector : ObjectDetector;
5   part obstacleDetector : ObstacleDetector;
6
7   connection c1 connect camera.imageOut
8                     to objectDetector.imageIn;
9   connection c2 connect lidar.scanOut
10                  to obstacleDetector.scanIn;
11 }

```

Abbildung ?? zeigt das zugehörige Verbindungsmodell mit den beteiligten Ports.

9 Verbindungsmodellierung mit *port* und *connection*



■ Port

~ = konjugierter Port (empfaengt statt sendet)
→ Verbindung (connection)

Figure 9.1: Verbindungsmodell des PerceptionSystem. Die kleinen Quadrate an den Boxrändern sind Ports. Das Tilde-Zeichen markiert einen konjugierten Port, der empfängt statt sendet.

9.5 Datenflüsse und ROS2

Die Verbindungsmodellierung passt gut zur ROS2-Kommunikation. ROS2-Topics können als Datenflüsse modelliert werden. Ein Topic `/camera/image_raw` verbindet `camera_node` und `object_detector_node`. Ein Topic `/cmd_vel` verbindet Navigation und Bewegungssteuerung.

Praxisbeispiel

Eine Verbindungssicht für das PerceptionSystem zeigt: Die Kamera liefert Bilddaten an den ObjectDetector, während der Lidar Laserscandaten an den ObstacleDetector liefert. Beide Ergebnisse gehen an die Navigation. Daraus lassen sich später ROS2-Topics direkt ableiten.

9.6 Verbindungsmodell für die Navigation

Eine Verbindungssicht für die Navigation macht sichtbar, dass Lokalisierung, Karte, Pfadplanung und Bewegungssteuerung zusammenarbeiten:

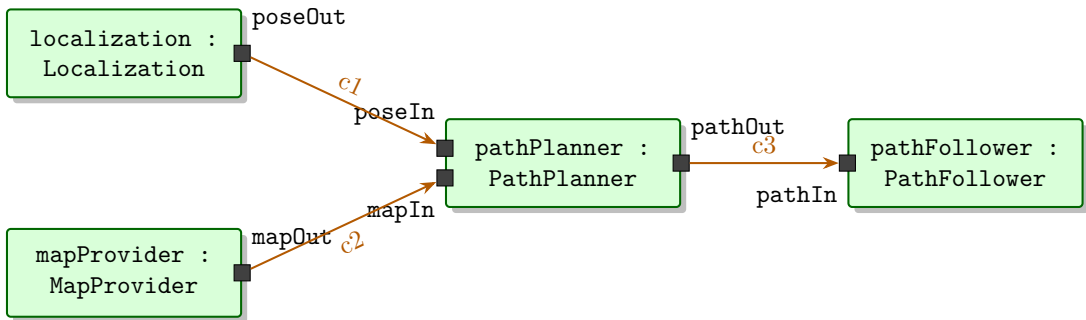
```
1 part def NavigationSystem {
2   part localization : Localization;
3   part mapProvider : MapProvider;
4   part pathPlanner : PathPlanner;
5   part pathFollower : PathFollower;
6
7   connection c1 connect localization.poseOut
```

```

8         to pathPlanner.poseIn;
9     connection c2 connect mapProvider.mapOut
10         to pathPlanner.mapIn;
11     connection c3 connect pathPlanner.pathOut
12         to pathFollower.pathIn;
13 }

```

Dabei liefert *c1* die aktuelle Pose der Lokalisierung an den Pfadplaner, *c2* liefert die Karte an denselben Pfadplaner, und *c3* gibt den geplanten Pfad an die Bewegungssteuerung weiter.



■ Port

→ Verbindung (*connection*)

Figure 9.2: Verbindungsmodell des `NavigationSystem`: Lokalisierung und Karte liefern an den Pfadplaner, dessen Pfad an die Bewegungssteuerung weitergegeben wird.

Navigation entsteht aus mehreren zusammenarbeitenden Teilen (siehe Abbildung ??). Gleichzeitig werden Schnittstellen sichtbar, die später in ROS2 umgesetzt werden können.

9.7 Annahmen dokumentieren

Ein gutes Verbindungsmodell sollte auch Annahmen dokumentieren. Wird die Karte vorher geladen oder während der Fahrt aktualisiert? Arbeitet die Lokalisierung relativ zu `map` oder `odom` (zwei gebräuchliche ROS2-Koordinatensysteme, die in Kapitel 10 näher erklärt werden)? Solche Fragen gehören als Dokumentation (`doc /* ... */`) zum Modellelement.

```

1 part def NavigationSystem {
2     doc /* Voraussetzung: Eine initiale Karte ist vorhanden.
3         Lokalisierung arbeitet relativ zum map-Frame. */

```

```
4 | ...
5 | }
```

9.8 Typische Fehler

9.8.1 Nur Teile verbinden ohne Ports

Verbindungen sollten immer über benannte Ports laufen. Direkte Verbindungen ohne Ports lassen offen, was genau übertragen wird.

9.8.2 Ports ohne Datentypen

Ein Port wird erst nützlich, wenn bekannt ist, welche Daten darüber fließen. Definieren Sie `port def` mit sprechenden Namen.

9.8.3 Zu viele Details in einem Modellabschnitt

Eine Verbindungssicht sollte eine klare Frage beantworten. Für große Systeme sind mehrere fokussierte Abschnitte besser als ein überladenes Modell.

9.9 Zusammenfassung

Die Verbindungsmodellierung mit `port` und `connection` zeigt die interne Zusammenarbeit von Systemteilen. Sie ist besonders nützlich für Schnittstellen und Datenflüsse. Im Roboterprojekt bildet sie eine Brücke von der SysML-v2-Struktur zur ROS2-Kommunikation.

9.10 Kontrollfragen

1. Was zeigt eine Verbindungssicht gegenüber einer Struktursicht?
2. Was beschreibt ein `port`?
3. Was bewirkt das Tilde-Zeichen bei einem Port?
4. Wozu dient `connection`?
5. Warum ist die Verbindungsmodellierung für ROS2 hilfreich?
6. Welches Topic könnte zwischen Kamera und ObjectDetector liegen?
7. Warum sollte ein Verbindungsmodell Annahmen dokumentieren?

9.11 Übungen

Übung

Übung 1: Verbindungsmodell PerceptionSystem

Erstellen Sie eine `part def PerceptionSystem` mit Kamera, Lidar, ObjectDetector und ObstacleDetector. Definieren Sie Ports und verbinden Sie sie mit `connection`.

Übung

Übung 2: Ports benennen

Definieren Sie für `BatterySystem`, `NavigationSystem` und `MotionControl` jeweils mindestens zwei Ports mit sprechenden Namen.

Übung

Übung 3: ROS2-Mapping vorbereiten

Wählen Sie drei Verbindungen aus Ihrem Modell und ordnen Sie ihnen jeweils ein mögliches ROS2-Topic zu.

9.12 Literaturhinweise

Ports und Verbindungen in SysML v2 sind im SysML-v2-Pilotprojekt dokumentiert [?]. Der Syside Editor bietet konkrete Beispiele für die Verbindungsmodellierung [?]. Für die Umsetzung in ROS2 sind die ROS2-Dokumentation und einschlägige Einführungen hilfreich [?].

10 Schnittstellenmodellierung

Lernziele

Nach diesem Kapitel können Sie Schnittstellen systematisch beschreiben und erkennen, warum klare Schnittstellen in Robotikprojekten entscheidend sind. Sie können Datenflüsse, Sender, Empfänger, Datentypen, Frequenzen, Qualitätsanforderungen und Fehlerfälle dokumentieren.

10.1 Einleitung

Viele technische Probleme entstehen nicht in einzelnen Komponenten, sondern an ihren Schnittstellen. Eine Kamera liefert Bilder in einem Format, das der Erkennungsalgorithmus nicht erwartet. Ein Navigationsknoten erwartet Koordinaten in einem anderen Bezugssystem. Ein Batteriemodul meldet Werte langsamer als die Missionsplanung annimmt. Solche Fehler sind typisch.

10.2 Warum Schnittstellen wichtig sind

Schnittstellen sind die Stellen, an denen Systemteile zusammenarbeiten. Je komplexer ein System wird, desto wichtiger werden klare Schnittstellen. Beim Indoor-Roboter betreffen Schnittstellen Sensorik, Software, Energie, Mechanik, Bedienung und Umgebung.

Eine unklare Schnittstelle führt zu Fragen wie:

- Wer sendet welche Daten?
- In welchem Format liegen die Daten vor?
- Wie oft werden die Daten gesendet?
- Was passiert bei fehlenden oder fehlerhaften Daten?
- Welche Einheit und welches Koordinatensystem gelten?

10.3 Schnittstellenbeschreibung

Eine gute Schnittstellenbeschreibung enthält:

- Name der Schnittstelle

- Sender und Empfänger
- Datentyp
- Richtung
- Frequenz oder Ereignis
- Einheit und Koordinatensystem
- Qualitätsanforderungen
- Fehlerfälle
- Bezug zu Anforderungen

10.4 Beispieltabelle

Schnittstelle	Sender	Empfänger	Datentyp
Kamerabild	Camera	ObjectDetector	<code>sensor_msgs/Image</code>
Laserscan	Lidar	SLAM	<code>sensor_msgs/LaserScan</code>
Bewegungsbefehl	Navigation	MotionControl	<code>geometry_msgs/Twist</code>
Akkustatus	BatterySystem	Navigation	<code>sensor_msgs/BatteryState</code>
Objektliste	ObjectDetector	Navigation	Detection-Liste
Roboterstatus	Robot	UserInterface	Statusnachricht

Abbildung ?? zeigt dieselben Schnittstellen im Zusammenhang: Sensordaten fließen von Camera, Lidar und BatterySystem über ObjectDetector und SLAM zur Navigation, während Steuer- und Statusnachrichten Navigation, MotionControl und die Bedienoberfläche verbinden.

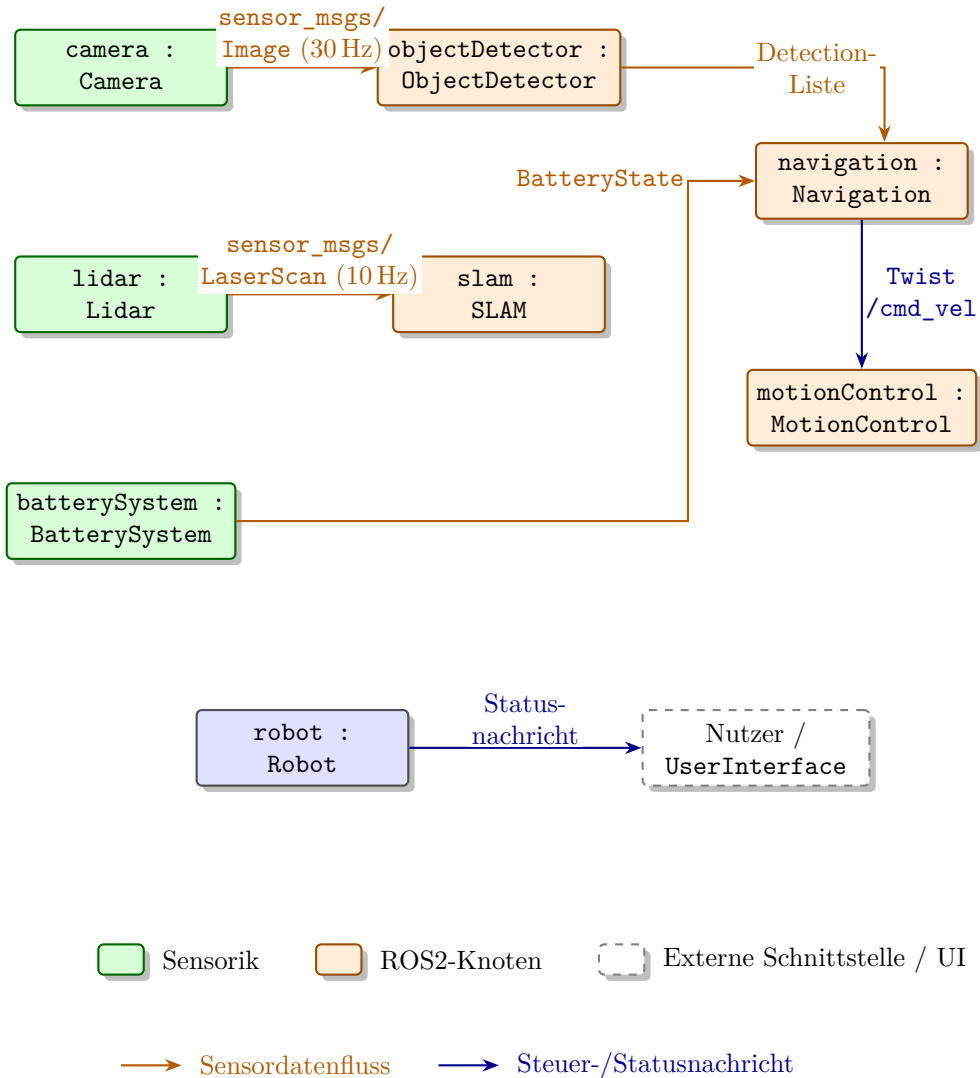


Figure 10.1: Schnittstellenübersicht des Indoor-Roboters: Sensordaten (orange) fließen über ObjectDetector und SLAM zur Navigation, Steuer- und Statusnachrichten (blau) verbinden Navigation, MotionControl und die Bedienoberfläche.

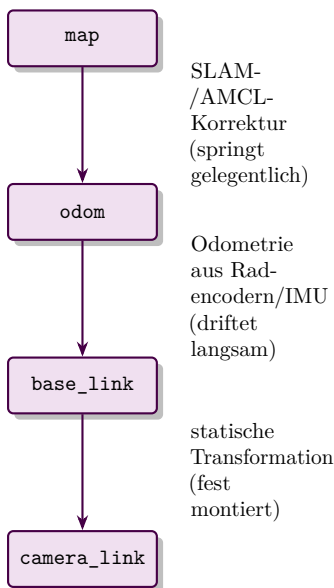
10.5 Koordinatensysteme

In Robotikprojekten sind Koordinatensysteme kritisch. ROS2 nutzt häufig Frames wie `map`, `odom`, `base_link` und `camera_link`. Diese Frames sollten im Modell dokumentiert werden.

Ein klassischer Fehler ist, dass Daten im falschen Frame interpretiert werden. Ein Hindernis wird relativ zur Kamera erkannt, die Navigation erwartet aber Positionen relativ zur Karte. Ohne klare Transformationen entstehen falsche Bewegungsentscheidungen.

ROS2 verwaltet diese Umrechnungen nicht manuell, sondern über den sogenannten TF-Baum (`tf2`): Jeder Frame kennt seine Beziehung zu seinem Elternframe, und das System kann daraus jede beliebige Umrechnung zwischen zwei Frames automatisch berechnen. `map` ist dabei der Wurzelframe und bewegt sich nicht. `odom` hängt an `map`, wird aber von der Lokalisierung (z. B. SLAM oder AMCL) gelegentlich sprunghaft korrigiert. `base_link` hängt an `odom` und wird kontinuierlich aus Radencodern oder einer IMU aktualisiert, driftet dabei aber langsam. `camera_link` schließlich hängt fest an `base_link`, weil die Kamera starr am Roboter montiert ist. Abbildung ?? zeigt diese Hierarchie links als Baum und rechts als physische Ursprünge im Gebäudegrundriss und am Roboter.

TF-Baum
(Hierarchie der Frames)



Physische Bedeutung
(Draufsicht)

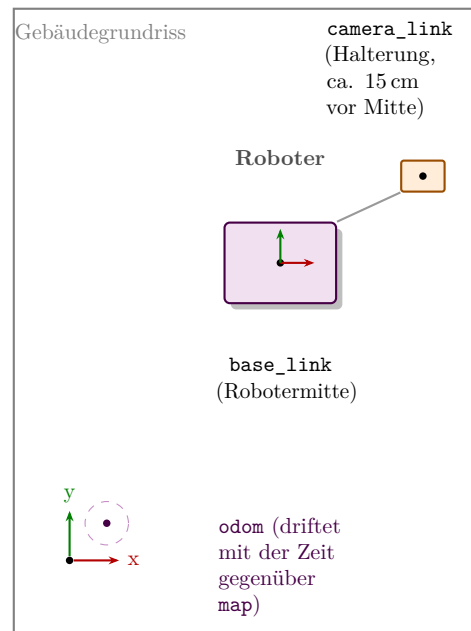


Figure 10.2: Links die Hierarchie der TF-Frames vom kartenfesten `map` über `odom` und `base_link` bis zu `camera_link`; rechts dieselben Frames als physische Koordinatenursprünge im Gebäudegrundriss und am Roboter.

Hinweis

Eine der häufigsten Fehlerquellen für Einsteiger ist ein fehlender oder veralteter Zweig im TF-Baum: Fehlt die Transformation zwischen `odom` und `base_link`, kann die Navigation die Position des Roboters nicht mehr bestimmen, selbst wenn alle Sensordaten korrekt ankommen.

10.6 Frequenzen und Timing

Sensordaten kommen nicht alle gleich schnell. Eine Kamera liefert vielleicht 30 Bilder pro Sekunde, ein Lidar 10 Scans pro Sekunde, der Akkustatus nur einmal pro Sekunde. Das Modell sollte solche Annahmen sichtbar machen, wenn sie für Architektur oder Tests wichtig sind.

10.7 Fehlerfälle

Schnittstellen sollten nicht nur den Normalfall beschreiben. Was passiert, wenn keine Kamerabilder ankommen? Was passiert, wenn der Laserscan veraltet ist? Was passiert, wenn der Akkustatus ungültig ist?

Praxisbeispiel

Wenn der `battery_monitor_node` keine gültigen Daten mehr liefert, darf die Navigation nicht einfach weiterfahren, als wäre alles in Ordnung. Eine mögliche Anforderung lautet: Bei fehlendem Batteriestatus länger als fünf Sekunden muss der Roboter einen sicheren Zustand einnehmen oder eine Warnung ausgeben.

10.8 Schnittstellen in SysML

In SysML v2 werden Schnittstellen in dieser vereinfachten Einsteigerversion über `port def`, `port` und `connection` dargestellt; ein `port` markiert dabei die Anschlussstelle eines Blocks, über die Daten ein- oder ausgehen, etwa `port def CameraPort { out image: Image; }` für einen Kamera-Ausgang. Das eigentliche SysML-v2-Konstrukt für Schnittstellenverträge, `interface def` und `interface`, lernen Sie als Konzept im nächsten Abschnitt kennen. Für den Einstieg reicht es, Ports und Datenflüsse sauber zu benennen und in Tabellen zu dokumentieren. Später kann die Modellierung formaler werden.

10.9 Schnittstellen als Vertrag

Eine Schnittstelle ist ein Vertrag zwischen Sender und Empfänger. Dieser Vertrag muss nicht juristisch gemeint sein, sondern technisch: Der Sender verspricht, bestimmte

Daten in einer bestimmten Form bereitzustellen. Der Empfänger darf sich darauf verlassen, solange die Schnittstelle gültig ist. Genau dieses Vertragskonzept bildet SysML v2 formal über **interface def** und **interface** ab; wir bleiben hier aber bei der einfacheren Port-Notation aus dem letzten Abschnitt, um den Einstieg zu erleichtern.

Beim Topic `/cmd_vel` (einem ROS2-Kommunikationskanal für Nachrichten) erwartet die Bewegungssteuerung zum Beispiel Geschwindigkeitsbefehle. Wenn die Navigation plötzlich andere Einheiten verwendet oder Werte außerhalb erlaubter Grenzen sendet, kann das System gefährlich reagieren. Deshalb sollte eine Schnittstellenbeschreibung auch Grenzen enthalten: maximale Geschwindigkeit, erlaubte Wertebereiche und Verhalten bei ungültigen Nachrichten. In ROS2 lassen sich solche Qualitätsanforderungen zusätzlich über sogenannte QoS-Profile (Quality of Service) technisch absichern, etwa bezüglich Zuverlässigkeit oder Aktualität einer Nachricht.

Für den Roboter lohnt sich eine kleine Schnittstellen-Checkliste:

- Ist die Richtung eindeutig?
- Ist der Datentyp bekannt?
- Sind Einheit und Frame dokumentiert?
- Gibt es einen Timeout (eine maximale Wartezeit, nach der fehlende Daten als Fehler gelten)?
- Ist bekannt, was bei ungültigen Daten passiert?
- Gibt es einen Test für die Schnittstelle?

10.10 Zusammenfassung

Schnittstellen sind entscheidend für die Integration. Eine gute Schnittstellenbeschreibung enthält Sender, Empfänger, Datentyp, Frequenz, Einheiten, Koordinatensysteme und Fehlerfälle. Im Roboterprojekt bilden Schnittstellen die Brücke zwischen SysML-Modell und ROS2-Kommunikation.

10.11 Kontrollfragen

1. Warum entstehen viele Fehler an Schnittstellen?
2. Welche Informationen gehören in eine Schnittstellenbeschreibung?
3. Warum sind Koordinatensysteme in Robotik wichtig?
4. Was bedeutet `base_link`?
5. Warum sollten Frequenzen dokumentiert werden?
6. Welche Fehlerfälle kann die Kameraschnittstelle haben?
7. Wie hängen IBD (Internal Block Diagram) und Schnittstellenmodellierung zusammen?

10.12 Übungen

Übung

Übung 1: Schnittstellentabelle

Erstellen Sie eine Schnittstellentabelle für mindestens sechs Datenflüsse des Roboters. Ergänzen Sie jeweils Sender, Empfänger, Datentyp, Frequenz und Fehlerfall.

Übung

Übung 2: Koordinatensysteme

Beschreiben Sie, wofür die Frames `map`, `odom`, `base_link` und `camera_link` im Roboterprojekt stehen könnten.

10.13 Literaturhinweise

Schnittstellenmodellierung ist ein zentrales Thema in SysML [?]. Für ROS2-Datentypen und Kommunikationsmuster ist die ROS2-Dokumentation [?] die wichtigste praktische Quelle.

11 Verhaltensmodellierung mit Activity Diagrams

Lernziele

Nach diesem Kapitel können Sie Abläufe des Roboters als Aktivitätsdiagramm beschreiben. Sie verstehen Aktionen, Kontrollflüsse, Entscheidungen, Parallelität und Objektflüsse und können typische Roboterprozesse wie Zielnavigation, Hindernisbehandlung und Rückkehr zur Ladestation modellieren.

11.1 Einleitung

Strukturdiagramme zeigen, woraus ein System besteht. Für ein funktionierendes System reicht das nicht. Wir müssen auch beschreiben, was im Betrieb passiert. Ein autonomer Roboter ist ein gutes Beispiel: Er nimmt Aufträge entgegen, plant Wege, fährt, reagiert auf Hindernisse, überwacht seine Batterie und meldet Zustände.

Activity Diagrams beschreiben solche Abläufe. Sie eignen sich besonders für Prozesse, Workflows und Algorithmen auf einer verständlichen Ebene.

11.2 Was beschreibt ein Aktivitätsdiagramm?

Ein Aktivitätsdiagramm beschreibt einen Ablauf aus Aktionen, Entscheidungen und Verzweigungen. Es zeigt, welche Schritte nacheinander oder parallel ausgeführt werden. In diesem Kapitel stellen wir Aktivitätsdiagramme bewusst über Prosa und Diagramme dar statt über die SysML-v2-Textnotation (`action def`, `action`, `then`, `decide`, `fork/join`); diese folgt denselben Prinzipien wie `state def` in Kapitel 12 und `calc def` in Kapitel 13, wird hier aber zugunsten der Einsteigerfreundlichkeit ausgespart.

Typische Elemente sind:

- Aktionen
- Kontrollflüsse
- Entscheidungsknoten
- Zusammenführungsknoten (führen mehrere alternative Pfade nach einer Entscheidung wieder zu einem einzigen Pfad zusammen)

- Start- und Endknoten
- parallele Aufspaltung und Synchronisation
- Objektflüsse (zeigen, wie ein Datenobjekt, z. B. ein geplanter Pfad, von einer Aktion zur nächsten weitergereicht wird, im Unterschied zum reinen Kontrollfluss)

11.3 Beispielablauf Navigation

1. System starten
2. Sensoren initialisieren
3. Karte laden
4. Ziel empfangen
5. Akkustand prüfen
6. Pfad planen
7. Bewegung ausführen
8. Hindernisse prüfen
9. Ziel erreicht melden

Dieser Ablauf kann als Aktivitätsdiagramm modelliert werden. Entscheidungen entstehen zum Beispiel bei niedrigem Akkustand oder blockiertem Pfad. Abbildung ?? zeigt den vollständigen Ablauf mit Start- und Endknoten, den beiden Entscheidungen und der parallelen Überwachung während der Bewegung.

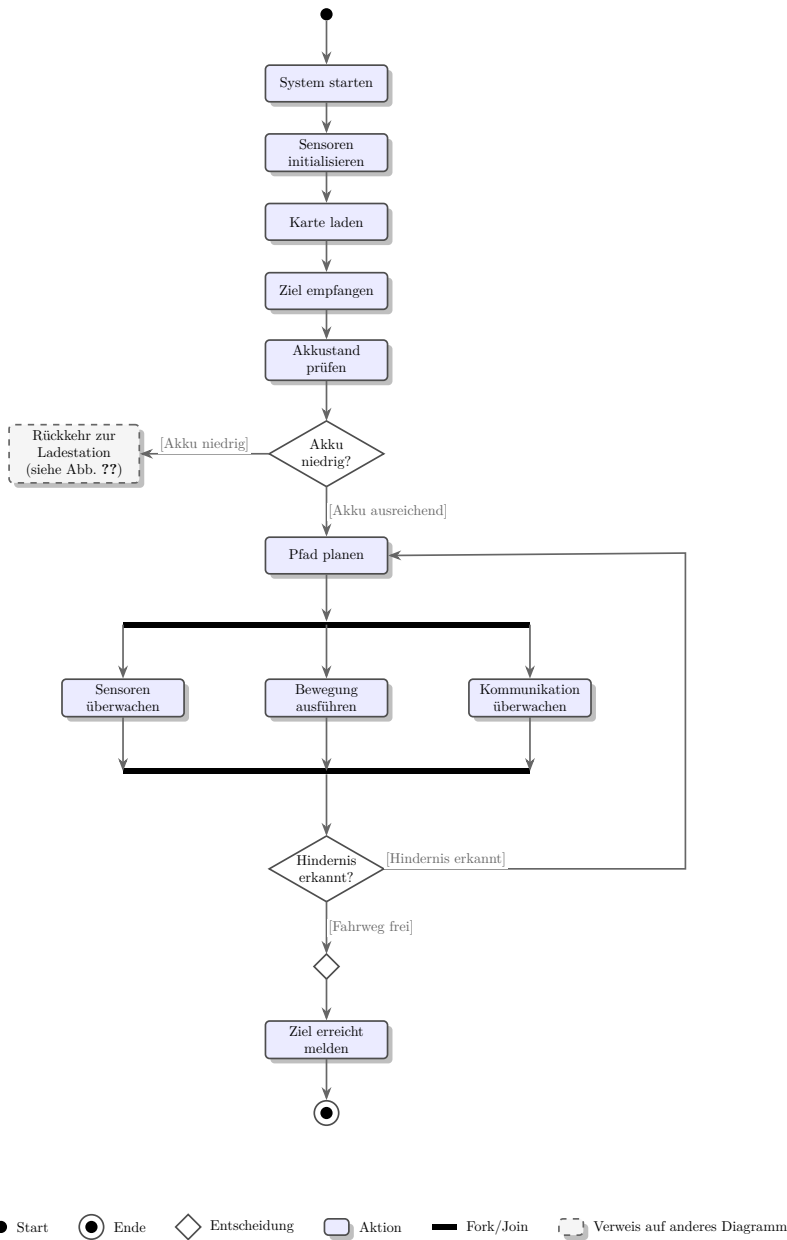


Figure 11.1: Aktivitätsdiagramm für die Zielnavigation: vom Systemstart über Pfadplanung und Hindernisbehandlung bis zur Meldung des erreichten Ziels. Bei niedrigem Akkustand wird auf den Ablauf ?Rückkehr zur Ladestation? verwiesen.

11.4 Entscheidungen

Entscheidungsknoten modellieren Alternativen. Beispiel: Wenn ein Hindernis erkannt wird, plant der Roboter neu oder stoppt. Entscheidungen sollten klare Bedingungen

haben, etwa [Hindernis erkannt] und [Fahrweg frei]; diese Bedingungen in eckigen Klammern werden in Kapitel 12.5 als *Guards* eingeführt und dort vertieft.

11.5 Parallelität

Ein Roboter führt oft mehrere Aktivitäten parallel aus. Während er fährt, überwacht er Sensoren, Batterie und Kommunikation. Aktivitätsdiagramme können solche parallelen Abläufe darstellen: Ein Fork-Knoten spaltet den Ablauf auf, ein Join-Knoten führt die parallelen Zweige wieder zusammen, etwa wenn parallel zur Fahrt die Aktionen ?Sensoren überwachen? und ?Kommunikation überwachen? laufen. Für Einsteiger ist es aber wichtig, Diagramme nicht zu überladen.

Praxisbeispiel

Der Ablauf ?Zur Ladestation fahren? enthält mehrere Entscheidungen: Ist der Akkustand niedrig? Ist die Ladestation erreichbar? Ist der Andockvorgang erfolgreich? Wenn nicht, muss der Roboter einen Fehler melden oder erneut versuchen.

11.6 Aktivitäten und Anforderungen

Aktivitätsdiagramme können Anforderungen verfeinern. Eine Anforderung ?Der Roboter muss ein Ziel autonom anfahren? wird durch einen Ablauf konkretisiert. Daraus ergeben sich weitere Anforderungen, Schnittstellen und Tests.

11.7 Typische Fehler

11.7.1 Zu viel Implementierungsdetail

Ein Aktivitätsdiagramm ist kein Ersatz für Programmcode. Es soll den Ablauf verständlich machen, nicht jede Codezeile modellieren.

11.7.2 Unklare Bedingungen

Entscheidungen brauchen Bedingungen. Ein Knoten ohne verständliche Bedingung hilft wenig.

11.7.3 Fehlerfälle vergessen

Gute Aktivitätsdiagramme betrachten nicht nur den Normalfall. Gerade bei Robotern sind Abbrüche, Blockaden und Sensorausfälle wichtig.

11.8 Praxisbeispiel: Rückkehr zur Ladestation

Der Ablauf 'Zur Ladestation zurückkehren' eignet sich besonders gut für ein Aktivitätsdiagramm. Er beginnt nicht erst mit der Fahrt zur Ladestation, sondern mit einer Entscheidung: Ist der Akkustand unterhalb eines Schwellwerts? Wenn ja, muss das System prüfen, ob gerade ein kritischer Auftrag läuft. Danach wird ein Ziel 'Ladestation' gesetzt, ein Pfad geplant und der Andockvorgang gestartet.

Ein möglicher Ablauf:

1. Akkustand überwachen.
2. Niedrigen Akkustand erkennen.
3. Nutzer informieren.
4. Aktuellen Auftrag bewerten.
5. Rückkehrziel setzen.
6. Pfad zur Ladestation planen.
7. Zur Ladestation fahren.
8. Andocken.
9. Ladevorgang starten.

Wichtig sind die Alternativen. Wenn die Ladestation blockiert ist, muss der Roboter warten, einen alternativen Platz anfahren oder einen Fehler melden. Wenn der Akku kritisch niedrig ist, darf er keine langen Ausweichmanöver mehr versuchen. Abbildung ?? stellt diesen gesamten Ablauf einschließlich der Alternativen bei blockierter Ladestation und bei fehlgeschlagenem Andocken dar.

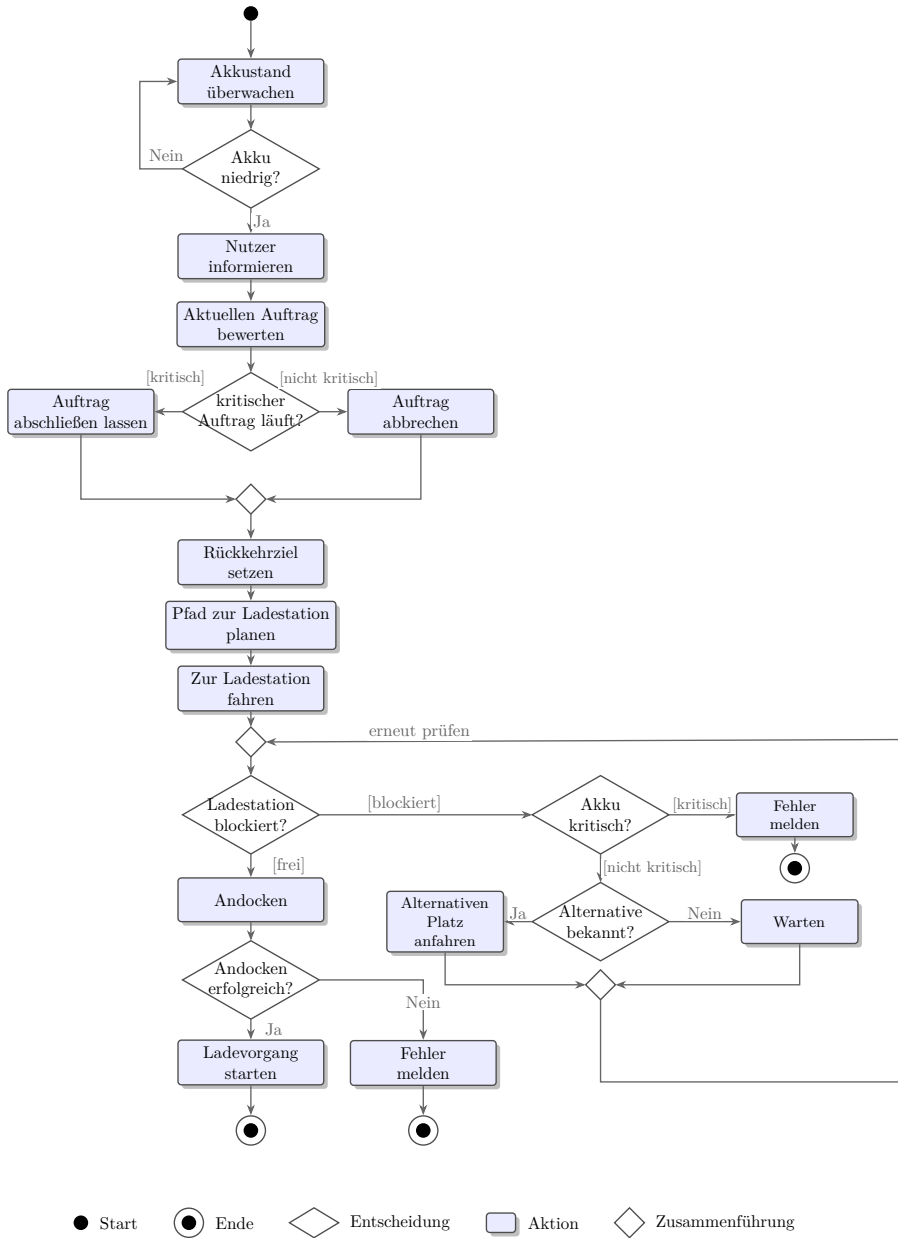


Figure 11.2: Aktivitätsdiagramm für die Rückkehr zur Ladestation: von der Akkuüberwachung über die Bewertung des laufenden Auftrags bis zum Andocken und Start des Ladevorgangs, inklusive der Alternativen bei blockierter Ladestation oder fehlgeschlagenem Andocken.

11.9 Zusammenfassung

Activity Diagrams beschreiben Abläufe und Entscheidungen. Im Roboterprojekt helfen sie, Use Cases wie Zielnavigation, Rückkehr zur Ladestation und Hindernisbehandlung in

nachvollziehbare Prozesse zu übersetzen. Sie bilden eine Brücke zwischen Anforderungen und späterer Softwarelogik.

11.10 Kontrollfragen

1. Wozu dient ein Aktivitätsdiagramm?
2. Was ist eine Aktion?
3. Wozu dienen Entscheidungsknoten?
4. Warum ist Parallelität bei Robotern relevant?
5. Warum sollte ein Aktivitätsdiagramm nicht zu detailliert sein?
6. Wie kann ein Aktivitätsdiagramm Anforderungen verfeinern?

11.11 Übungen

Übung

Übung 1: Ziel anfahren

Modellieren Sie den Ablauf ?Ziel anfahren?. Ergänzen Sie mindestens eine Entscheidung für Hinderniserkennung und eine für niedrigen Akkustand.

Übung

Übung 2: Fehlerfall ergänzen

Erweitern Sie den Ablauf um den Fall, dass keine Karte verfügbar ist. Beschreiben Sie die erwartete Systemreaktion.

11.12 Literaturhinweise

Aktivitätsdiagramme werden in SysML und UML ähnlich verwendet. Für praxisnahe Beispiele eignen sich SysML-Einführungen [?, ?].

12 Zustandsmodellierung mit State Machines

Lernziele

Nach diesem Kapitel können Sie Betriebszustände und Zustandswechsel eines Roboters modellieren. Sie verstehen Zustände, Ereignisse, Übergänge, Start- und Endzustände sowie einfache Guard-Bedingungen und können eine State Machine für den autonomen Indoor-Roboter entwerfen.

12.1 Einleitung

Ein Roboter verhält sich nicht in jeder Situation gleich. Wenn er lädt, soll er nicht losfahren. Wenn er einen Fehler hat, soll er keine neuen Aufträge ausführen. Wenn der Akku niedrig ist, soll er zur Ladestation zurückkehren. Solche Fragen lassen sich mit State Machines sehr gut modellieren.

12.2 Zustände

Ein Zustand beschreibt eine Situation, in der sich das System über eine gewisse Zeit befindet. Beispiele für den Roboter sind:

- `Idle`
- `Navigating`
- `AvoidingObstacle`
- `ReturnToDock`
- `Charging`
- `Error`

Im Fließtext schreiben wir Zustände zur besseren Lesbarkeit in PascalCase (`Idle`, `Navigating`); im SysML-v2-Code weiter unten und im Zustandsdiagramm erscheinen dieselben Zustände in camelCase (`idle`, `navigating`) – gemeint ist jeweils derselbe Zustand.

12.3 Ereignisse

Ereignisse lösen Übergänge aus. Beispiele:

- GoalReceived
- BatteryLow
- ObstacleDetected
- PathFree
- DockReached
- ChargingComplete
- ErrorDetected
- Reset

12.4 Beispiel

In SysML v2 wird eine State Machine mit `state def` modelliert. Jeder Zustand kann Übergänge mit `transition` enthalten; die Zeile `entry; then idle;` markiert dabei den Startzustand (auch *Pseudozustand* genannt), von dem die State Machine beim Systemstart automatisch in den Zustand `idle` wechselt:

```

1 state def RobotOperation {
2   entry; then idle;
3
4   state idle {
5     transition accept GoalReceived then navigating;
6   }
7   state navigating {
8     transition accept ObstacleDetected then avoidingObstacle;
9     transition accept BatteryLow then returnToDock;
10    transition accept ErrorDetected then error;
11  }
12  state avoidingObstacle {
13    transition accept PathFree then navigating;
14  }
15  state returnToDock {
16    transition accept DockReached then charging;
17  }
18  state charging {
19    transition accept ChargingComplete then idle;
20  }
21  state error {
22    transition accept Reset then idle;
23  }
24 }
```

Abbildung ?? stellt diesen Zustandsautomaten grafisch dar.

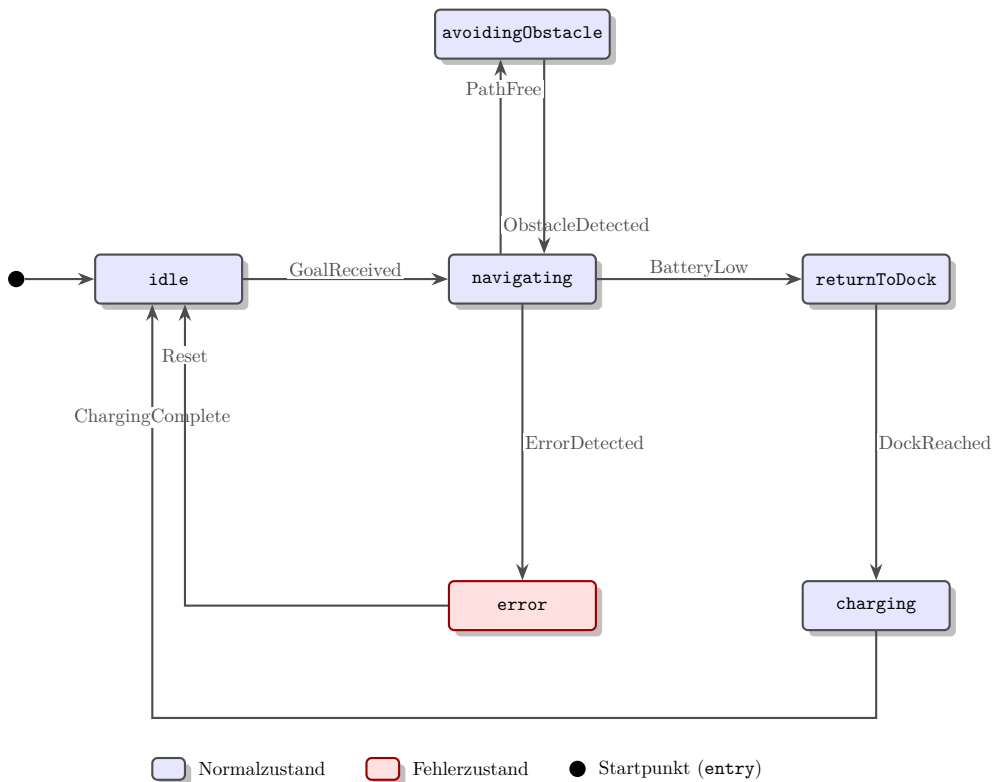


Figure 12.1: Zustandsdiagramm zu `state def RobotOperation`. Jeder Pfeil entspricht einer `transition accept ... then ...`-Zeile aus dem Listing oben. Der Fehlerzustand ist rot hervorgehoben.

12.5 Guard-Bedingungen

Guards sind Bedingungen an Übergängen. Ein Übergang findet nur statt, wenn die Bedingung erfüllt ist. Beispiel: `GoalReceived [batteryOk]` führt zu `Navigating`. Wenn der Akku nicht ausreichend ist, führt das Ereignis vielleicht zu `ReturnToDock` oder zu einer Ablehnung des Auftrags. Im Codebeispiel weiter oben ist dieser Guard der Übersichtlichkeit halber weggelassen; ergänzt sähe die Zeile im `idle`-Zustand etwa so aus: `transition accept GoalReceived[batteryOk] then navigating;`

12.6 Warum State Machines nützlich sind

Zustandsdiagramme verhindern unklare Logik. Sie zeigen, welche Übergänge erlaubt sind. Darf der Roboter während eines Fehlers ein neues Ziel annehmen? Darf er während des Ladens navigieren? Was passiert, wenn während der Navigation der Akku niedrig wird?

Praxisbeispiel

Ohne Zustandsmodell landet solche Logik oft verteilt im Code. Ein Node prüft den Akku, ein anderer prüft Ziele, ein dritter behandelt Fehler. Eine State Machine schafft eine gemeinsame Sicht auf erlaubte Betriebsmodi.

12.7 State Machines und Tests

Aus Zustandsdiagrammen lassen sich Tests ableiten. Jeder wichtige Übergang kann geprüft werden: Ziel empfangen, Hindernis erkannt, Akku niedrig, Ladestation erreicht, Fehler erkannt, Reset durchgeführt.

12.8 Typische Fehler

12.8.1 Zustände und Aktionen verwechseln

`Navigating` ist ein Zustand. `Pfad planen` ist eher eine Aktion. Zustände sollten eine gewisse Dauer haben.

12.8.2 Zu viele Zustände

Ein Diagramm mit zu vielen Zuständen wird schwer lesbar. Beginnen Sie mit den wichtigsten Betriebsmodi.

12.8.3 Fehlerzustand vergessen

Jedes reale System braucht eine Vorstellung davon, was bei Fehlern passiert.

12.9 Praxisbeispiel: Betriebsmodi des Roboters

Für den Indoor-Roboter ist eine State Machine auf Systemebene besonders hilfreich. Sie legt fest, welche Betriebsmodi grundsätzlich erlaubt sind. Ein einfaches Modell beginnt mit `Idle`. Von dort kann der Roboter bei einem Zielauftrag in `Navigating` wechseln. Erkennt er einen niedrigen Akkustand, wechselt er zu `ReturnToDock`. Bei erfolgreichem Andocken wechselt er zu `Charging`.

Der Zustand `Error` sollte grundsätzlich aus mehreren Zuständen erreichbar sein, auch wenn das Codebeispiel oben aus Gründen der Übersichtlichkeit nur den Übergang von `navigating` zeigt. Ein Fehler kann während der Navigation, beim Laden oder während der Initialisierung auftreten; ein vollständigeres Modell würde daher zusätzliche Übergänge wie `transition accept ErrorDetected then error;` auch in `charging` und einem möglichen Initialisierungszustand ergänzen. Wichtig ist, dass der Rückweg aus dem Fehlerzustand bewusst modelliert wird. Darf ein Nutzer einfach Reset drücken? Muss Wartungspersonal eingreifen? Muss der Roboter vorher die Motoren deaktivieren?

State Machines helfen außerdem bei der Bedienoberfläche. Wenn das System im Zustand **Charging** ist, sollte die Oberfläche keine normale Zielnavigation anbieten oder zumindest klar anzeigen, dass der Roboter gerade nicht verfügbar ist.

12.10 Zusammenfassung

State Machines beschreiben Betriebszustände und erlaubte Übergänge. Für den Indoor-Roboter sind sie besonders wichtig, um Navigation, Laden, Fehlerbehandlung und Sicherheitsreaktionen sauber zu modellieren.

12.11 Kontrollfragen

1. Was ist ein Zustand?
2. Was ist ein Ereignis?
3. Was ist ein Übergang?
4. Was ist eine Guard-Bedingung?
5. Warum ist ein Fehlerzustand wichtig?
6. Welche Zustände sollte der Indoor-Roboter mindestens besitzen?
7. Wie können State Machines Tests unterstützen?

12.12 Übungen

Übung

Übung 1: State Machine erstellen

Erstellen Sie eine State Machine für den Roboter mit mindestens sechs Zuständen und acht Übergängen.

Übung

Übung 2: Guards ergänzen

Ergänzen Sie Bedingungen für die Übergänge `GoalReceived`, `BatteryLow` und `ErrorDetected`.

12.13 Literaturhinweise

State Machines sind ein etabliertes Mittel zur Verhaltensmodellierung. SysML-Literatur [?, ?] zeigt, wie sie mit Anforderungen und Tests verbunden werden können.

13 Parametrische Modellierung

Lernziele

Nach diesem Kapitel verstehen Sie, wie technische Zusammenhänge im Modell beschrieben werden können. Sie können einfache Formeln zu Akkulaufzeit, Leistungsaufnahme und Bremsweg verwenden und erklären, wie SysML v2 **calc**, **def** und **constraint** für quantitative Aussagen genutzt werden.

13.1 Einleitung

Bisher haben wir Anforderungen, Struktur, Schnittstellen und Verhalten betrachtet. Viele technische Entscheidungen benötigen zusätzlich Zahlen. Wie lange fährt der Roboter mit einer Akkuladung? Wie viel Leistung benötigen Rechner, Sensoren und Motoren? Wie groß muss der Sicherheitsabstand bei einer bestimmten Geschwindigkeit sein?

Parametrische Modellierung hilft, solche Zusammenhänge sichtbar zu machen.

13.2 Zweck parametrischer Modelle

Parametrische Modelle beschreiben mathematische Beziehungen. Sie helfen, technische Entscheidungen früh zu prüfen. Beim Roboter sind Energieverbrauch, Reichweite, Geschwindigkeit, Nutzlast und Bremsweg wichtige Größen.

13.3 Einfaches Beispiel: Betriebszeit

$$t_{\text{betrieb}} = \frac{E_{\text{akku}}}{P_{\text{gesamt}}}$$

Dabei ist E_{akku} die nutzbare Energie der Batterie und P_{gesamt} die durchschnittliche Leistungsaufnahme.

Wenn die Batterie 100 Wh nutzbare Energie besitzt und das System im Mittel 25 W aufnimmt, ergibt sich:

$$t_{\text{betrieb}} = \frac{100 \text{ Wh}}{25 \text{ W}} = 4 \text{ h}$$

13.4 Leistungsaufnahme

Die Gesamtleistung kann grob abgeschätzt werden:

$$P_{\text{gesamt}} = P_{\text{rechner}} + P_{\text{sensoren}} + P_{\text{motoren}} + P_{\text{sonstiges}}$$

Diese Formel ist einfach, aber nützlich für frühe Architekturentscheidungen. Wenn ein stärkerer Rechner 15 W mehr benötigt, sinkt die Betriebszeit. Das kann Anforderungen an Batteriegröße, Gewicht und Ladezeit beeinflussen.

13.5 Bremsweg

Auch Sicherheitsfragen lassen sich grob parametrisch betrachten. Ein vereinfachter Bremsweg kann mit folgender Beziehung abgeschätzt werden:

$$s = \frac{v^2}{2a}$$

Dabei ist v die Geschwindigkeit und a die Bremsverzögerung. Die Formel ist vereinfacht, zeigt aber einen wichtigen Zusammenhang: Wenn die Geschwindigkeit steigt, wächst der Bremsweg quadratisch. Analog zu `OperatingTimeCalc` ließe sich das als `calc def BrakingDistanceCalc { in v: Real; in a: Real; return s: Real = v*v/(2*a); }` formulieren. Abbildung ?? veranschaulicht diesen quadratischen Zusammenhang für Geschwindigkeiten zwischen 0 und 1,5 m/s.

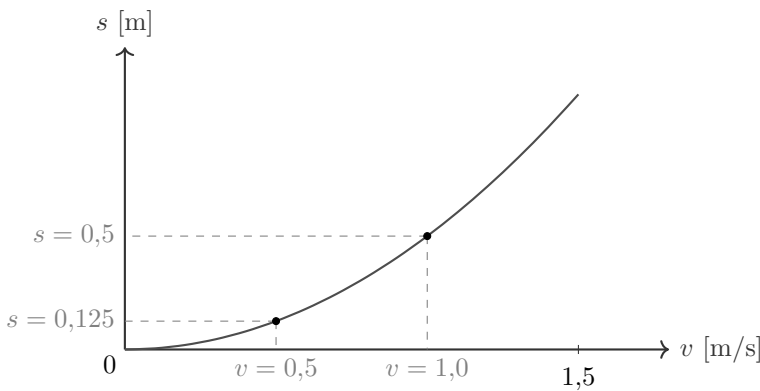


Figure 13.1: Bremsweg $s = v^2/(2a)$ in Abhängigkeit von der Geschwindigkeit v (hier mit $a = 1 \text{ m/s}^2$). Der Bremsweg wächst quadratisch; bei $v = 0,5 \text{ m/s}$ und $v = 1,0 \text{ m/s}$ sind die Werte markiert.

Als parametrisches Diagramm lässt sich der Bremsweg ebenfalls modellieren (siehe Abbildung ??): Die Geschwindigkeit des Roboters und die Bremsverzögerung des `BrakeSystem` fließen in `BrakingDistanceCalc` ein, dessen Ergebnis von der Bedingung `SafeBrakingDistance` gegen den maximal zulässigen Bremsweg geprüft wird.

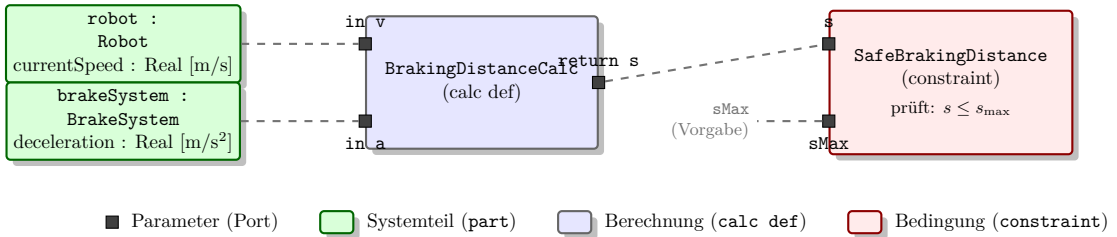


Figure 13.2: Parametrisches Diagramm zum Bremsweg: Geschwindigkeit und Bremsverzögerung fließen in **BrakingDistanceCalc** ein, das Ergebnis wird von der Bedingung **SafeBrakingDistance** gegen den maximal zulässigen Bremsweg geprüft.

Praxisbeispiel

Wenn der Roboter schneller fahren soll, reicht es nicht, nur die Motorsteuerung anzupassen. Auch Hinderniserkennung, Reaktionszeit, Bremsweg, Sicherheitsabstand und Tests müssen betrachtet werden.

13.6 Parametric Diagram in SysML

In SysML v2 werden solche Zusammenhänge mit **calc def** (Berechnungen) und **constraint** (Bedingungen) modelliert. Eine **calc def** beschreibt eine Formel. Parameter werden als Attribute mit Systemteilen verbunden. Grafisch besteht ein Parametric Diagram aus Kästen für **calc def**- und **constraint**-Verwendungen mit ihren Parametern am Rand, die über gestrichelte Bindungslinien mit den Attributen der beteiligten Systemteile verbunden werden – so wie es die Abbildungen in diesem Abschnitt zeigen.

Beispiel:

- **calc def**: **OperatingTimeCalc**
- Formel: $t = E/P$
- Parameter: t , E , P
- Verbindung: E mit Batteriekapazität, P mit Leistungsaufnahme

Als Code sieht dieselbe **calc def** so aus:

```

1 calc def OperatingTimeCalc {
2   in energy : Real; // nutzbare Energie in Wh
3   in power : Real; // mittlere Leistungsaufnahme in W
4   return operatingTime : Real = energy / power;
5 }
```

Die **in**-Parameter **energy** und **power** – **in** kennzeichnet dabei die Parameterrichtung als Eingang, analog gibt es **return** für das Ergebnis – werden im Modell mit den Attributen von Batterie und Verbrauchern verbunden. Das Ergebnis **operatingTime** steht dann als berechneter Wert am Systemteil zur Verfügung. Abbildung ?? zeigt dieses parametrische Diagramm grafisch: Die gestrichelten Bindungslinien verbinden die Parameter von **OperatingTimeCalc** mit den Attributen von **Battery**, **PowerConsumers** und **Mission**.

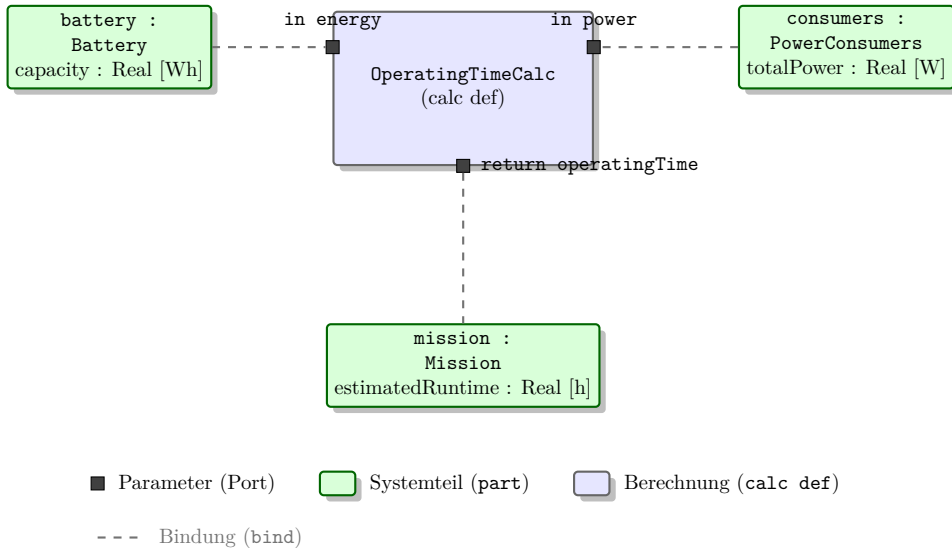


Figure 13.3: Parametrisches Diagramm zur Berechnung der Betriebszeit: **OperatingTimeCalc** verbindet die Batteriekapazität und die Leistungsaufnahme der Verbraucher zur geschätzten Laufzeit der Mission.

Neben **calc def** bietet SysML v2 mit **constraint def** ein Konstrukt für Bedingungen, die eine berechnete Größe gegen einen Grenzwert prüfen. Für den Bremsweg aus Abschnitt 13.5 ließe sich das so formulieren:

```

1 constraint def SafeBrakingDistance {
2   in s : Real; // berechneter Bremsweg in m
3   in sMax : Real; // maximal zulaessiger Bremsweg in m
4   assert s <= sMax;
5 }

```

Die Bedingung **assert s <= sMax** ist genau dann erfüllt, wenn der berechnete Bremsweg den zulässigen Grenzwert nicht überschreitet; verletzt sie, weist das Modell auf eine unsichere Kombination aus Geschwindigkeit und Bremsverzögerung hin.

13.7 Grenzen einfacher Modelle

Parametrische Modelle sind nur so gut wie ihre Annahmen. Ein Roboter verbraucht beim Anfahren mehr Energie als im Stillstand. Teppichboden, Nutzlast und Fahrprofil beeinflussen die Laufzeit. Ein einfaches Modell ist trotzdem nützlich, weil es früh Größenordnungen sichtbar macht.

13.8 Parametrik als Entscheidungswerkzeug

Parametrik ist besonders wertvoll, wenn mehrere Entwurfsvarianten verglichen werden. Nehmen wir an, das Team betrachtet zwei Rechner. Rechner A benötigt 10 W und schafft einfache Personenerkennung. Rechner B benötigt 25 W und erkennt Personen zuverlässiger. Die Entscheidung betrifft nicht nur die Qualität des maschinellen Lernens (ML), sondern auch Akkulaufzeit, Wärme, Kosten und Gewicht.

Ein parametrisches Modell kann solche Auswirkungen sichtbar machen. Wenn die restliche Leistungsaufnahme 20 W beträgt, ergibt sich mit Rechner A eine Gesamtleistung von 30 W. Mit Rechner B steigt sie auf 45 W. Bei 100 Wh nutzbarer Energie sinkt die theoretische Betriebszeit von etwa 3,3 h auf etwa 2,2 h.

Das Modell trifft die Entscheidung nicht automatisch. Es zeigt aber, welche Fragen diskutiert werden müssen: Ist die bessere Erkennung die kürzere Laufzeit wert? Kann ein größerer Akku eingebaut werden? Wird das System zu schwer? Reicht ein optimiertes Modell auf Rechner A?

13.9 Zusammenfassung

Parametrische Modellierung ergänzt qualitative Modelle um quantitative Zusammenhänge. Für den Indoor-Roboter sind Akkulaufzeit, Leistungsaufnahme und Bremsweg gute Einstiegsthemen. Parametrik hilft, Trade-offs zwischen Leistung, Gewicht, Sicherheit und Kosten zu verstehen.

13.10 Kontrollfragen

1. Wozu dient parametrische Modellierung?
2. Welche Größen beeinflussen die Betriebszeit?
3. Warum steigt der Bremsweg stark mit der Geschwindigkeit?
4. Was ist `calc def` in SysML v2?
5. Warum sind Annahmen in parametrischen Modellen wichtig?
6. Welche Trade-offs kann Parametrik sichtbar machen?

13.11 Übungen

Übung

Übung 1: Betriebszeit berechnen

Nehmen Sie eine Batteriekapazität von 100 Wh und eine Leistungsaufnahme von 25 W an. Berechnen Sie die theoretische Betriebszeit.

Übung

Übung 2: Leistungsänderung

Der Rechner benötigt statt 10 W nun 20 W. Die übrige Leistungsaufnahme beträgt 20 W. Berechnen Sie die neue Betriebszeit bei 100 Wh.

Übung

Übung 3: Bremsweg vergleichen

Vergleichen Sie den Bremsweg für 0,5 m/s und 1,0 m/s bei gleicher Bremsverzögerung. Beschreiben Sie, was das für Sicherheitsanforderungen bedeutet.

13.12 Literaturhinweise

Parametrische Diagramme werden in SysML-Literatur als Mittel für technische Analysen beschrieben [?]. Für reale Robotikprojekte sollten zusätzlich Fachquellen zu Batterietechnik, Antriebstechnik und Sicherheit herangezogen werden.

14 ROS2-Grundlagen

Lernziele

Nach diesem Kapitel kennen Sie die wichtigsten Grundbegriffe von ROS2: Nodes, Topics, Messages, Services, Actions, Parameter und Launch-Dateien. Sie können diese Begriffe auf den autonomen Indoor-Roboter anwenden und verstehen, warum ROS2 gut zu einer modellbasiert vorbereiteten Architektur passt.

14.1 Einleitung

Bis hierher haben wir den Roboter vor allem aus Systems-Engineering- und SysML-Sicht betrachtet. Nun wechseln wir näher zur Umsetzung. ROS2 ist die Softwareplattform, mit der wir die Roboterfunktionen später in ausführbare Komponenten übersetzen können. Dieses Kapitel führt die Grundbegriffe zunächst ohne Code ein; ein vollständiges, lauffähiges Beispiel für einen Publisher- und Subscriber-Node in Python (`rclpy`, der ROS-Client-Bibliothek für Python) finden Sie in Kapitel 17.

14.2 Was ist ROS2?

Robot Operating System 2 (ROS2) ist ein Framework für Robotiksoftware. Es stellt Kommunikationsmechanismen, Werkzeuge und Konventionen bereit. ROS2 ist kein Betriebssystem im klassischen Sinn, sondern eine **Middleware** und Entwicklungsumgebung. Technische Basis dafür ist DDS (Data Distribution Service), ein Standard für verteilte Kommunikation, den ROS2 im Hintergrund nutzt, um Nachrichten zwischen Nodes zu verteilen. Ob diese Nachrichten zuverlässig, nur bestmöglich, mit gespeicherten letzten Werten oder mit einer bestimmten Historie übertragen werden, hängt in ROS2 von den gewählten **Quality of Service (QoS)**-Einstellungen ab.

ROS2 hilft dabei, ein Robotiksystem aus mehreren Programmen aufzubauen, die miteinander Nachrichten austauschen. Das passt sehr gut zu unserem SysML-Modell: Systemteile (`part def`), Schnittstellen und Datenflüsse können später auf Nodes, Topics, Services und Actions abgebildet werden.

14.3 Nodes

Ein Node ist ein ausführbares Programm mit einer klaren Aufgabe. Beispiele:

- `camera_node`: liest Kamerabilder

- `lidar_node`: liest Laserscans
- `object_detector_node`: erkennt Objekte oder Personen
- `navigation_node`: plant Bewegung
- `battery_monitor_node`: überwacht den Akkustand

Ein guter Node hat eine klare Verantwortung. Zu große Nodes werden unübersichtlich. Zu viele kleine Nodes erhöhen Kommunikationsaufwand und Komplexität.

14.4 Topics

Topics sind Nachrichtenkanäle. Ein Node veröffentlicht Nachrichten (er handelt dabei als *Publisher*), andere Nodes abonnieren sie (sie handeln als *Subscriber*). Diese Publish-Subscribe-Kommunikation passt sehr gut zu Sensordaten. Dabei kann ein Publisher gleichzeitig von mehreren Subscribern empfangen werden (siehe Abbildung ??). Wie zuverlässig Nachrichten dabei zugestellt werden, regeln sogenannte Quality-of-Service-Einstellungen, die in Kapitel 16 genauer erklärt werden.

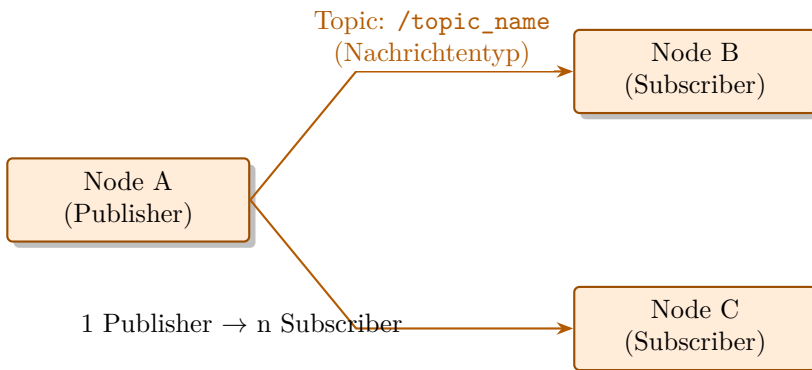


Figure 14.1: Publish-Subscribe-Prinzip: Ein Publisher-Node sendet Nachrichten auf einem Topic, mehrere Subscriber-Nodes können sie gleichzeitig empfangen.

Beispiele:

<code>/camera/image_raw</code>	Kamerabild
<code>/scan</code>	Laserscan
<code>/cmd_vel</code>	Bewegungsbefehl
<code>/battery_state</code>	Akkustatus
<code>/detected_objects</code>	erkannte Objekte

Diese Namen sind keine ROS2-Vorgabe, sondern eine in diesem Buch verwendete Projektkonvention; in eigenen Projekten können Sie andere Topic-Namen wählen, sollten dabei aber konsistent bleiben.

14.5 Messages

Messages definieren die Struktur der Daten, die über Topics, Services oder Actions übertragen werden. ROS2 bringt viele Standardnachrichten mit, etwa `sensor_msgs/Image`, `sensor_msgs/LaserScan`, `geometry_msgs/Twist` oder `sensor_msgs/BatteryState`.

14.6 Services

Services sind Anfrage-Antwort-Kommunikation. Sie eignen sich für kurze, gezielte Abfragen oder Befehle, bei denen eine direkte Antwort erwartet wird. Beispiel: Eine Benutzerschnittstelle fragt einen Node, ob eine Karte verfügbar ist.

14.7 Actions

Actions sind für länger laufende Aufgaben gedacht. Navigation zu einem Ziel ist ein typischer Action-Anwendungsfall, weil der Vorgang länger dauert, Feedback liefern kann und abgebrochen werden muss.

Praxisbeispiel

?Fahre zu Raum 205? ist kein kurzer Service-Aufruf. Die Fahrt kann dauern, zwischendurch Feedback liefern und fehlschlagen. Deshalb passt eine ROS2-Action besser als ein einfacher Service.

14.8 Parameter

Parameter erlauben Konfiguration. Ein Node kann Parameter wie maximale Geschwindigkeit, Akkuswellwert oder Kamerafrequenz besitzen. Parameter sollten zu Modellannahmen und Anforderungen passen.

14.9 Launch-Dateien

Launch-Dateien starten mehrere Nodes gemeinsam mit passenden Parametern. Für den Roboter könnte eine Launch-Datei Kamera, Lidar, Navigation, Batteriemonitor und Objekterkennung starten.

14.10 Zusammenhang mit SysML

SysML beschreibt die Architektur auf Modellebene. ROS2 setzt sie technisch um. Ein SysML-Systemelement (`part def`) kann zu einem Node werden. Ein Verbindungsmodell-Datenfluss kann zu einem Topic werden. Ein Use Case ?Ziel anfahren? kann später eine Action nutzen.

14.11 Typische Fehler

14.11.1 Nodes nach Zufall schneiden

Nodes sollten klare Verantwortlichkeiten haben. Ein Node für alles ist schwer wartbar, ein Node pro Kleinigkeit ist übertrieben.

14.11.2 Topics schlecht benennen

Topic-Namen sollten verständlich und konsistent sein.

14.11.3 Services für lange Vorgänge verwenden

Lange Vorgänge mit Feedback passen besser zu Actions.

14.12 ROS2-Werkzeuge für Einsteiger

ROS2 bringt Kommandozeilenwerkzeuge mit, die beim Verstehen und Testen helfen. Für Einsteiger sind sie besonders wertvoll, weil sie sichtbar machen, was im laufenden System passiert.

- `ros2 node list`: zeigt laufende Nodes.
- `ros2 topic list`: zeigt verfügbare Topics.
- `ros2 topic echo`: zeigt Nachrichten auf einem Topic.
- `ros2 topic hz`: misst die Veröffentlichungsfrequenz.
- `ros2 service list`: zeigt Services.
- `ros2 action list`: zeigt Actions.

Diese Werkzeuge passen gut zur Modellprüfung. Wenn im SysML-Modell ein Topic `/battery_state` vorgesehen ist, sollte es im laufenden ROS2-System auffindbar sein. Wenn ein Node `object_detector_node` modelliert wurde, sollte er später auch als laufender Node erscheinen.

14.13 Zusammenfassung

ROS2 stellt die grundlegenden Kommunikations- und Strukturmechanismen für die Roboterimplementierung bereit. Nodes übernehmen Aufgaben, Topics transportieren Nachrichten, Services beantworten kurze Anfragen, Actions steuern längere Vorgänge. Diese Konzepte lassen sich gut aus einem SysML-Modell ableiten. Am Indoor-Roboter zeigt sich das Zusammenspiel exemplarisch: Der `camera_node` veröffentlicht Bilder als Topic, der `object_detector_node` abonniert sie, ein Service liefert auf Anfrage den Kartenstatus, und eine Action steuert die länger dauernde Fahrt zur Ladestation.

14.14 Kontrollfragen

1. Was ist ROS2?
2. Was ist ein Node?
3. Was ist ein Topic?
4. Wann verwendet man einen Service?
5. Wann verwendet man eine Action?
6. Welche ROS2-Nachricht passt zu einem Kamerabild?
7. Warum passen SysML-Datenflüsse gut zu ROS2-Topics?

14.15 Übungen

Übung

Übung 1: Nodes und Topics

Ordnen Sie den Funktionen Kamera, Navigation, Batterieüberwachung und Objekterkennung passende ROS2-Nodes und Topics zu.

Übung

Übung 2: Service oder Action

Entscheiden Sie für folgende Vorgänge, ob Topic, Service oder Action besser passt: Akkustatus melden, Ziel anfahren, Karte speichern, Kamerabild übertragen.

14.16 Literaturhinweise

Für ROS2 ist die offizielle Dokumentation die wichtigste Quelle [?]. Für dieses Buch genügt zunächst ein Grundverständnis der Kommunikationskonzepte.

15 Architektur von ROS2-Systemen

Lernziele

Nach diesem Kapitel können Sie eine einfache ROS2-Architektur für den autonomen Roboter entwerfen. Sie verstehen Node-Granularität, Kommunikationsstruktur, Fehlerfälle, Parameter und Lifecycle-Überlegungen.

15.1 Einleitung

Eine ROS2-Architektur beschreibt, aus welchen Nodes ein System besteht und wie diese Nodes miteinander kommunizieren. Sie ist die technische Entsprechung vieler SysML-Struktur- und Schnittstellenentscheidungen.

15.2 Typische Node-Struktur

Eine sinnvolle erste Architektur besteht aus folgenden Nodes:

- `camera_node`
- `lidar_node`
- `slam_node`
- `navigation_node`
- `object_detector_node`
- `battery_monitor_node`
- `motion_control_node`
- `user_interface_node`
- `robot_state_node`

15.3 Kommunikationsstruktur

```

camera_node -> /camera/image_raw -> object_detector_node
lidar_node   -> /scan                    -> slam_node
slam_node    -> /map                     -> navigation_node
navigation_node -> /cmd_vel              -> motion_control_node
battery_monitor_node -> /battery_state -> navigation_node
object_detector_node -> /detected_objects -> navigation_node
robot_state_node -> /robot/status -> user_interface_node

```

Der vollständige Node-Graph mit allen neun Nodes und ihren Topic-Verbindungen ist in Abbildung ?? dargestellt.

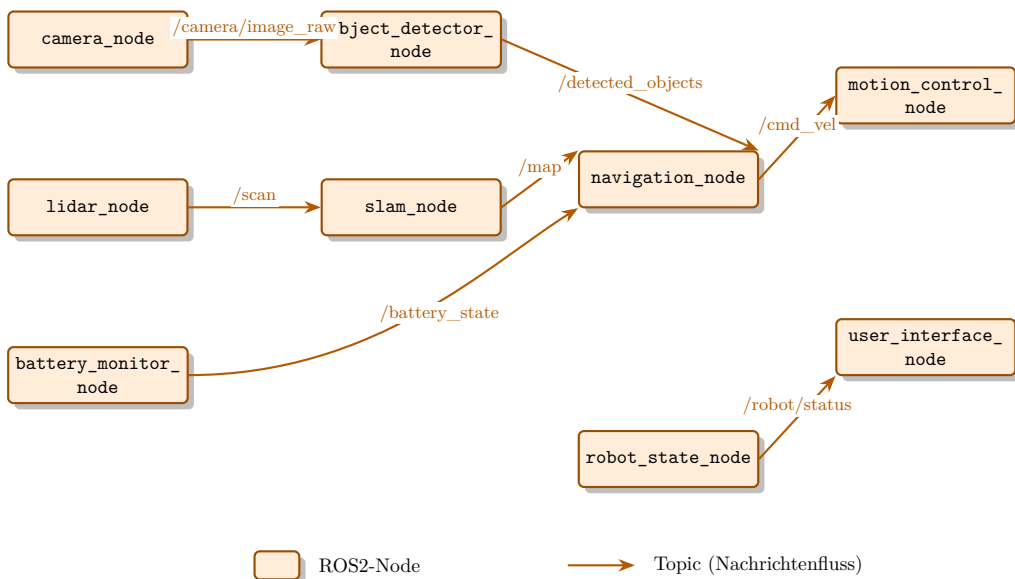


Figure 15.1: Node-Graph des autonomen Indoor-Roboters mit den wichtigsten Topics zwischen Sensor-, Verarbeitungs-, Navigations- und Aktuatorik-Nodes.

15.4 Granularität

Eine wichtige Entscheidung ist die Granularität. Zu viele Nodes erhöhen Kommunikationsaufwand. Zu wenige Nodes machen das System unübersichtlich. Für den Einstieg gilt: Eine klar abgegrenzte Funktion darf ein eigener Node sein. Ein Node, der gleichzeitig Kamera, Lidar und Navigation übernimmt, wäre zu grob geschnitten; ein eigener Node für jede einzelne Sensor-Lesefunktion wäre dagegen meist zu fein.

15.5 Architekturentscheidungen

Wichtige Entscheidungen sind:

- Welche Funktionen werden eigene Nodes?
- Welche Daten werden als Topics veröffentlicht?
- Welche Vorgänge werden als Actions modelliert?
- Welche Parameter müssen konfigurierbar sein?
- Welche Nodes sind sicherheitsrelevant?
- Welche Fehler müssen zentral gemeldet werden?

15.6 Fehlerfälle

Eine Architektur sollte nicht nur den Normalfall zeigen. Was passiert, wenn die Kamera ausfällt? Was passiert bei leerem Akku? Was passiert, wenn keine Karte verfügbar ist?

Praxisbeispiel

Wenn `camera_node` keine Bilder liefert, darf die Personenerkennung nicht einfach alte Ergebnisse weiterverwenden. Der `object_detector_node` sollte fehlende Daten erkennen und einen entsprechenden Status melden.

15.7 Parameter

Typische Parameter sind maximale Geschwindigkeit, Akkuschwelldwert, Topic-Namen, Erkennungsgrenzen oder Timeout-Werte. Parameter sollten im Modell mit Annahmen und Anforderungen zusammenpassen.

15.8 Lifecycle Nodes

ROS2 unterstützt Lifecycle Nodes, also Nodes mit einem definierten Zustandsautomaten zur kontrollierten Aktivierung und Deaktivierung. Sie besitzen definierte Zustände wie *unconfigured*, *inactive*, *active* und *finalized*. Für Einsteiger ist das optional, aber für robuste Systeme nützlich: Der Zustandsautomat ermöglicht zum Beispiel ein deterministisches Hochfahren, bei dem ein Sensor-Node zunächst seine Parameter lädt (Zustand *configure*), bevor er aktiv Daten liefert (Zustand *active*) – so startet die Navigation nicht mit ungültigen oder unvollständigen Daten. Abbildung ?? zeigt den zugehörigen Zustandsautomaten.

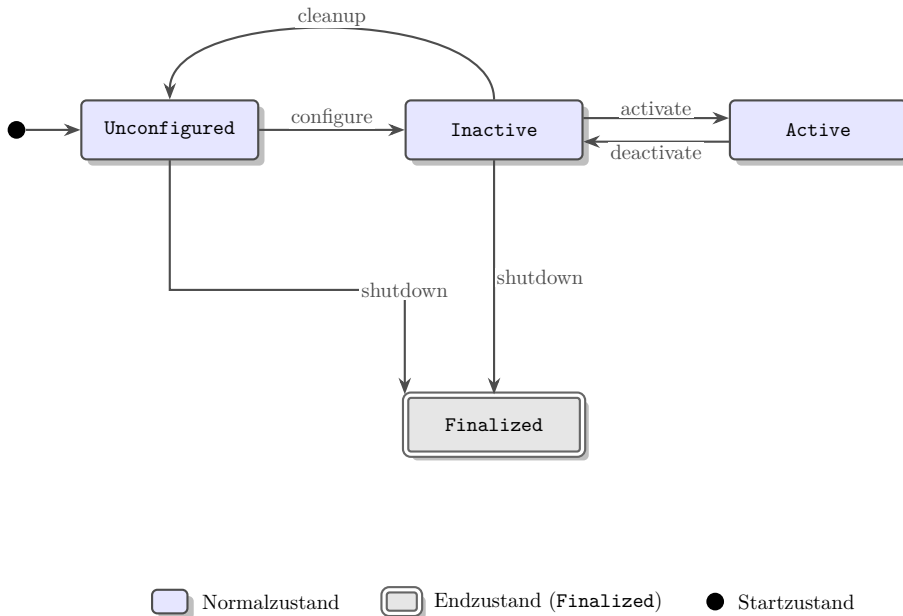


Figure 15.2: Zustandsautomat eines ROS2-Lifecycle-Nodes von **Unconfigured** über **Inactive** und **Active** bis zum Endzustand **Finalized**.

15.9 Architekturreview

Bevor Code geschrieben wird, sollte die ROS2-Architektur kurz geprüft werden. Ein Review kann anhand einfacher Fragen erfolgen:

- Hat jeder Node eine klare Verantwortung?
- Gibt es Topics mit eindeutigem Sender und Empfänger?
- Sind sicherheitsrelevante Datenflüsse erkennbar?
- Sind Fehlerfälle beschrieben?
- Sind Parameter für veränderliche Werte vorgesehen?
- Gibt es eine Verbindung zu Anforderungen und Tests?

Das Werkzeug `rqt_graph` kann dabei helfen, die entworfene Node-Struktur später im laufenden System visuell zu überprüfen.

Für den Indoor-Roboter ist besonders wichtig, dass Navigation, Hinderniserkennung und Bewegungssteuerung nicht unkontrolliert gekoppelt sind. Ein falscher Bewegungsbefehl kann direkt physische Folgen haben. Deshalb sollte der Datenfluss zu `/cmd_vel` klar nachvollziehbar sein.

15.10 Zusammenfassung

Eine gute ROS2-Architektur hat klare Verantwortlichkeiten, verständliche Kommunikation und definierte Fehlerreaktionen. Sie sollte aus dem SysML-Modell ableitbar sein und später durch Tests überprüft werden.

15.11 Kontrollfragen

1. Was beschreibt eine ROS2-Architektur?
2. Warum ist Node-Granularität wichtig?
3. Welche Nodes benötigt der Indoor-Roboter mindestens?
4. Welche Topics sind für Navigation wichtig?
5. Warum müssen Fehlerfälle in der Architektur betrachtet werden?
6. Was sind Parameter?
7. Was sind Lifecycle Nodes?

15.12 Übungen

Übung

Übung 1: Architektur skizzieren

Erstellen Sie eine ROS2-Architekturskizze mit mindestens sechs Nodes und acht Topics.

Übung

Übung 2: Fehlerfall analysieren

Beschreiben Sie, welche Nodes betroffen sind, wenn der Lidar ausfällt. Welche Statusmeldung sollte erzeugt werden?

15.13 Literaturhinweise

Die ROS2-Dokumentation [?] beschreibt Kommunikationsmechanismen, Lifecycle Nodes, Launch-Dateien und Tools. Für Architekturentscheidungen ist zusätzlich das SysML-Modell aus den vorherigen Kapiteln hilfreich.

16 Von SysML v2 zu ROS2

Lernziele

Nach diesem Kapitel können Sie SysML-v2-Elemente auf ROS2-Elemente abbilden. Sie verstehen, warum Mapping für Traceability wichtig ist und wie Requirements, Part Definitions, Parts, Ports, Connections, Actions und Tests mit Nodes, Topics, Services, Actions und Testcode verbunden werden können.

16.1 Einleitung

Das SysML-v2-Modell beschreibt die Architektur. ROS2 ist die Umsetzung. Ohne Verbindung zwischen beiden entsteht eine Lücke. Das Mapping schließt diese Lücke. Es zeigt, wie Modellentscheidungen in Softwareelemente übersetzt werden.

16.2 Grundprinzip

Eine SysML-v2-Part-Definition kann einem ROS2-Node entsprechen, wenn sie eine klare softwarebezogene Verantwortung beschreibt. Ein Port oder eine Connection kann einem Topic, Service oder einer Action entsprechen. Eine Action im Modell kann durch einen Node oder eine Gruppe von Nodes realisiert werden. Ein Testfall im Modell kann einem automatisierten Test entsprechen.

Dieses Mapping ist eine Architekturentscheidung und keine feste Übersetzungsregel. In diesem Kapitel beginnen wir bewusst mit einem einfachen Orientierungsmapping, weil es den Einstieg erleichtert. In realen Systemen kann ein SysML-Element mehrere ROS2-Nodes beeinflussen, und ein ROS2-Node kann mehrere modellierte Verantwortlichkeiten bündeln.

Hinweis: Der Begriff **Action** wird hier in zwei unterschiedlichen Bedeutungen verwendet – als SysML-v2-Verhaltenselement (Bestandteil einer Aktivität im Modell) und als ROS2-Action (ein asynchrones Kommunikationsmuster mit Feedback, siehe Kapitel 14). Im Mapping wird eine SysML-Action häufig durch eine ROS2-Action realisiert, die beiden Begriffe bezeichnen aber unterschiedliche Konzepte auf unterschiedlichen Ebenen.

Nicht jedes Modellelement wird eins zu eins umgesetzt. Ein physisches Element wie **Camera** kann durch Hardware beschrieben werden, während die Ansteuerung über einen **camera_node** erfolgt. Ein größeres Systemelement wie **NavigationSystem** kann aus mehreren Nodes bestehen (zum Beispiel **navigation_node** für die Bewegungsplanung und **slam_node** für die Kartierung, vgl. Kapitel 15).

16.3 Beispieltabelle

SysML-v2-Element		ROS2-Element	Bemerkung
Part	Definition	object_detector_node	Führt ML-Inferenz aus
ObjectDetector			
Part	Definition	camera_node	Liefert Bilddaten
Camera			
Part	Definition	navigation_node	Plant und überwacht
NavigationSystem			Bewegung
Part	Definition	battery_monitor_node	Überwacht Akkustatus
BatterySystem			
Part	Definition	motion_control_node	Setzt
MotionControl			Geschwindigkeitsbefehle um
Part	Definition	user_interface_node	Nimmt Aufträge entgegen
UserInterface			

Abbildung ?? stellt dieses einfache Orientierungsmapping grafisch dar.

Vereinfachte Zuordnung als Einstieg: Die tatsächliche Softwarearchitektur kann später zusammenfassen, aufteilen oder ergänzen.

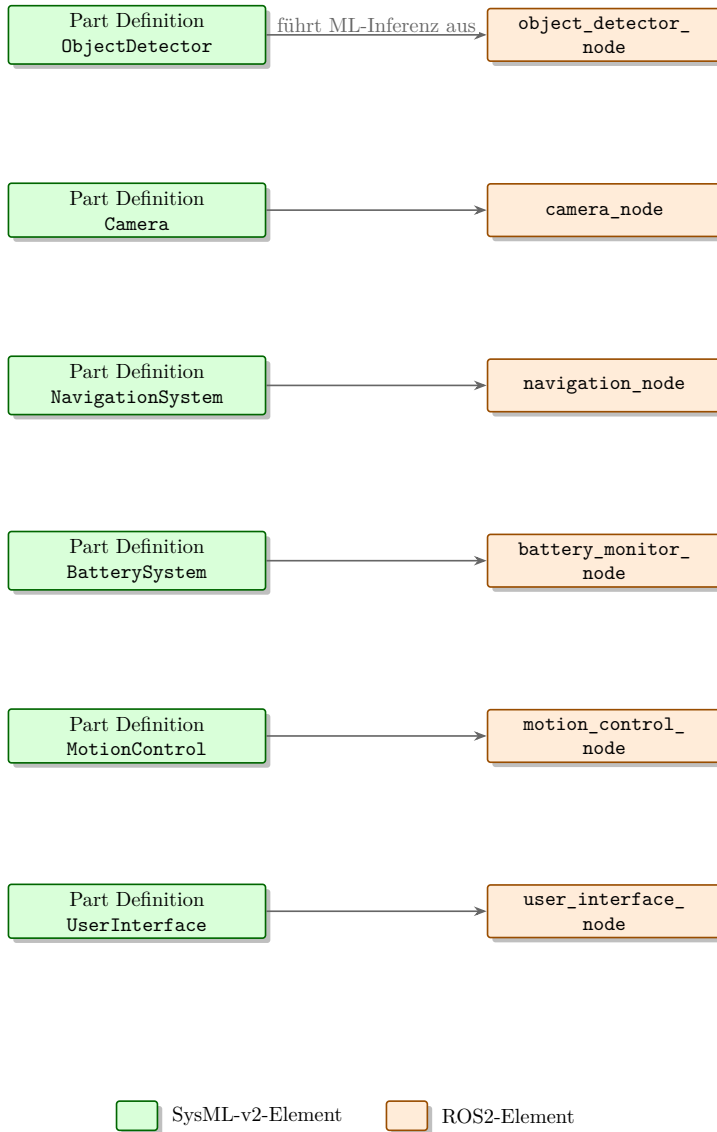


Figure 16.1: Grafische Darstellung eines einfachen Orientierungsmappings zwischen SysML-v2-Part-Definitions und zugehörigen ROS2-Nodes. In realen Systemen muss dieses Mapping projektspezifisch geprüft und verfeinert werden.

16.4 Textuelles Mapping-Beispiel

In SysML v2 kann ein kleiner Ausschnitt des Modells direkt als Text formuliert werden:

```

1 package AutonomousIndoorRobot {
2   part def BatterySystem;
3   part def NavigationSystem;
4
5   part def Robot {
6     part batterySystem: BatterySystem;
7     part navigationSystem: NavigationSystem;
8   }
9
10  requirement REQ_ReturnToDock {
11    doc /* Der Roboter muss bei niedrigem Akkustand
12        selbstständig zur Ladestation zurückkehren. */
13  }
14 }

```

Das ROS2-Mapping ergänzt dazu die technische Entscheidung: `BatterySystem` wird durch `battery_monitor_node` realisiert, der den Akkustatus über `/battery_state` veröffentlicht. `NavigationSystem` reagiert darauf und startet bei Bedarf eine Navigation zur Ladestation.

16.5 Mapping von Schnittstellen

Ports, Connections und modellierte Datenflüsse werden auf Topics, Services oder Actions abgebildet:

- Bilddaten: `/camera/image_raw`
- Laserscan: `/scan`
- Bewegungsbefehl: `/cmd_vel`
- Akkustatus: `/battery_state`
- Zielnavigation: Action `NavigateToPose`

Für ein Einsteigermodell reicht zunächst die fachliche Richtung des Datenflusses. Später sollten Nachrichtentyp, Frequenz, Einheiten, QoS-Einstellungen (z. B. ob Nachrichten zuverlässig oder nur bestmöglich zugestellt werden) und Fehlerfälle ergänzt werden.

16.6 Traceability

Das Mapping ist ein Teil der Traceability (deutsch: Nachverfolgbarkeit von Anforderungen bis zur Implementierung). Es zeigt, welche Implementierung eine Architekturentscheidung realisiert.

Praxisbeispiel

REQ-004 ?Personen erkennen? wird durch die Part Definition `ObjectDetector` erfüllt. Diese Modellentscheidung wird durch den ROS2-Node `object_detector_node` umgesetzt. Die Ergebnisse werden über `/detected_objects` veröffentlicht und durch einen Testfall zur Personenerkennung geprüft.

16.7 Von Anforderungen bis Code

Eine vollständige Kette kann so aussehen:

```
REQ-006 Zur Ladestation zurückkehren
-> Action: Akkustand überwachen
-> Part Definition: BatterySystem
-> Node: battery_monitor_node
-> Topic: /battery_state
-> Node: navigation_node
-> Action: NavigateToDock
-> Test: TC-006 Rückkehr bei niedrigem Akkustand
```

Abbildung ?? zeigt diese Traceability-Kette grafisch, von der Anforderung über SysML- und ROS2-Elemente bis zum Testfall.

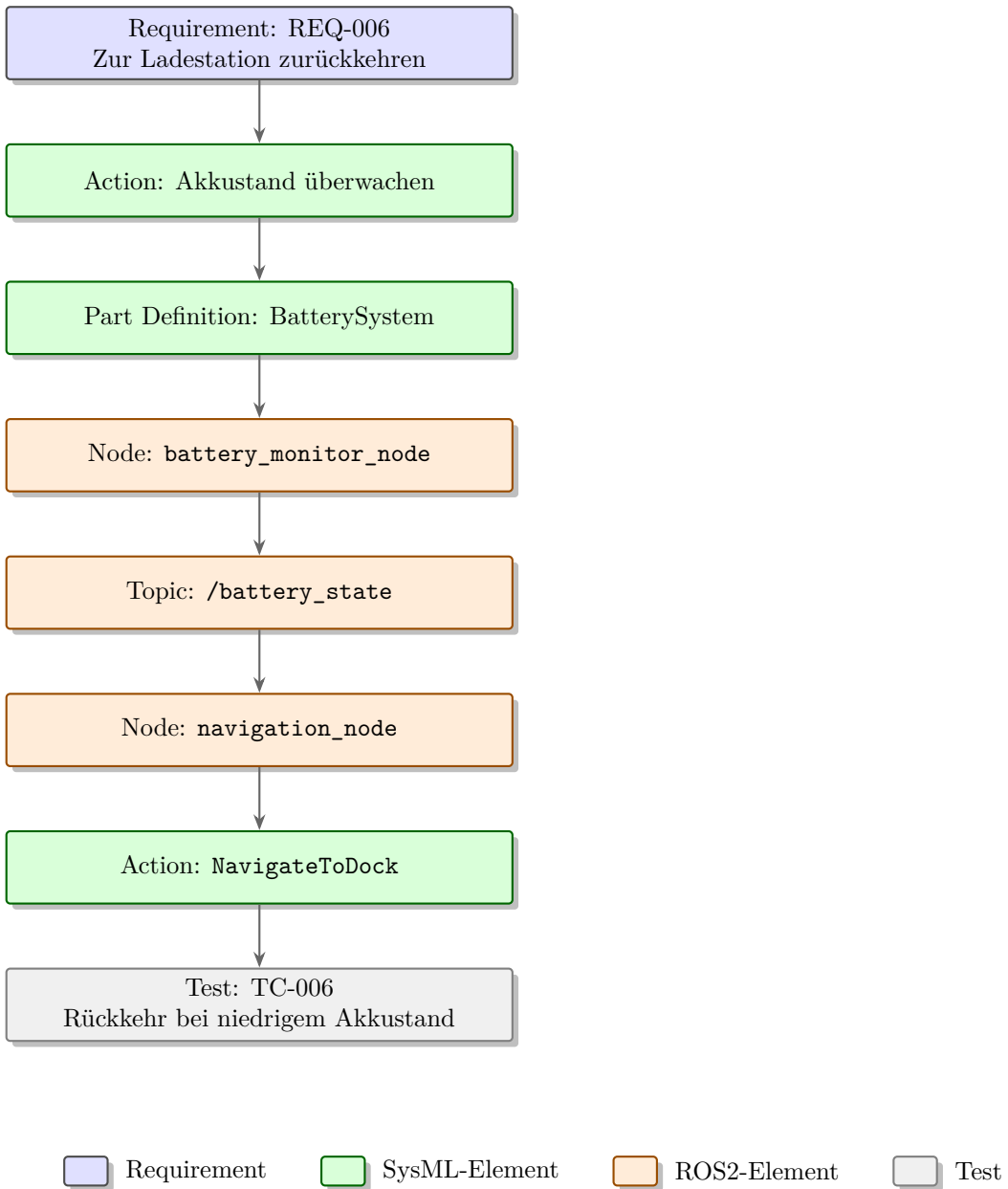


Figure 16.2: Traceability-Kette vom Requirement REQ-006 über SysML- und ROS2-Elemente bis zum Testfall TC-006.

16.8 Typische Fehler

16.8.1 Eins-zu-eins-Zwang

Nicht jede Part Definition muss genau ein Node werden. Mapping ist eine bewusste Architekturentscheidung.

16.8.2 Mapping nicht pflegen

Wenn sich die ROS2-Architektur ändert, muss das Mapping aktualisiert werden. Sonst verliert die Traceability ihren Wert.

16.8.3 Schnittstellen nur grob abbilden

Ein Mapping ohne Topic-Namen, Nachrichtentypen oder Verantwortlichkeiten ist zu ungenau.

16.9 Mapping als lebendes Artefakt

Das Mapping ist kein einmaliges Abschlussdokument. Es wächst mit dem Projekt. Am Anfang reicht eine grobe Tabelle: Part Definition zu Node, Connection zu Topic. Später kommen Nachrichtentypen, Parameter, Launch-Dateien und Tests hinzu.

Eine gute Mapping-Tabelle kann zusätzliche Spalten enthalten:

- SysML-v2-Element
- Action oder Verantwortung
- ROS2-Node
- Topic, Service oder Action
- Nachrichtentyp
- zugehörige Anforderung
- Testfall
- Status der Umsetzung

Damit wird das Mapping zu einem Arbeitsmittel. Es hilft bei Reviews, bei der Codeplanung und beim Erkennen von Lücken. Wenn ein wichtiges Modellelement keinen Node und keinen Test besitzt, ist das ein Signal für weitere Klärung.

16.10 Zusammenfassung

Das SysML-v2-ROS2-Mapping verbindet Modell und Umsetzung. Es hilft, Anforderungen bis zur Software nachzuverfolgen und Änderungen gezielt zu analysieren. Für dieses Lehrbuch ist es die zentrale Brücke zwischen MBSE und praktischer Robotiksoftware.

16.11 Kontrollfragen

1. Warum ist Mapping zwischen SysML v2 und ROS2 wichtig?
2. Welche SysML-v2-Elemente können auf ROS2-Nodes abgebildet werden?
3. Welche SysML-v2-Elemente passen zu Topics?
4. Warum muss Mapping gepflegt werden?
5. Warum ist ein Eins-zu-eins-Mapping nicht immer sinnvoll?
6. Erklären Sie eine Traceability-Kette für Batteriemanagement.

16.12 Übungen

Übung

Übung 1: Mapping-Tabelle

Erstellen Sie eine Mapping-Tabelle für die wichtigsten Part Definitions Ihres Robotermodells.

Übung

Übung 2: Vollständige Kette

Erstellen Sie eine Kette von einer Anforderung über Part Definition, Node, Topic und Testfall für die Hinderniserkennung.

16.13 Literaturhinweise

Das Mapping verbindet SysML-v2-Modellierung [?, ?] mit ROS2-Kommunikation [?]. In realen Projekten wird diese Verbindung oft projektspezifisch definiert.

17 ROS2-Implementierung mit Python

Lernziele

Nach diesem Kapitel verstehen Sie den Aufbau eines einfachen ROS2-Python-Nodes. Sie können Publisher, Subscriber, Timer und grundlegende Package-Strukturen einordnen und wissen, wie ein einfacher Node aus dem Modell abgeleitet werden kann.

17.1 Einleitung

Nachdem die Architektur modelliert wurde, folgt die Umsetzung. In diesem Kapitel betrachten wir Python mit `rclpy`. Python eignet sich gut für den Einstieg, weil der Code vergleichsweise leicht lesbar ist.

17.2 Package erstellen

Ein Python-Package wird typischerweise mit folgendem Befehl erstellt:

```
1 ros2 pkg create robot_basic_nodes --build-type ament_python --dependencies  
  rclpy std_msgs sensor_msgs geometry_msgs
```

Der Parameter `-build-type ament_python` legt den Build-Typ fest: `ament_python` eignet sich für reine Python-Packages, während `ament_cmake` für C++- oder gemischte Packages verwendet wird.

Ein Package bündelt Code, Konfiguration und Startinformationen. Für größere Systeme werden mehrere Packages sinnvoll. Abbildung ?? zeigt die resultierende Workspace- und Package-Struktur.

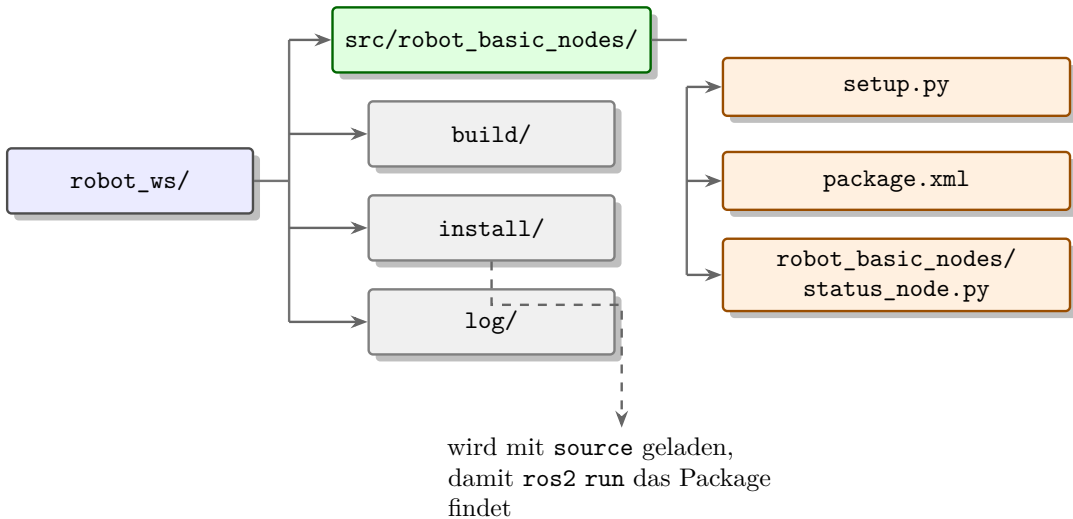


Figure 17.1: Verzeichnisstruktur eines ROS2-Workspace mit dem Package `robot_basic_nodes` sowie den Geschwisterordnern `build/`, `install/` und `log/`.

17.3 Ein einfacher Node

```

1 import rclpy
2 from rclpy.node import Node
3 from std_msgs.msg import String
4
5 class StatusNode(Node):
6     def __init__(self):
7         super().__init__('status_node')
8         self.publisher = self.create_publisher(String, '/robot/status', 10)
9         self.timer = self.create_timer(1.0, self.publish_status)
10
11     def publish_status(self):
12         msg = String()
13         msg.data = 'Robot is running'
14         self.publisher.publish(msg)
15
16 def main():
17     rclpy.init()
18     node = StatusNode()
19     rclpy.spin(node)
20     node.destroy_node()
21     rclpy.shutdown()
22
23 if __name__ == '__main__':
24     main()
  
```

Damit `ros2 run robot_basic_nodes status_node` diesen Code später findet, muss zusätzlich ein passender Eintrag in den `entry_points` der `setup.py` des Packages angelegt werden; darauf gehen wir hier nicht im Detail ein.

17.4 Publisher

Ein Publisher veröffentlicht Nachrichten auf einem Topic. Im Beispiel sendet der Node regelmäßig einen Status auf `/robot/status`. Ein Batteriemonitor würde ähnlich regelmäßig `/battery_state` veröffentlichen.

17.5 Subscriber

Ein Subscriber empfängt Nachrichten von einem Topic. Der ObjectDetector würde zum Beispiel `/camera/image_raw` abonnieren. Eine Callback-Funktion (eine Methode, die automatisch aufgerufen wird, sobald eine neue Nachricht eintrifft) verarbeitet eingehende Nachrichten.

17.6 Timer

Timer lösen Funktionen regelmäßig aus. Sie eignen sich für Statusmeldungen, simulierte Sensordaten oder periodische Prüfungen. Abbildung ?? zeigt den Ablauf im `StatusNode` vom Timer bis zur Nachricht auf dem Topic.

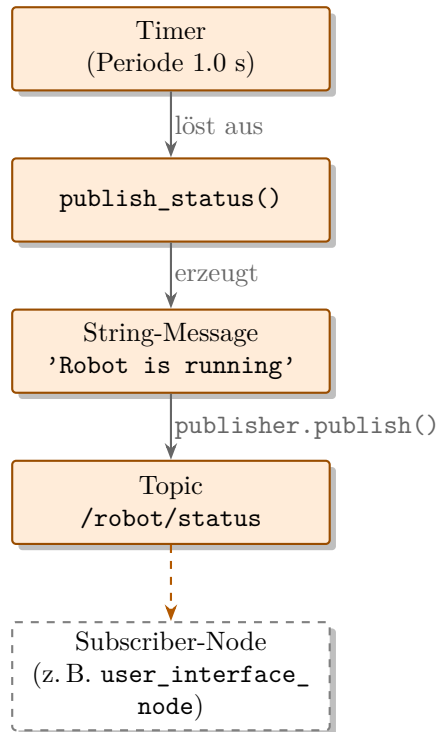


Figure 17.2: Ablauf im `StatusNode`: Der Timer löst periodisch `publish_status()` aus, das eine Nachricht erzeugt und auf dem Topic `/robot/status` veröffentlicht.

17.7 Build und Start

`colcon` ist das Standard-Build-Werkzeug für ROS2-Workspaces; es übersetzt und verlinkt alle Packages im Workspace gemeinsam. Der Workspace gliedert sich dabei typischerweise in die Ordner `src/` (Quellcode), `build/` und `install/` (Build-Ergebnisse) sowie `log/` (Protokolle), wie in Abbildung ?? gezeigt.

```

1 colcon build
2 source install/setup.bash
3 ros2 run robot_basic_nodes status_node

```

17.8 Vom Modell zum Node

Die `part def BatterySystem` kann zur Entscheidung führen, einen `battery_monitor_node` zu implementieren. Die Schnittstelle `?Akkustatus?` wird zum Topic `/battery_state`. Der Testfall prüft später, ob der Node plausible Werte veröffentlicht.

Hinweis

Der Code ist bewusst minimal. In realen Projekten kommen Fehlerbehandlung, Parameter, Logging, Launch-Dateien, Tests und saubere Package-Strukturen hinzu.

17.9 Typische Fehler

17.9.1 Keine klare Verantwortung

Ein Node sollte eine verständliche Aufgabe haben.

17.9.2 Topic-Namen hart verteilen

Topic-Namen sollten konsistent sein und bei Bedarf über Parameter konfigurierbar werden.

17.9.3 Keine Fehlerbehandlung

Auch einfache Nodes sollten ungültige Daten und Ausnahmen nicht ignorieren.

17.10 Vom Minimalbeispiel zum Projektcode

Das gezeigte Beispiel ist absichtlich klein. In einem echten Projekt sollte ein Node mehr können: Parameter lesen, Logging nutzen, ungültige Daten erkennen und sauber herunterfahren. Für den `battery_monitor_node` könnten Parameter definieren, ab welchem Prozentwert eine Warnung veröffentlicht wird.

Ein sinnvoller nächster Schritt ist ein simulierter Batterieknoten. Er veröffentlicht regelmäßig einen Akkustand, der langsam sinkt. Ein zweiter Node kann diesen Wert abonnieren und bei niedrigem Stand eine Warnung ausgeben. Damit entsteht ein kleines, testbares ROS2-System, das direkt zu Anforderungen und Modellbeziehungen passt.

Wichtig ist die Rückbindung an das Modell. Der Code sollte nicht isoliert entstehen. Jede Node sollte im Modell einen Zweck haben, einer Funktion zugeordnet sein und über definierte Topics kommunizieren.

17.11 Zusammenfassung

ROS2-Python-Nodes setzen Teile der modellierten Architektur um. Publisher, Subscriber und Timer bilden die Grundbausteine. Für den Einstieg genügt ein kleiner Node, solange seine Rolle im Modell klar ist.

17.12 Kontrollfragen

1. Was ist ein ROS2-Package?
2. Was macht ein Publisher?
3. Was macht ein Subscriber?
4. Wozu dient ein Timer?
5. Warum sollte ein Node eine klare Verantwortung besitzen?
6. Wie kann ein SysML-Systemelement (`part def`) zu einem ROS2-Node führen?

17.13 Übungen

Übung

Übung 1: Battery Monitor

Implementieren Sie konzeptionell einen `battery_monitor_node`, der regelmäßig einen simulierten Akkustand veröffentlicht.

Übung

Übung 2: Modellbezug

Beschreiben Sie, welche Anforderung, welches Systemelement (`part def`), welches Topic und welcher Testfall zu Ihrem `battery_monitor_node` gehören.

17.14 Literaturhinweise

Für praktische ROS2-Implementierung ist die offizielle Dokumentation [?] zentral. Das Kapitel bleibt bewusst ein Einstieg und bereitet das spätere vollständige ROS2-Projekt vor.

18 Machine Learning in ROS2 integrieren

Lernziele

Nach diesem Kapitel verstehen Sie, wie ein ML-Modell als ROS2-Node in eine Roboterarchitektur eingebunden wird. Sie können Datenfluss, Inferenz, Ergebnismeldungen, Latenz, Modellqualität und typische Integrationsprobleme beschreiben.

18.1 Einleitung

Machine Learning wird in der Robotik häufig für Wahrnehmung eingesetzt: Objekterkennung, Personenerkennung, Segmentierung oder Klassifikation. Im Beispielprojekt verwenden wir ML zur Personenerkennung.

18.2 Rolle von Machine Learning

Klassische Software arbeitet mit expliziten Regeln. ML-Modelle lernen Muster aus Daten. Für Personenerkennung ist das nützlich, weil Personen in Bildern sehr unterschiedlich aussehen können. Gleichzeitig bringt ML neue Herausforderungen: Trainingsdaten, Fehlklassifikationen, Latenz und schwer erklärbares Verhalten.

18.3 Architektur

Camera

- > /camera/image_raw
- > object_detector_node
- > /detected_objects
- > navigation_node oder user_interface_node

Der ML-Node ist Teil der Wahrnehmungsarchitektur. Er sollte klar von Navigation und Sicherheitslogik getrennt werden. Die Navigation bekommt Ergebnisse, aber sie sollte nicht blind jedem Ergebnis vertrauen.

18.4 Inference-Node

Der ML-Node lädt ein trainiertes Modell, verarbeitet eingehende Sensordaten und veröffentlicht Ergebnisse. Dieser Vorgang wird als *Inferenz* bezeichnet: das Anwenden

eines bereits trainierten Modells auf neue Daten, im Unterschied zum vorherigen Training des Modells. In Python kann das mit PyTorch, TensorFlow oder [Open Neural Network Exchange \(ONNX\)](#) Runtime umgesetzt werden.

18.5 Pseudocode

```

1 class ObjectDetectorNode(Node):
2     def __init__(self):
3         super().__init__('object_detector_node')
4         self.model = load_model('model.pt')
5         self.subscription = self.create_subscription(
6             Image,
7             '/camera/image_raw',
8             self.image_callback,
9             10
10        )
11        self.publisher = self.create_publisher(
12            DetectionArray,
13            '/detected_objects',
14            10
15        )
16
17    def image_callback(self, msg):
18        image = convert_ros_image_to_tensor(msg)
19        detections = self.model(image)
20        self.publisher.publish(convert_to_ros_msg(detections))

```

Die Dateiendung `.pt` im Beispiel deutet auf ein PyTorch-Modell hin; bei TensorFlow oder [ONNX](#) Runtime als Framework sähen Modell-Dateiformat und Ladefunktion entsprechend anders aus. Auch der Nachrichtentyp `DetectionArray` ist hier vereinfacht dargestellt: In der Praxis würde man beispielsweise den Standardtyp `vision_msgs/Detection2DArray` verwenden oder einen eigenen Nachrichtentyp definieren.

18.6 Datenkette

Die eigentliche Schwierigkeit liegt oft nicht im Modell, sondern in der Datenkette:

- Bildformat
- Auflösung
- Farbraum
- Kamerakalibrierung
- Latenz
- GPU-Nutzung (Grafikprozessor)

- Synchronisation
- Ergebnisformat

Jeder dieser Punkte kann in der Praxis zu Fehlern führen: Ein falscher Farbraum oder eine falsche Auflösung liefert dem Modell unbrauchbare Daten, und eine zu hohe Latenz kann veraltete Erkennungsergebnisse zur Folge haben.

Abbildung ?? veranschaulicht diese Datenkette vom Kamerabild über die Tensor-Konvertierung und Modell-Inferenz bis zur Veröffentlichung und Weiterverarbeitung der Erkennungsergebnisse.

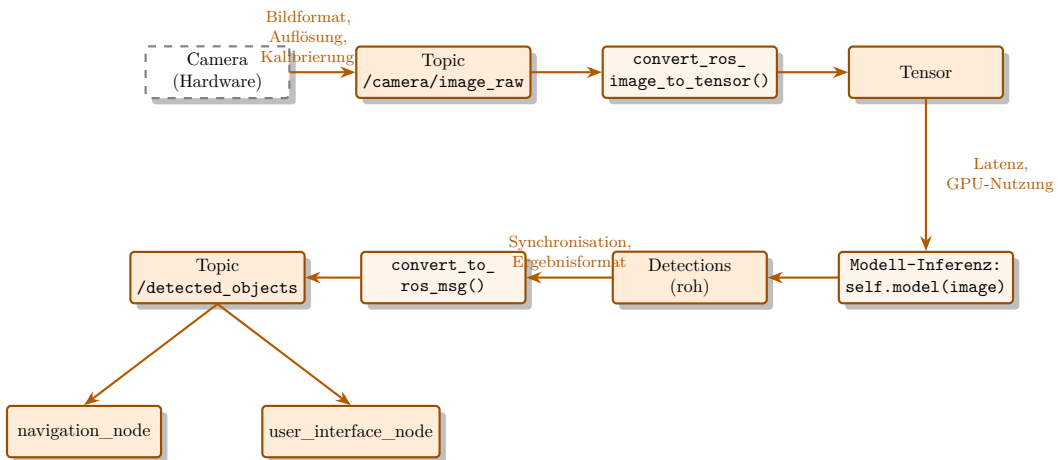


Figure 18.1: Datenkette der ML-Inferenz im `object_detector_node` vom Kamerabild bis zur Veröffentlichung der Erkennungsergebnisse.

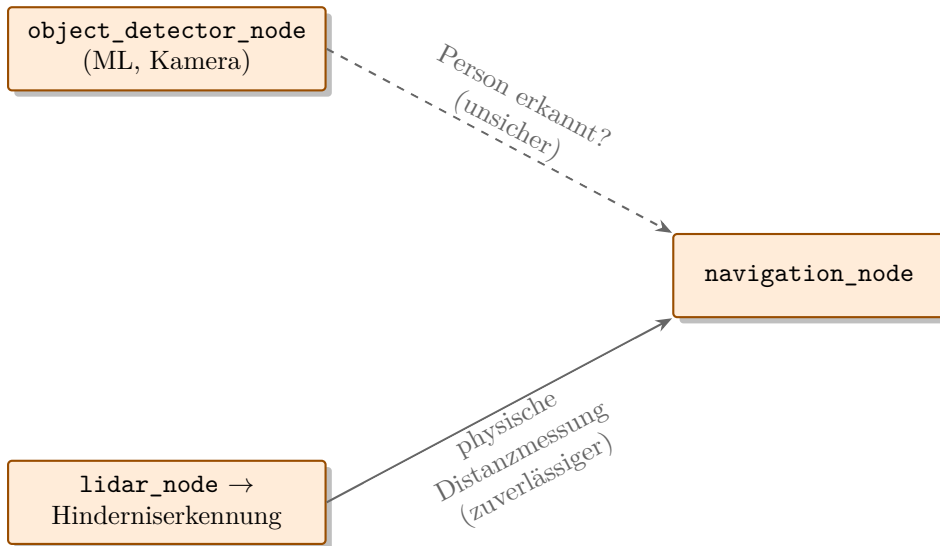
18.7 Qualität und Sicherheit

ML-Ergebnisse sind nicht perfekt. Ein Modell kann Personen übersehen oder Gegenstände fälschlich als Personen erkennen. Deshalb sollten Anforderungen und Tests realistisch formuliert werden. Für sicherheitskritische Funktionen sollte ML nicht ohne weitere Sicherheitsmechanismen verwendet werden.

Praxisbeispiel

Wenn der ML-Node eine Person erkennt, kann die Navigation vorsichtiger fahren. Wenn der ML-Node keine Person erkennt, bedeutet das aber nicht automatisch, dass der Bereich sicher ist. Lidar-basierte Hinderniserkennung bleibt weiterhin wichtig.

Abbildung ?? stellt diese Kontrastierung zwischen der unsicheren ML-Erkennung und der zuverlässigeren Lidar-basierten Distanzmessung dar.



ML ergänzt, ersetzt aber nicht die
Lidar-basierte Sicherheitsprüfung.

Figure 18.2: Der ML-basierte `object_detector_node` liefert eine unsichere Zusatzinformation, während die Lidar-basierte Hinderniserkennung die zuverlässigere Sicherheitsgrundlage bleibt.

18.8 Optimierung

Für reale Roboter können [ONNX Runtime](#), [TensorRT](#) oder [OpenVINO](#) sinnvoll sein – plattform- beziehungsweise hardwareoptimierte Laufzeitumgebungen, die trainierte Modelle schneller und ressourcenschonender ausführen. Ein großes PyTorch-Modell direkt auf einem kleinen Rechner kann zu langsam sein. Optimierung betrifft nicht nur Geschwindigkeit, sondern auch Speicherverbrauch, Temperatur und Energie.

18.9 Modellierung in SysML

Im SysML-Modell sollte sichtbar sein:

- welche Anforderung der ML-Node unterstützt
- welche Daten er benötigt
- welches Ergebnis er liefert
- welche Latenz akzeptabel ist
- welcher Test die Erkennung prüft

- welche Fehlerfälle berücksichtigt werden

18.10 Teststrategie für ML-Funktionen

ML-Funktionen müssen anders getestet werden als klassische Regeln. Ein einzelner Test mit einem Bild reicht nicht aus. Sinnvoll ist ein kleiner Testdatensatz mit unterschiedlichen Lichtverhältnissen, Perspektiven und Situationen. Für den Einstieg kann dieser Datensatz klein sein, aber er sollte bewusst zusammengestellt werden.

Mögliche Testfragen:

- Erkennt das Modell Personen bei normaler Beleuchtung?
- Wie verhält es sich bei Gegenlicht?
- Gibt es viele falsche Erkennungen?
- Wie lange dauert die Inferenz pro Bild?
- Was passiert, wenn keine Kamera-Nachricht ankommt?

Im SysML-Modell kann ein Testfall TC-004 **Personenerkennung** auf mehrere konkrete Testdatensätze verweisen. Damit bleibt sichtbar, dass die Anforderung nicht nur durch Code, sondern durch Daten und Bewertungskriterien geprüft wird.

18.11 Zusammenfassung

ML kann die Wahrnehmung des Roboters verbessern, bringt aber neue Integrations- und Qualitätsfragen. Ein ML-Node sollte sauber in die ROS2-Architektur eingebunden, im SysML-Modell nachvollziehbar beschrieben und durch Tests überprüft werden.

18.12 Kontrollfragen

1. Wofür wird ML im Roboterprojekt eingesetzt?
2. Welche Daten benötigt der ObjectDetector?
3. Welches Topic könnte Erkennungsergebnisse veröffentlichen?
4. Warum ist Latenz wichtig?
5. Warum sollte ML nicht blind als Sicherheitsfunktion betrachtet werden?
6. Welche Optimierungswege gibt es für ML-Inferenz?

18.13 Übungen

Übung

Übung 1: Modellbezug

Beschreiben Sie im SysML-Modell, welche Anforderung durch den ML-Node erfüllt wird und welches Topic die Ergebnisse veröffentlicht.

Übung

Übung 2: Fehlerfälle

Nennen Sie drei mögliche Fehlerfälle bei der Personenerkennung und beschreiben Sie eine sinnvolle Systemreaktion.

18.14 Literaturhinweise

Für ROS2-Integration ist die ROS2-Dokumentation [?] wichtig. Für ML-Modelle sollten zusätzlich Quellen zu PyTorch, YOLO, ONNX und Embedded-AI-Optimierung genutzt werden.

19 Test und Verifikation

Lernziele

Nach diesem Kapitel können Sie Anforderungen mit Testfällen verknüpfen und einfache Verifikationsszenarien beschreiben. Sie verstehen den Unterschied zwischen Verifikation und Validierung und können Testfälle für Navigation, Hinderniserkennung, Personenerkennung, Batteriemanagement und Ladestation formulieren.

19.1 Einleitung

Ein Systemmodell ist nur dann wirklich nützlich, wenn es bis zum Nachweis führt. Anforderungen sollen nicht nur schön formuliert werden. Am Ende muss geprüft werden, ob das System sie erfüllt. Genau darum geht es in diesem Kapitel.

19.2 Verifikation und Validierung

Verifikation fragt: Wurde das System richtig gebaut? Das bedeutet: Erfüllt das System die spezifizierten Anforderungen?

Validierung fragt: Wurde das richtige System gebaut? Das bedeutet: Löst das System tatsächlich das Problem der Stakeholder?

Beide Fragen sind wichtig. Ein Roboter kann alle spezifizierten Anforderungen erfüllen und trotzdem für Nutzer unpraktisch sein. Umgekehrt kann ein Prototyp nützlich wirken, aber wichtige Anforderungen nicht nachweisbar erfüllen.

Das klassische V-Modell veranschaulicht diesen Zusammenhang: Jede Spezifikationsebene auf dem absteigenden Ast hat eine entsprechende Verifikations- oder Validierungsebene auf dem aufsteigenden Ast (siehe Abbildung ??).

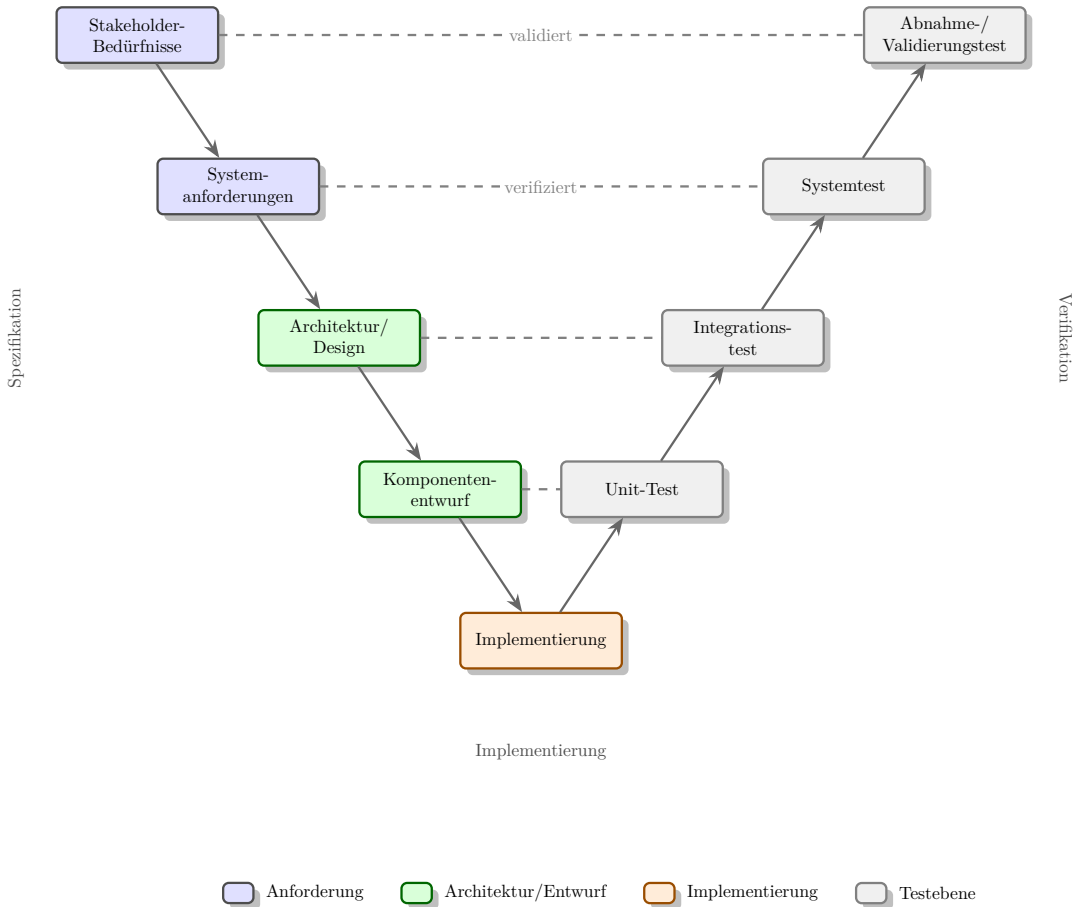


Figure 19.1: Das V-Modell verbindet den absteigenden Spezifikationsast mit dem aufsteigenden Verifikationsast.

19.3 Testfälle

Die folgende Tabelle listet die Testfälle des Roboterprojekts auf. Das Kürzel ?TC? steht dabei für *Test Case* (Testfall), das Kürzel ?REQ? für *Requirement* (Anforderung, siehe Kapitel 6).

ID	Testfall	Verifizierte Anforderung
TC-001	Autonome Navigation zu Zielpunkt	REQ-001
TC-002	Hindernis rechtzeitig erkennen	REQ-002
TC-003	Hindernis sicher behandeln	REQ-003
TC-004	Person im Kamerabild erkennen	REQ-004
TC-005	Niedrigen Akkustand melden	REQ-005

TC-006	Zur Ladestation fahren	REQ-006
TC-007	Betriebszustand melden	REQ-007
TC-008	Sensorausfall sicher behandeln	REQ-008

19.4 Testbeschreibung

Ein Testfall sollte enthalten:

- ID
- Ziel
- verifizierte Anforderung
- Vorbedingungen
- Testaufbau
- Ablauf
- erwartetes Ergebnis
- Bewertungskriterium
- benötigte Messdaten

19.5 Beispiel TC-002

Vorbedingung: Roboter fährt mit definierter Geschwindigkeit.

Ablauf: Hindernis wird in den Fahrweg gestellt.

Erwartung: Roboter erkennt Hindernis und stoppt vor Kollision.

Kriterium: Mindestabstand 20 cm.

19.6 Testarten

19.6.1 Unit-Tests

Unit-Tests prüfen kleine Softwareeinheiten. Ein Batteriemonitor kann zum Beispiel getestet werden, indem simulierte Spannungswerte eingegeben und erwartete Akkustände geprüft werden.

19.6.2 Integrationstests

Integrationstests prüfen das Zusammenspiel mehrerer Komponenten. Beispiel: `battery_monitor_r` meldet niedrigen Akkustand, `navigation_node` plant Rückkehr zur Ladestation.

19.6.3 Systemtests

Systemtests prüfen das Verhalten des gesamten Roboters in definierten Szenarien.

19.6.4 Simulationstests

Simulationstests erlauben viele Tests ohne echte Hardware. Sie ersetzen reale Tests nicht vollständig, sind aber für frühe Entwicklung sehr wertvoll. Kapitel 22 stellt mit Gazebo ein konkretes Simulationswerkzeug für solche Tests vor.

19.7 Tests im Modell

In SysML können Testfälle mit **verify**-Beziehungen an Anforderungen gehängt werden. Dadurch ist erkennbar, welche Anforderungen noch keine Tests haben. Abbildung ?? zeigt dies am Beispiel von TC-006 und REQ-006.

In SysML v2 kann eine solche **verify**-Beziehung textuell etwa so notiert werden:

```

1 package Tests {
2   import Requirements::*;
3
4   test def TC_006 {
5     doc /* Prüft, ob der Roboter bei niedrigem Akkustand
6         selbststaendig zur Ladestation faehrt. */
7     verify requirement REQ_006;
8   }
9 }
```

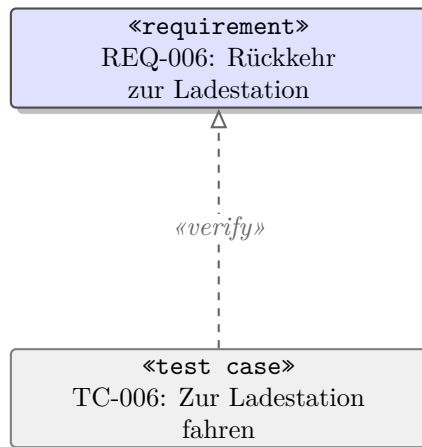


Figure 19.2: Der Testfall TC-006 verifiziert die Anforderung REQ-006 über eine **verify**-Beziehung.

Praxisbeispiel

Wenn REQ-006 eine Rückkehr zur Ladestation fordert, sollte ein Testfall prüfen, ob der Roboter bei unterschrittenem Akkuswellwert tatsächlich ein Rückkehrziel setzt und keinen neuen normalen Transportauftrag startet.

19.8 Typische Fehler

19.8.1 Tests zu spät planen

Wenn Tests erst am Ende entstehen, sind Anforderungen oft unprüfbar.

19.8.2 Nur den Normalfall testen

Fehlerfälle wie Sensorausfall, blockierter Weg und niedriger Akku sind besonders wichtig.

19.8.3 Keine klaren Kriterien

?Roboter stoppt rechtzeitig? muss in ein messbares Kriterium übersetzt werden.

19.9 Testplanung über den Projektverlauf

Tests sollten nicht erst am Ende des Projekts entstehen. Bereits beim Formulieren einer Anforderung sollte gefragt werden: Wie wird diese Anforderung später geprüft? Dadurch erkennt man früh, ob eine Anforderung zu unklar ist.

Für das Roboterprojekt kann die Testplanung schrittweise wachsen:

- Anforderungen erhalten erste Akzeptanzkriterien.
- Wichtige Funktionen erhalten Testideen.
- Schnittstellen erhalten Integrationstests.
- ROS2-Nodes erhalten Unit- oder Komponententests.
- Szenarien wie Zielnavigation werden als Systemtests beschrieben.

Sinnvoll ist außerdem, bestehende Tests nach jeder Änderung erneut auszuführen (Regressionstest), damit neu eingeführte Fehler frühzeitig auffallen – besonders relevant im Zusammenhang mit dem Änderungsmanagement aus Kapitel 20.

Ein gutes Testmodell macht auch Lücken sichtbar. Wenn eine Anforderung keinen Testfall besitzt, ist das nicht automatisch falsch, aber es ist eine offene Frage. Vielleicht wird sie durch Analyse, Review oder Inspektion nachgewiesen. Test, Analyse, Inspektion und Demonstration gelten dabei als die vier klassischen Verifikationsmethoden. Auch das sollte dokumentiert werden.

19.10 Zusammenfassung

Tests und Verifikation schließen den Kreis von Anforderungen zur Umsetzung. Ein gutes Modell zeigt nicht nur, was gebaut werden soll, sondern auch, wie die Erfüllung geprüft wird.

19.11 Kontrollfragen

1. Was ist Verifikation?
2. Was ist Validierung?
3. Welche Informationen enthält ein guter Testfall?
4. Wozu dient die SysML-Beziehung `verify`?
5. Was ist der Unterschied zwischen Unit-Test und Systemtest?
6. Warum sind Fehlerfälle wichtig?

19.12 Übungen

Übung

Übung 1: Testfälle erstellen

Erstellen Sie Testfälle für alle Anforderungen aus Kapitel 6. Markieren Sie Anforderungen ohne Test.

Übung

Übung 2: Testfall detaillieren

Beschreiben Sie TC-006 *Zur Ladestation fahren?* mit Vorbedingungen, Ablauf, Erwartung und Bewertungskriterium.

19.13 Literaturhinweise

Verifikation und Validierung sind zentrale Themen im Systems Engineering. SysML-Literatur [?] zeigt, wie Testfälle mit Anforderungen verbunden werden können.

20 Traceability und Änderungsmanagement

Lernziele

Nach diesem Kapitel können Sie eine durchgängige Nachvollziehbarkeit von Anforderungen bis Tests aufbauen. Sie verstehen Traceability-Ketten, Impact Analysis und den Nutzen systematischer Änderungsbewertung im Roboterprojekt.

20.1 Einleitung

Traceability bedeutet Nachvollziehbarkeit. Sie zeigt, wie Artefakte zusammenhängen. Bei MBSE ist das einer der größten praktischen Vorteile: Anforderungen, Funktionen, Systemteile, Schnittstellen, ROS2-Nodes und Tests bleiben verbunden.

20.2 Was ist Traceability?

Traceability beantwortet Fragen wie:

- Welche Komponente erfüllt diese Anforderung?
- Welcher Test verifiziert diese Anforderung?
- Welche ROS2-Node implementiert dieses Systemelement?
- Welche Schnittstellen sind betroffen, wenn ein Sensor geändert wird?
- Welche Anforderungen haben noch keinen Test?

20.3 Kette im Roboterprojekt

REQ-002 Hindernisse erkennen

-> PerceptionSystem

-> ObstacleDetector

-> obstacle_detector_node

-> /scan

-> TC-002 Hinderniserkennung

Diese Kette zeigt, wie eine Anforderung bis zu Implementierung und Test verfolgt werden kann. In SysML v2 lassen sich solche Verbindungen über die Beziehungen **satisfy**, **derive** und **trace** konkret im Modell verankern, zum Beispiel:

```
1 satisfy requirement REQ_002 by PerceptionSystem;
2 derive requirement REQ_002 from REQ_SAFE_002;
3 trace PerceptionSystem to obstacle_detector_node;
```

Abbildung ?? stellt dieselbe Kette grafisch dar, einschließlich der jeweiligen Beziehungsart zwischen den Elementen.

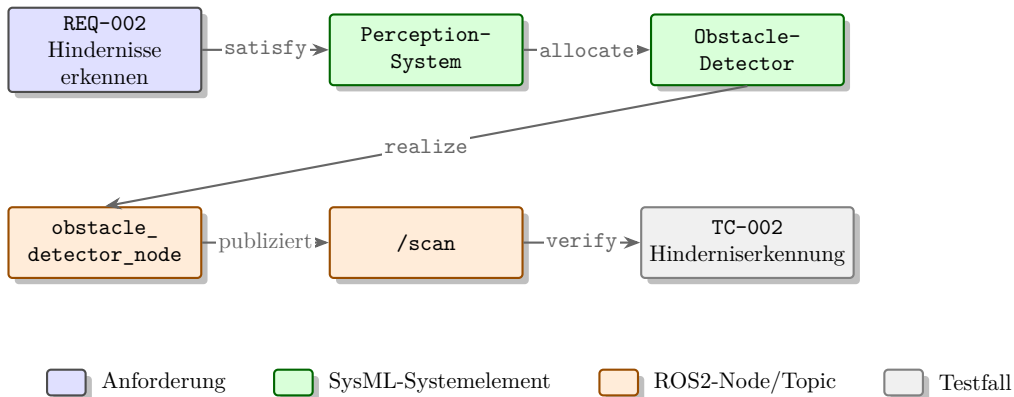


Figure 20.1: Traceability-Kette von REQ-002 über SysML-Systemelemente und ROS2-Node/Topic bis zum Testfall TC-002.

20.4 Nutzen

Wenn sich REQ-002 ändert, kann analysiert werden, welche Komponenten betroffen sind. Vielleicht muss der Lidar ausgetauscht werden. Vielleicht muss der Test angepasst werden. Vielleicht reicht eine Softwareänderung.

20.5 Impact Analysis

Impact Analysis bedeutet Auswirkungsanalyse. Sie beantwortet die Frage: Was ändert sich, wenn eine Anforderung, Komponente oder Schnittstelle geändert wird?

Methodisch lässt sich diese Frage beantworten, indem man von der geänderten Anforderung ausgehend die Traceability-Matrix rückwärts verfolgt: Welche Systemelemente, Schnittstellen, ROS2-Nodes und Tests sind über eine Beziehung mit ihr verbunden?

Beispiel: Die maximale Geschwindigkeit wird erhöht. Betroffen sein können:

- Sicherheitsanforderungen
- Bremswegmodell
- Hinderniserkennung

- Motorsteuerung
- Akkulaufzeit
- Tests zur Stoppdistanz
- ROS2-Parameter für Geschwindigkeit

20.6 Traceability-Matrix

Eine einfache Matrix kann Spalten enthalten:

- Anforderung
- Funktion
- SysML-Systemelement (`part def`)
- ROS2-Node
- Schnittstelle
- Testfall

Diese Matrix ist kein Ersatz für das Modell, aber eine nützliche Sicht. Abbildung ?? zeigt einen Auszug für zwei Anforderungen des Indoor-Roboters. Moderne SysML-v2-Werkzeuge können eine solche Matrix häufig automatisch aus den Modellbeziehungen erzeugen, was den manuellen Pflegeaufwand deutlich reduziert.

Anforderung	Funktion	SysML-Systemelement	ROS2-Node	Schnittstelle	Testfall
REQ-002	Hinderniserkennung	ObstacleDetector	obstacle_detector_node	/scan	TC-002
REQ-006	Rückkehr zur Ladestation	ChargingController	navigation_node	/battery_status	TC-006

Figure 20.2: Auszug aus einer Traceability-Matrix: Jede Zeile verbindet eine Anforderung über Funktion, Systemelement und ROS2-Node bis zum Testfall.

20.7 Änderungsmanagement

Änderungen sollten bewusst bewertet werden. Ein einfacher Ablauf:

1. Änderungswunsch beschreiben.
2. Betroffene Anforderungen identifizieren.
3. Betroffene Architekturteile und Schnittstellen ermitteln.
4. Betroffene Tests prüfen.
5. Aufwand und Risiko bewerten.
6. Entscheidung dokumentieren.

Am Beispiel der Geschwindigkeitserhöhung aus dem vorigen Abschnitt lässt sich dieser Ablauf konkret durchspielen: Der Änderungswunsch (höhere Maximalgeschwindigkeit) macht die betroffenen Sicherheitsanforderungen und das Bremswegmodell sichtbar, die Tests zur Stoppdistanz müssen angepasst werden, und erst nach Bewertung von Aufwand und Risiko wird die Entscheidung dokumentiert. Bei größeren Projekten übernimmt häufig ein eigenes Genehmigungsgremium, das Change Control Board (CCB), diese Entscheidung; für das Einsteigerprojekt reicht in der Regel eine einfache, dokumentierte Entscheidung.

Abbildung ?? zeigt diesen Ablauf inklusive der Entscheidung über Genehmigung oder Ablehnung.

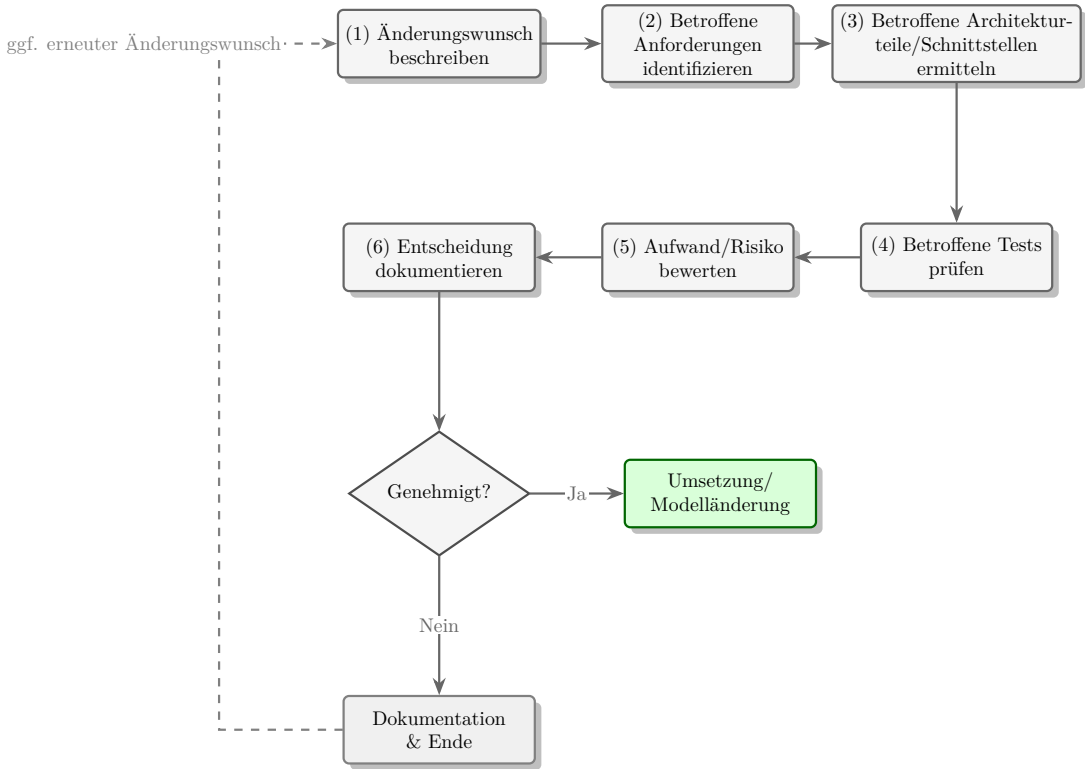


Figure 20.3: Ablauf des Änderungsmanagements von der Beschreibung des Änderungswunschs bis zur dokumentierten Entscheidung.

20.8 Typische Fehler

Traceability wird oft zu spät aufgebaut. Dann ist sie mühsam. Besser ist es, Beziehungen während der Modellierung schrittweise zu pflegen. Ein weiterer Fehler ist zu viel Traceability ohne Nutzen. Verknüpfen Sie vor allem die Elemente, die für Entscheidungen, Änderungen und Tests wichtig sind.

20.9 Pragmatischer Umfang

Traceability kann sehr schnell groß werden. Für ein Einsteigerprojekt sollte sie bewusst begrenzt werden. Wichtig sind zunächst die Hauptanforderungen, Hauptblöcke, zentralen Schnittstellen, ROS2-Nodes und Testfälle.

Eine gute Startregel lautet: Jede Hauptanforderung sollte mindestens eine Beziehung zu einem erfüllenden Modellelement und einem Nachweis besitzen. Jede sicherheitsrelevante Anforderung sollte zusätzlich mit betroffenen Schnittstellen und Fehlerfällen verbunden sein.

Für den Indoor-Roboter sind besonders traceability-würdig:

- Hinderniserkennung
- Personenerkennung
- Rückkehr zur Ladestation
- sicherer Stopp
- Akkustandsüberwachung
- Status- und Fehlermeldungen

Diese Ketten liefern viel Nutzen, ohne dass das Modell sofort überladen wird.

20.10 Zusammenfassung

Traceability verbindet Anforderungen, Architektur, Umsetzung und Tests. Sie macht Änderungen beherrschbar und hilft, Lücken im Modell zu finden. Für den Indoor-Roboter ist sie besonders wertvoll, weil Sensorik, Software, Energie und Sicherheit eng zusammenhängen.

20.11 Kontrollfragen

1. Was bedeutet Traceability?
2. Warum ist Traceability in MBSE wichtig?
3. Was ist Impact Analysis?
4. Welche Elemente enthält eine Traceability-Kette?
5. Warum sollte Traceability früh aufgebaut werden?
6. Welche Elemente wären von einer höheren Robotergeschwindigkeit betroffen?

20.12 Übungen

Übung

Übung 1: Traceability-Kette

Wählen Sie eine Anforderung aus und erstellen Sie eine vollständige Traceability-Kette bis zu einem Testfall.

Übung

Übung 2: Impact Analysis

Analysieren Sie die Änderung: Die Kamera wird durch eine Tiefenkamera ersetzt. Welche Anforderungen, Systemteile, Schnittstellen, ROS2-Nodes und Tests sind betroffen?

20.13 Literaturhinweise

Traceability ist ein Kernthema von MBSE und Requirements Engineering. SysML v2 bietet dafür Beziehungen wie `satisfy`, `verify`, `derive` und `trace` [?].

21 Abschlussprojekt

Lernziele

Nach diesem Kapitel können Sie ein kleines, aber vollständiges MBSE-Projekt für einen ROS2-Roboter strukturieren. Sie können Anforderungen, Struktur, Verhalten, Schnittstellen, ROS2-Mapping, Tests und Traceability zu einem zusammenhängenden Modell verbinden.

21.1 Einleitung

Das Abschlussprojekt bündelt die bisherigen Kapitel. Sie erstellen ein vollständiges Modell des autonomen Indoor-Roboters. Es muss nicht industriell perfekt sein. Es soll aber zeigen, dass Anforderungen, Architektur und Tests zusammenhängen.

21.2 Aufgabe

Erstellen Sie ein vollständiges Modell des autonomen Indoor-Roboters. Das Modell soll Anforderungen, Struktur, Verhalten, Schnittstellen, ROS2-Mapping und Tests enthalten.

21.3 Abzugebendes Modell

Das Modell sollte folgende Inhalte enthalten:

- Kontextbeschreibung
- Stakeholderliste
- Mindestens zehn Anforderungen
- Requirement Diagram
- BDD mit Hauptsystemen (siehe Kapitel 8)
- IBD für Perception oder Navigation (siehe Kapitel 9)
- Aktivitätsdiagramm für Zielnavigation
- Zustandsdiagramm für Betriebsmodi

- Schnittstellentabelle
- ROS2-Mapping-Tabelle
- Mindestens fünf Testfälle
- Traceability-Matrix

Abbildung ?? fasst diese Inhalte als Checkliste zusammen, mit der Sie Ihr Modell vor der Abgabe prüfen können.

- ☐ Kontextbeschreibung
- ☐ Stakeholderliste
- ☐ Mindestens zehn Anforderungen
- ☐ Requirement Diagram
- ☐ BDD mit Hauptsystemen
- ☐ IBD für ein Teilsystem (z. B. Perception oder Navigation)
- ☐ Aktivitätsdiagramm für Zielnavigation
- ☐ Zustandsdiagramm für Betriebsmodi
- ☐ Schnittstellentabelle
- ☐ ROS2-Mapping-Tabelle
- ☐ Mindestens fünf Testfälle
- ☐ Traceability-Matrix

Figure 21.1: Checkliste mit den zwölf Inhalten, die das Abschlussmodell enthalten sollte.

21.4 Empfohlene Vorgehensweise

1. Systemziel und Systemgrenze formulieren.
2. Stakeholder und Bedürfnisse sammeln.
3. Anforderungen formulieren und nummerieren.
4. Hauptstruktur im BDD modellieren.

5. Ein wichtiges Teilsystem im IBD darstellen.
6. Verhalten mit Aktivitäts- und Zustandsdiagramm beschreiben.
7. Schnittstellen tabellarisch dokumentieren.
8. SysML-Elemente auf ROS2-Elemente abbilden.
9. Testfälle erstellen und mit Anforderungen verknüpfen.
10. Traceability-Matrix prüfen.

Abbildung ?? fasst diese zehn Schritte als Fahrplan in vier Phasen zusammen.

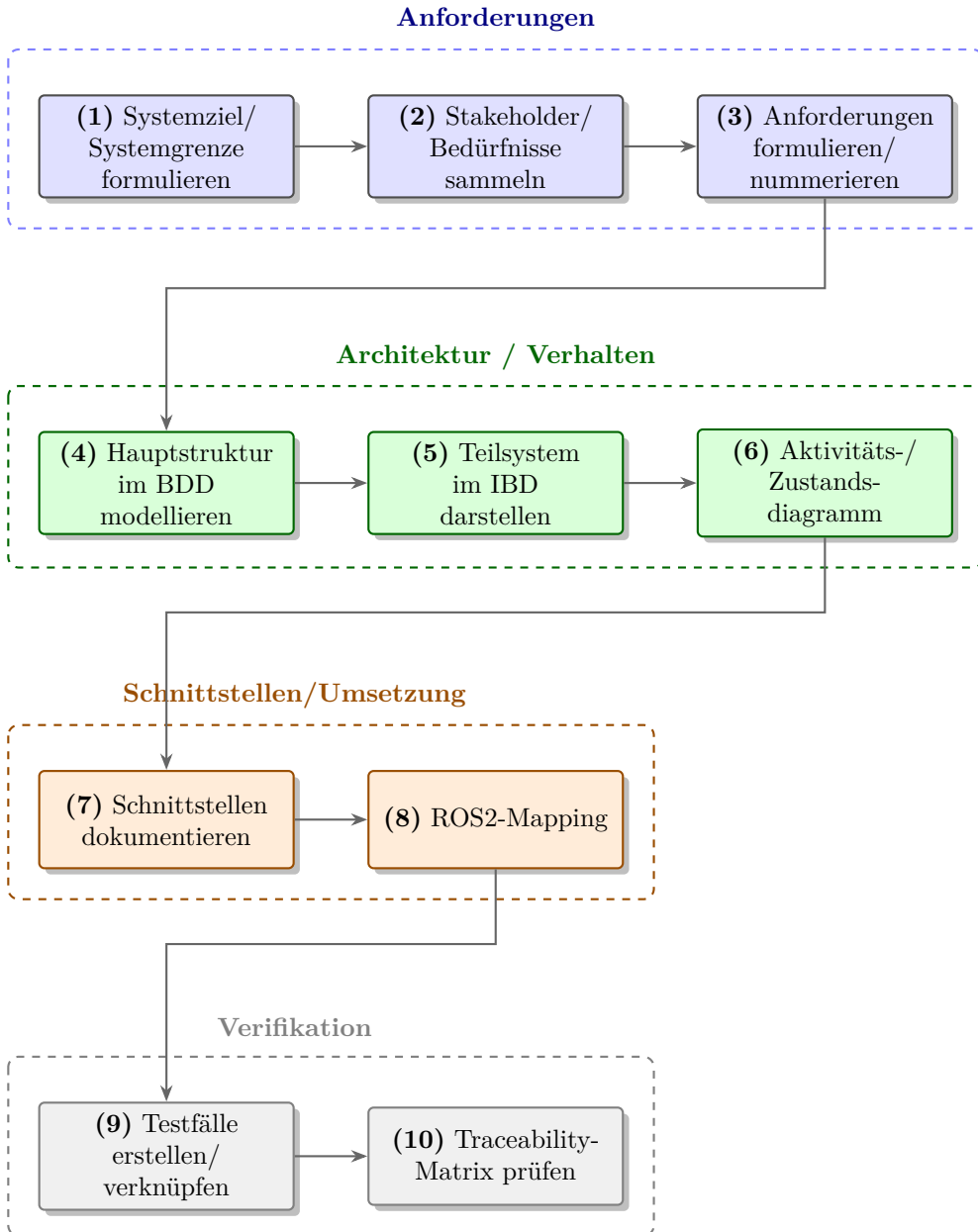


Figure 21.2: Zehn Schritte des Abschlussprojekts, gruppiert in vier Phasen von den Anforderungen bis zur Verifikation.

21.5 Bewertungskriterien

Ein gutes Modell ist verständlich, konsistent und nachvollziehbar. Es muss nicht maximal detailliert sein. Es muss zeigen, dass die Architektur aus Anforderungen abgeleitet wurde.

Wichtige Kriterien:

- klare Systemgrenze
- verständliche Anforderungen
- konsistente Namen
- sinnvolle Struktur
- beschriebene Schnittstellen
- nachvollziehbares ROS2-Mapping
- Testfälle mit `verify`-Beziehungen
- keine isolierten, unbegründeten Diagramme

Diese Kriterien eignen sich auch als kurze Checkliste, die Sie vor der Abgabe Punkt für Punkt durchgehen können.

21.6 Erweiterung

Wer weitergehen möchte, kann eine einfache ROS2-Simulation ergänzen. Möglich sind ein simulierter Batterieknoten, ein Statusknoten und ein Dummy-Object-Detector, orientiert an den Node-Beispielen aus Kapitel 17. Wichtig ist, dass die Umsetzung auf das Modell zurückgeführt wird. Kapitel 22 zeigt mit Gazebo eine Möglichkeit, eine solche Simulation weiter auszubauen.

21.7 Typische Projektfallen

21.7.1 Zu viel Detail

Ein Abschlussmodell muss nicht jedes Detail enthalten. Fokus ist der durchgängige Zusammenhang.

21.7.2 Keine Traceability

Wenn Anforderungen, Systemteile und Tests nicht verbunden sind, bleibt das Projekt eine Sammlung einzelner Diagramme.

21.7.3 Unklare Abgabe

Dokumentieren Sie, welche Diagramme und Tabellen Teil der Abgabe sind. Eine Startseite im Modell hilft.

21.8 Empfohlene Abschlussdokumentation

Neben dem Modell sollte eine kurze Abschlussdokumentation entstehen. Sie muss nicht lang sein, sollte aber den Leser durch das Modell führen. Eine sinnvolle Struktur ist:

- Ziel und Umfang des Systems
- wichtigste Stakeholder
- wichtigste Anforderungen
- Architekturüberblick
- zentrale Schnittstellen
- ROS2-Mapping
- Test- und Verifikationskonzept
- offene Annahmen und Grenzen

Diese Dokumentation ist keine Wiederholung aller Diagramme. Sie erklärt, wie die Diagramme zusammenhängen. Damit wird aus dem Modell ein nachvollziehbares Projektergebnis.

21.9 Zusammenfassung

Das Abschlussprojekt zeigt, ob die Grundidee des Buches verstanden wurde: vom Problem über Anforderungen und SysML-Modellierung bis zur ROS2-nahen Architektur und Verifikation. Der autonome Indoor-Roboter dient dabei als durchgängiges, praxisnahes Beispiel.

21.10 Kontrollfragen

1. Welche Inhalte muss das Abschlussmodell enthalten?
2. Warum ist eine Traceability-Matrix wichtig?
3. Was unterscheidet ein gutes Modell von einer Diagrammsammlung?
4. Welche Bewertungskriterien sind besonders wichtig?
5. Wie kann eine ROS2-Erweiterung sinnvoll angebunden werden?

21.11 Übungen

Übung

Übung 1: Traceability-Matrix

Erstellen Sie eine Traceability-Matrix mit den Spalten Anforderung, Systemelement (`part def`), ROS2-Node und Testfall.

Übung

Übung 2: Modellreview

Prüfen Sie Ihr Modell: Gibt es Anforderungen ohne Test? Systemteile ohne Zweck? Schnittstellen ohne Datentyp? Notieren Sie mindestens fünf Verbesserungen.

21.12 Literaturhinweise

Für das Abschlussprojekt sollten die vorherigen Kapitel sowie die SysML- und ROS2-Quellen [?, ?, ?] erneut herangezogen werden.

22 Ausblick

Lernziele

Nach diesem Kapitel kennen Sie sinnvolle nächste Schritte zur Vertiefung. Sie können einordnen, welche Themen nach diesem Einsteigerbuch folgen: vertiefte SysML-Modellierung, SysML v2, ROS2-Praxis, Simulation, Safety Engineering, Machine Learning und berufliche Anwendung.

22.1 Einleitung

Dieses Buch bietet einen Einstieg. Es zeigt, wie SysML v2, MBSE und ROS2 am Beispiel eines autonomen Indoor-Roboters zusammenspielen. Nach dem Einstieg gibt es viele Wege zur Vertiefung. Dieses Kapitel ordnet die wichtigsten nächsten Schritte ein. Abbildung ?? gibt einen Überblick über die sechs Themenfelder, die in den folgenden Abschnitten behandelt werden.

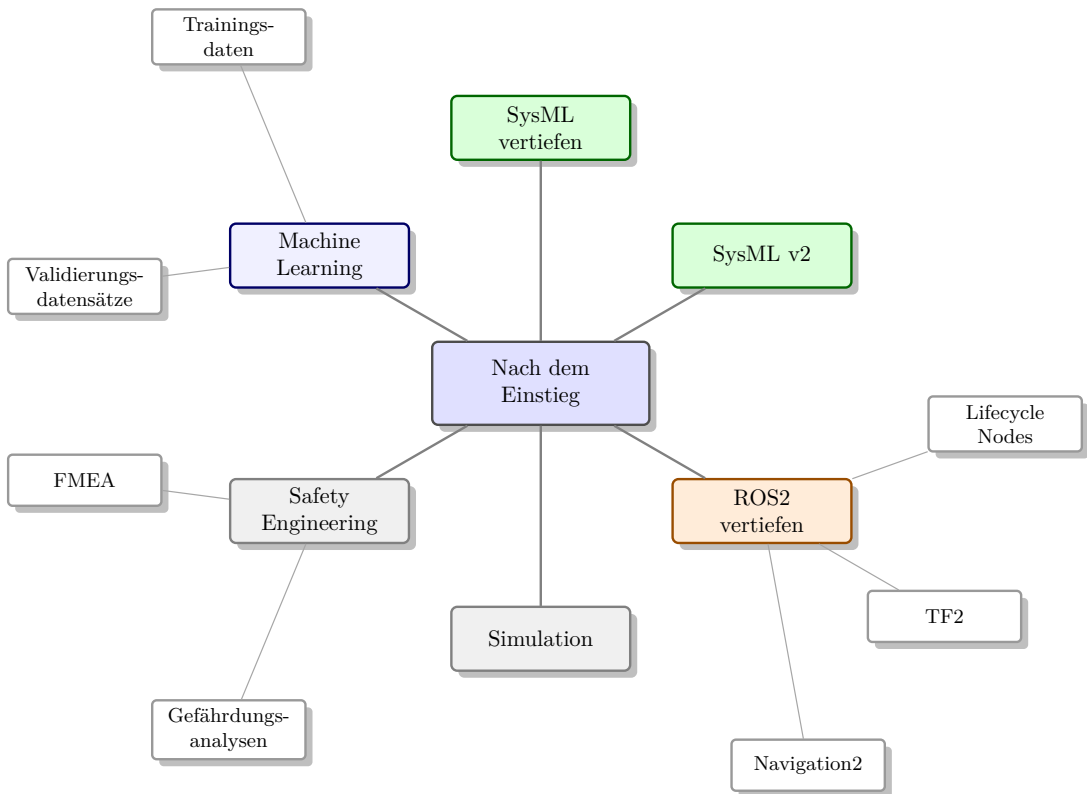


Figure 22.1: Mögliche Vertiefungsrichtungen nach dem Einstieg, gegliedert in sechs Themenfelder mit ausgewählten Unterpunkten.

22.2 SysML vertiefen

Nach dem Einstieg lohnt es sich, SysML genauer zu lernen. Besonders relevant sind:

- saubere Modellorganisation
- Wiederverwendung von Modellelementen
- Variantenmodellierung
- Schnittstellenkonzepte
- formale Anforderungen
- Parametrik und technische Analysen
- Modellreview und Modellqualität

22.3 SysML v2

SysML v2 modernisiert die Sprache und bringt eine präzisere textuelle und grafische Modellierung. Für Einsteiger ist SysML v1.x weiterhin verbreitet, aber SysML v2 wird langfristig wichtiger.

Wichtige Themen für später sind:

- textuelle Modellierung
- präzisere Semantik
- bessere Werkzeugunterstützung
- Migration bestehender Modelle
- Verbindung zu modernen MBSE-Methoden

22.4 ROS2 vertiefen

Für Robotik sind folgende ROS2-Themen wichtige nächste Schritte:

- Navigation2 (Navigationsstack für Pfadplanung und Hindernisvermeidung)
- TF2 (Bibliothek zur Verwaltung von Koordinatentransformationen)
- Lifecycle Nodes (Nodes mit definierten, verwalteten Zustandsübergängen)
- Launch-Dateien
- Parameter
- rosbag (Werkzeug zum Aufzeichnen und Abspielen von Nachrichten)
- Simulation
- Tests mit launch testing

22.5 Simulation

Gazebo oder andere Simulatoren ermöglichen Tests ohne echte Hardware. Das ist besonders wertvoll, wenn Algorithmen schnell ausprobiert werden sollen. Simulation kann helfen, Navigation, Sensorik, Hindernisse und Fehlerfälle früh zu prüfen.

22.6 Safety Engineering

Sobald Roboter mit Menschen in realen Umgebungen interagieren, werden Sicherheitsfragen wichtig. Dann reichen einfache Funktionsmodelle nicht mehr aus. Risikoanalysen, Sicherheitsanforderungen, FMEA (Failure Mode and Effects Analysis – systematische Fehlermöglichkeits- und Einflussanalyse), Gefährdungsanalysen und Normen müssen berücksichtigt werden. Für einen Serviceroboter wie den Indoor-Roboter ist beispielsweise die Norm ISO 13482 für persönliche Assistenzroboter relevant.

22.7 Machine Learning vertiefen

ML-Integration (Machine Learning) endet nicht beim Laden eines Modells. Wichtige Vertiefungen sind:

- Trainingsdaten
- Validierungsdatensätze
- Fehlklassifikationen
- Laufzeitoptimierung
- Edge-AI-Hardware
- Monitoring im Betrieb
- Umgang mit Unsicherheit

22.8 Berufliche Relevanz

Die Kombination aus Systems Engineering, SysML, ROS2 und KI ist in Robotik, Automotive, Industrieautomation und Forschung relevant. Wer diese Verbindung versteht, kann zwischen Architektur, Software und Tests vermitteln. Genau diese Schnittstellenkompetenz ist in vielen Projekten wertvoll.

22.9 Mögliche nächste Projekte

- SysML-v2-Modellierungsworkshop mit vollständigem Modellaufbau
- kleines ROS2-Workspace mit simulierten Nodes
- Navigation2-Demo in Simulation
- Object-Detection-Node mit Beispielmodell
- SysML-v2-Vergleichskapitel
- vollständige Traceability von Anforderungen bis Testcode

22.10 Vom Lernprojekt zum echten Projekt

Der wichtigste Übergang nach diesem Buch ist der Schritt von einem didaktischen Modell zu einem echten Projekt. In einem echten Projekt werden Anforderungen widersprüchlicher, Schnittstellen technischer und Tests aufwendiger. Außerdem kommen organisatorische Themen hinzu: Versionierung, Reviews, Verantwortlichkeiten, Änderungsmanagement und Abnahme. Abbildung ?? stellt diesen Übergang gegenüber.

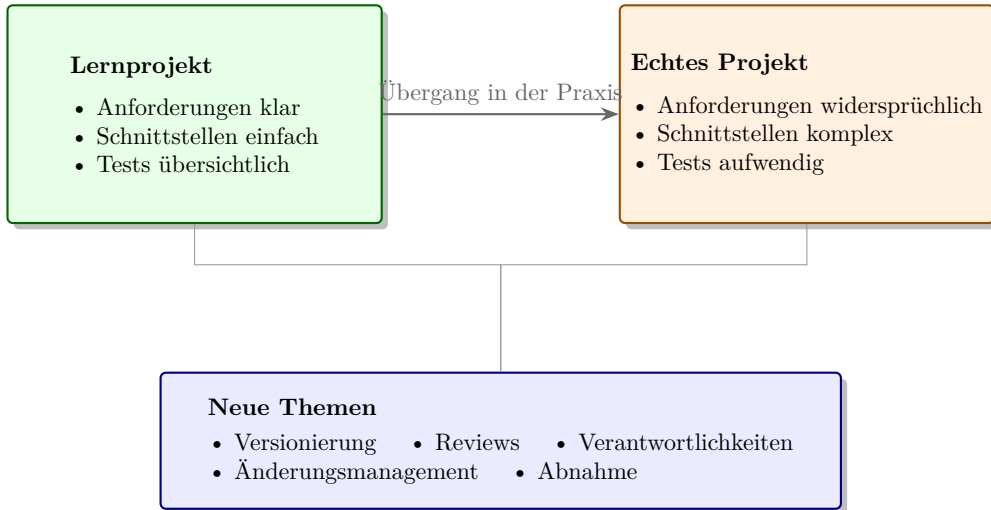


Figure 22.2: Der Übergang vom Lernprojekt zum echten Projekt bringt komplexere Anforderungen und Schnittstellen sowie neue, organisatorische Themen mit sich.

Das Lernprojekt liefert dafür eine Grundlage. Wer verstanden hat, wie eine Anforderung zur Architektur und weiter zu ROS2-Nodes und Tests führt, kann dieses Denken auf größere Systeme übertragen. Der Maßstab wächst, aber das Prinzip bleibt gleich: Problem verstehen, Modell aufbauen, Schnittstellen klären, Umsetzung ableiten und Nachweise planen.

22.11 Zusammenfassung

Der Einstieg ist geschafft, wenn Sie Anforderungen, Struktur, Verhalten, Schnittstellen, ROS2-Architektur und Tests zusammen denken können. Der nächste Schritt besteht darin, diese Konzepte in Werkzeugen und Code zu üben. MBSE wird erst durch Anwendung lebendig.

22.12 Kontrollfragen

1. Welche SysML-Themen eignen sich zur Vertiefung?
2. Warum wird SysML v2 wichtig?

3. Welche ROS2-Themen sollten nach dem Einstieg gelernt werden?
4. Warum ist Simulation für Robotik wertvoll?
5. Warum ist Safety Engineering bei Robotern wichtig?
6. Welche berufliche Rolle kann die Verbindung aus SysML und ROS2 stärken?

22.13 Übungen

Übung

Übung 1: Persönlicher Lernplan

Definieren Sie Ihren persönlichen nächsten Lernschritt: SysML vertiefen, ROS2 implementieren, Simulation aufbauen oder ML-Integration ausprobieren.

Übung

Übung 2: Projektidee

Beschreiben Sie ein eigenes kleines Robotik- oder Automatisierungsprojekt, auf das Sie die Methoden dieses Buches übertragen könnten.

22.14 Literaturhinweise

Zur Vertiefung eignen sich die SysML-v2-Spezifikation, aktuelle Werkzeugdokumentationen, das SysML-v2-Pilotprojekt und die ROS2-Dokumentation [?, ?, ?]. Klassische SysML-Literatur [?, ?] bleibt als Hintergrund nützlich, muss aber bewusst auf SysML v2 übertragen werden.

Abkürzungsverzeichnis

API	Application Programming Interface.	181
BDD	Block Definition Diagram (SysML v1).	181
DDS	Data Distribution Service.	181
FMEA	Failure Mode and Effects Analysis.	181
GPU	Graphics Processing Unit.	181
IBD	Internal Block Diagram (SysML v1).	181
KI	Künstliche Intelligenz.	6 , 181
MBSE	Model-Based Systems Engineering.	21 , 39 , 181
ML	Machine Learning.	181
ONNX	Open Neural Network Exchange.	150 , 152 , 181
QoS	Quality of Service.	125 , 139 , 181
REQ	Requirement.	181
ROS2	Robot Operating System 2.	125 , 181
SLAM	Simultaneous Localization and Mapping.	181
SysML	Systems Modeling Language.	42 , 181
TC	Test Case.	181

Glossar

Action In SysML v2 ein Verhaltenselement innerhalb eines Ablaufs; nicht zu verwechseln mit einer ROS2-Action als Kommunikationsmuster. [181](#)

Aktor Technisches Element, das Befehle in eine physische Wirkung umsetzt, zum Beispiel ein Motor, ein Bremsmechanismus oder ein Greifer. [181](#)

Anforderung Beschreibung einer Fähigkeit, Eigenschaft oder Einschränkung, die ein System erfüllen muss. [181](#)

Architektur Grundlegende Struktur eines Systems: seine wichtigsten Elemente, Verantwortlichkeiten, Schnittstellen und Beziehungen. [181](#)

Block Strukturelement in SysML v1. In SysML v2 ersetzt durch **part def** (Typdefinition) und **part** (Instanz). [181](#)

calc def SysML-v2-Definition für eine Berechnung, die Eingangsgrößen, Rechenlogik und Ergebnisgrößen strukturiert beschreibt. [181](#)

Connection Verbindung zwischen Ports oder Systemelementen, die zeigt, dass ein Austausch oder eine Kopplung zwischen ihnen besteht. [181](#)

Constraint Einschränkung oder Regel, die im Modell gelten muss, zum Beispiel eine maximale Bremsdistanz oder eine zulässige Latenz. [181](#)

cyber-physisches System System, in dem physische Komponenten durch Software gesteuert werden und über Sensoren, Aktoren oder Netzwerke eng mit dieser Software verbunden sind. [6](#), [181](#)

Datenfluss Gerichteter Austausch von Informationen zwischen Systemteilen, zum Beispiel Sensordaten von einer Kamera zu einem Auswertungs-Node. [181](#)

Funktion Fähigkeit oder Aufgabe, die ein System erfüllen soll, unabhängig davon, welche konkrete Komponente sie später realisiert. [181](#)

Guard Bedingung an einer Verzweigung oder Transition, die erfüllt sein muss, damit ein bestimmter Pfad genommen wird. [181](#)

Impact Analysis Analyse, welche Anforderungen, Komponenten, Schnittstellen, Tests oder Dokumente von einer Änderung betroffen sind. [181](#)

- Inferenz** Anwendung eines bereits trainierten Machine-Learning-Modells auf neue Daten, zum Beispiel auf ein aktuelles Kamerabild. [181](#)
- Komponente** Abgrenzbarer Teil eines Systems, zum Beispiel ein Sensor, ein Softwarebaustein, ein Teilsystem oder ein mechanisches Bauteil. [181](#)
- konjugierter Port** Port, dessen Flussrichtung gegenüber der ursprünglichen Portdefinition umgekehrt ist (Kennzeichen: Tilde ~); ein sendender Port wird dadurch zu einem empfangenden Port. [93](#), [181](#)
- Latenz** Zeitverzögerung zwischen einem Ereignis, einer Messung oder einer Eingabe und der daraus folgenden Systemreaktion. [181](#)
- Launch-Datei** Datei, mit der mehrere ROS2-Nodes gemeinsam mit Parametern und Startoptionen gestartet werden. [181](#)
- Lidar** Sensorprinzip zur Abstandsmessung mit Laserlicht; in der Robotik häufig zur Hinderniserkennung und Kartierung eingesetzt. [181](#)
- Lifecycle Node** ROS2-Node mit definierten Zuständen wie `unconfigured`, `inactive`, `active` und `finalized`. [181](#)
- Mapping** Zuordnung zwischen Elementen unterschiedlicher Ebenen, zum Beispiel von SysML-Elementen zu ROS2-Nodes, Topics und Tests. [181](#)
- Message** Struktur einer Nachricht in ROS2, die festlegt, welche Daten über ein Topic, einen Service oder eine Action übertragen werden. [181](#)
- Middleware** Software-Schicht, die Kommunikation zwischen verteilten Prozessen (z. B. ROS2-Nodes) vermittelt, ohne dass diese die technischen Transportdetails der Gegenseite kennen müssen. [125](#), [181](#)
- Modell** Vereinfachte, zweckgebundene Beschreibung eines Systems, die relevante Informationen, Beziehungen und Entscheidungen sichtbar macht. [181](#)
- Package** Organisierte Einheit in ROS2, die Code, Konfiguration, Schnittstellendefinitionen und Metadaten zusammenfasst. [181](#)
- Parameter** Konfigurationswert eines Nodes oder Modells, zum Beispiel ein Akkuswellwert, eine Maximalgeschwindigkeit oder ein Topic-Name. [181](#)
- part** Benannte Instanz eines mit `part def` definierten Typs innerhalb eines übergeordneten Systemelements. [181](#)
- part def** Typdefinition in SysML v2. Entspricht dem Block aus SysML v1. Definiert, welche Eigenschaften und Teile ein Systemelement besitzen kann. [181](#)

- Port** Schnittstellenpunkt eines SysML-Elements, über den Flüsse oder Kommunikationsbeziehungen modelliert werden. [181](#)
- Publisher** ROS2-Rolle eines Nodes, der Nachrichten auf einem Topic veröffentlicht. [181](#)
- ROS2 Node** Eigenständiger Prozess in ROS2, auch ROS2-Node geschrieben, der eine klar abgegrenzte Aufgabe übernimmt, Daten verarbeitet und über Topics, Services oder Actions mit anderen Nodes kommuniziert. [181](#)
- ROS2-Action** ROS2-Kommunikationsmuster für länger laufende Aufgaben mit Ziel, Feedback, Ergebnis und möglicher Abbruchmöglichkeit. [181](#)
- Schnittstelle** Definierter Übergang zwischen Systemteilen oder zwischen System und Umgebung, über den Daten, Energie, Material oder Befehle ausgetauscht werden. [181](#)
- Sensor** Technisches Element, das eine physikalische Größe oder Umgebungseigenschaft erfasst und als Signal oder Datenwert bereitstellt. [181](#)
- Sensorfusion** Kombination mehrerer Sensordaten, um eine robustere oder vollständigere Einschätzung der Umgebung zu erhalten. [181](#)
- Service** ROS2-Kommunikationsmuster für kurze Anfrage-Antwort-Vorgänge. [181](#)
- Sicht** Ausschnitt aus einem Modell, der bestimmte Informationen für eine Rolle oder Fragestellung gezielt darstellt. [181](#)
- Simulation** Nachbildung eines Systems oder seiner Umgebung, um Verhalten, Algorithmen oder Tests ohne vollständige reale Hardware zu untersuchen. [181](#)
- Stakeholder** Person, Gruppe oder Organisation, die ein Interesse am System hat, Anforderungen stellt oder von den Systemwirkungen betroffen ist. [181](#)
- State** Zustand eines Systems oder Systemteils, in dem bestimmte Regeln, Ereignisse oder Übergänge gelten. [181](#)
- Subscriber** ROS2-Rolle eines Nodes, der Nachrichten von einem Topic empfängt. [181](#)
- System** Zusammenwirken mehrerer Elemente mit einem gemeinsamen Zweck, einer Systemgrenze und Beziehungen zur Umgebung. [181](#)
- Systemgrenze** Festlegung, welche Elemente zum betrachteten System gehören und welche Elemente zur Umgebung oder zu externen Systemen zählen. [181](#)
- Systemkontext** Umgebung eines Systems mit externen Akteuren, Nachbarsystemen, Randbedingungen und wichtigen Wechselwirkungen. [181](#)

Testfall Beschriebene Prüfung mit Ziel, Vorgehen und erwartbarem Ergebnis, mit der eine Anforderung oder Funktion nachgewiesen wird. [181](#)

Topic Nachrichtenkanal in ROS2 für publish-subscribe-Kommunikation. [181](#)

Traceability Nachvollziehbare Verbindung zwischen Anforderungen, Architektur, Implementierung und Tests im Systemmodell. [161](#), [181](#)

Transition Übergang von einem Zustand in einen anderen, meist ausgelöst durch ein Ereignis oder eine Bedingung. [181](#)

Use Case Beschreibung eines Ziels oder Ablaufs aus Sicht eines Nutzers oder externen Akteurs, zum Beispiel ?Ziel anfahren?. [181](#)

Validierung Prüfung, ob das entwickelte System das tatsächliche Problem der Stakeholder löst und für den vorgesehenen Einsatz geeignet ist. [181](#)

Verifikation Prüfung, ob ein System oder Systemteil die spezifizierten Anforderungen korrekt erfüllt. [181](#)

Workspace Arbeitsverzeichnis für ein ROS2-Projekt, in dem Quellcode, Pakete, Build-Ergebnisse und Installationsartefakte organisiert werden. [181](#)

Bibliography

- [1] Lenny Delligatti. *SysML Distilled: A Brief Guide to the Systems Modeling Language*. Addison-Wesley, 2014.
- [2] Sanford Friedenthal, Alan Moore, and Rick Steiner. *A Practical Guide to SysML*. Morgan Kaufmann, 2014.
- [3] Ros 2 documentation. <https://docs.ros.org/>. Zugriff abhängig vom verwendeten ROS2-Release.
- [4] Syside editor. <https://sensmetry.com/syside/>. Kostenlose SysML-v2-Erweiterung für VS Code.
- [5] Sysml v2 pilot implementation. <https://github.com/Systems-Modeling/SysML-v2-Pilot-Implementation>. Referenzimplementierung für SysML-v2-Textnotation und Visualisierung.
- [6] Omg systems modeling language version 2.0 specification. <https://www.omg.org/spec/SysML>. Aktuelle Spezifikation der Object Management Group.
- [7] Eclipse syson. <https://projects.eclipse.org/projects/modeling.syson>. Open-Source-Projekt für webbasierte grafische SysML-v2-Modellierung.

List of Figures

1.1	Wirkkette der Hinderniserkennung von der Sensorik bis zu den Motoren, darunter der Regelkreis der Energieversorgung.	7
1.2	Stakeholder des autonomen Indoor-Roboters mit jeweils einem typischen Interesse am System.	11
1.3	Systemgrenze und Kontext des autonomen Indoor-Roboters mit Energie-, Daten- und Bewegungsflüssen über die Grenze hinweg.	13
2.1	Dokumentenzentrierte Entwicklung mit vielen lose verknüpften Dokumenten im Vergleich zu einem zentralen, klar verknüpften Systemmodell.	25
2.2	Farbkodierte Modellkette von einem Stakeholderbedarf über Anforderung, Funktionen und Systemelemente bis zu ROS2-Node und Testfall.	30
2.3	Vier Rollen betrachten dasselbe MBSE-Modell des Roboters aus unterschiedlichen Blickwinkeln.	36
3.1	Die sieben für dieses Buch wichtigsten SysML-v2-Konzepte rund um das gemeinsame Modell.	44
3.2	Struktursicht von <code>Robot</code> mit seinen Subsystemen und Detailsicht des <code>PerceptionSystem</code> mit Port <code>sensorData</code>	47
4.1	Empfohlene Werkzeugkette: VS Code mit Syside Editor und Git als Kern, optional ergänzt um Eclipse SysON und das SysML-v2-Pilotprojekt; parallel dazu der ROS2-Code.	52
4.2	Projektstruktur <code>AutonomousIndoorRobot</code> mit den Ordnern <code>model/</code> , <code>code/</code> und <code>docs/</code> sowie den wichtigsten Dateien.	53
4.3	Strukturbaum des Roboters mit <code>part def Robot</code> und seinen Subsystemen. Grün hinterlegte Kästen sind im Modell bereits weiter zerlegt, grau hinterlegte Kästen sind an dieser Stelle noch leere <code>part defs</code>	57
4.4	Der SysML-v2-Text im Syside Editor (links) ist die Quelle der Wahrheit; Eclipse SysON erzeugt daraus automatisch die grafische Strukturansicht (rechts), ohne dass diese von Hand gezeichnet werden muss.	62
5.1	Kontextdiagramm des Indoor-Roboters mit den wichtigsten externen Elementen seiner Betriebsumgebung.	68
5.2	Erste, noch grobe Systemstruktur des Roboters mit sieben gleichrangigen Teilsystemen.	70
6.1	Traceability-Kette von REQ-002 über <code>satisfy</code> und <code>verify</code> zum <code>PerceptionSystem</code> und per <code>trace</code> weiter zum ROS2-Node.	76

7.1	Use-Case-Diagramm des Indoor-Roboters mit den Akteuren Nutzer, Sicherheitsverantwortliche und Wartungspersonal.	83
8.1	Vollständige Dekomposition von Robot in sieben Hauptsysteme und deren wichtigste Teile der zweiten Ebene.	88
9.1	Verbindungsmodell des PerceptionSystem . Die kleinen Quadrate an den Boxrändern sind Ports. Das Tilde-Zeichen markiert einen konjugierten Port, der empfängt statt sendet.	94
9.2	Verbindungsmodell des NavigationSystem : Lokalisierung und Karte liefern an den Pfadplaner, dessen Pfad an die Bewegungssteuerung weitergegeben wird.	95
10.1	Schnittstellenübersicht des Indoor-Roboters: Sensordaten (orange) fließen über ObjectDetector und SLAM zur Navigation, Steuer- und Statusnachrichten (blau) verbinden Navigation, MotionControl und die Bedienoberfläche. .	100
10.2	Links die Hierarchie der TF-Frames vom kartenfesten map über odom und base_link bis zu camera_link ; rechts dieselben Frames als physische Koordinatenursprünge im Gebäudegrundriss und am Roboter.	102
11.1	Aktivitätsdiagramm für die Zielnavigation: vom Systemstart über Pfadplanung und Hindernisbehandlung bis zur Meldung des erreichten Ziels. Bei niedrigem Akkustand wird auf den Ablauf ?Rückkehr zur Ladestation? verwiesen.	108
11.2	Aktivitätsdiagramm für die Rückkehr zur Ladestation: von der Akkuüberwachung über die Bewertung des laufenden Auftrags bis zum Andocken und Start des Ladevorgangs, inklusive der Alternativen bei blockierter Ladestation oder fehlgeschlagenem Andocken.	111
12.1	Zustandsdiagramm zu state def RobotOperation . Jeder Pfeil entspricht einer transition accept ... then ... -Zeile aus dem Listing oben. Der Fehlerzustand ist rot hervorgehoben.	115
13.1	Bremsweg $s = v^2/(2a)$ in Abhängigkeit von der Geschwindigkeit v (hier mit $a = 1 \text{ m/s}^2$). Der Bremsweg wächst quadratisch; bei $v = 0,5 \text{ m/s}$ und $v = 1,0 \text{ m/s}$ sind die Werte markiert.	120
13.2	Parametrisches Diagramm zum Bremsweg: Geschwindigkeit und Bremsverzögerung fließen in BrakingDistanceCalc ein, das Ergebnis wird von der Bedingung SafeBrakingDistance gegen den maximal zulässigen Bremsweg geprüft. .	121
13.3	Parametrisches Diagramm zur Berechnung der Betriebszeit: OperatingTimeCalc verbindet die Batteriekapazität und die Leistungsaufnahme der Verbraucher zur geschätzten Laufzeit der Mission.	122
14.1	Publish-Subscribe-Prinzip: Ein Publisher-Node sendet Nachrichten auf einem Topic, mehrere Subscriber-Nodes können sie gleichzeitig empfangen. .	126

15.1	Node-Graph des autonomen Indoor-Roboters mit den wichtigsten Topics zwischen Sensor-, Verarbeitungs-, Navigations- und Aktuatorik-Nodes. .	131
15.2	Zustandsautomat eines ROS2-Lifecycle-Nodes von Unconfigured über Inactive und Active bis zum Endzustand Finalized	133
16.1	Grafische Darstellung eines einfachen Orientierungsmappings zwischen SysML-v2-Part-Definitions und zugehörigen ROS2-Nodes. In realen Systemen muss dieses Mapping projektspezifisch geprüft und verfeinert werden.	138
16.2	Traceability-Kette vom Requirement REQ-006 über SysML- und ROS2-Elemente bis zum Testfall TC-006.	141
17.1	Verzeichnisstruktur eines ROS2-Workspace mit dem Package robot_basic_nodes sowie den Geschwisterordnern build/ , install/ und log/	145
17.2	Ablauf im StatusNode : Der Timer löst periodisch publish_status() aus, das eine Nachricht erzeugt und auf dem Topic /robot/status veröffentlicht.	146
18.1	Datenkette der ML-Inferenz im object_detector_node vom Kamerabild bis zur Veröffentlichung der Erkennungsergebnisse.	151
18.2	Der ML-basierte object_detector_node liefert eine unsichere Zusatzinformation, während die Lidar-basierte Hinderniserkennung die zuverlässigere Sicherheitsgrundlage bleibt.	152
19.1	Das V-Modell verbindet den absteigenden Spezifikationsast mit dem aufsteigenden Verifikationsast.	156
19.2	Der Testfall TC-006 verifiziert die Anforderung REQ-006 über eine verify -Beziehung.	158
20.1	Traceability-Kette von REQ-002 über SysML-Systemelemente und ROS2-Node/Topic bis zum Testfall TC-002.	162
20.2	Auszug aus einer Traceability-Matrix: Jede Zeile verbindet eine Anforderung über Funktion, Systemelement und ROS2-Node bis zum Testfall.	163
20.3	Ablauf des Änderungsmanagements von der Beschreibung des Änderungswunschs bis zur dokumentierten Entscheidung.	165
21.1	Checkliste mit den zwölf Inhalten, die das Abschlussmodell enthalten sollte.	169
21.2	Zehn Schritte des Abschlussprojekts, gruppiert in vier Phasen von den Anforderungen bis zur Verifikation.	171
22.1	Mögliche Vertiefungsrichtungen nach dem Einstieg, gegliedert in sechs Themenfelder mit ausgewählten Unterpunkten.	176
22.2	Der Übergang vom Lernprojekt zum echten Projekt bringt komplexere Anforderungen und Schnittstellen sowie neue, organisatorische Themen mit sich.	179